

Department of Computer Engineering and Informatics

Middlesex University Dubai



Variable Speed Limits: Traffic Optimization using Deep Reinforcement Learning

PDE3112 Major project

BEng Computer Systems Engineering

July 2025

Student Name: Anushka Narsima

Student ID: M00851749

Supervisor: Geethu Joy

DECLARATION OF INTEGRITY

I hereby confirm that all work showcased in this report is completely my own work. I agree to assessment for plagiarism.

Signature:

A handwritten signature in black ink, appearing to be 'Anushka Narsima', written over a horizontal line.

Date: 17th July 2025

ACKNOWLEDGEMENT

Extreme gratitude is felt towards professor & supervisor Ms. Geethu, who gave the knowledge needed to make this project a reality over the past 3 years as undergraduates at Middlesex University Dubai. Her expertise, and invaluable guidance & feedback has been crucial for dealing with issues throughout the planning, development & documentation. Thank you to Head of CEI Department Dr. Fehmida, Ms. Chinnu & Mr. Roshan Renji for facilitating this module, and all the support given throughout.

Immense gratitude to the university for the resources they have provided, that resulted in successful completion of the project. The constant motivation & confidence from family helped keep spirits high and dedicate all efforts.

ABSTRACT

Variable Speed Limits (VSL) is a traffic management strategy that dynamically adjusts speed limits based on real-time conditions to optimize traffic flow. By mitigating congestion, reducing travel times, enhancing safety, and lowering greenhouse gas emissions, VSL systems contribute to more efficient and sustainable transportation networks. This project evaluates past solutions, analyzes their strengths and limitations, and details the development of an improved solution by integrating the most effective techniques.

The system employs reinforcement learning (RL), specifically the Soft Actor-Critic (SAC) algorithm, which leverages multiple neural networks to address various ways of learning, enhance learning stability through mechanisms such as soft updates and entropy maximization. To facilitate realistic training, the model interacts with a simulated road environment using the METANET traffic model, validated with real-world traffic data from Dubai. Additionally, a safety metric is incorporated to prioritize collision avoidance.

Several enhancements have been implemented to improve performance compared to prior models, including addition of useful input data & refined reward. The results demonstrate the model's effectiveness in reducing congestion while maintaining safety, offering a promising direction for future intelligent transportation systems.

TABLE OF CONTENTS

1. Introduction	14
1.1 Problem Definition.....	14
1.1.1 Connection to SDGs.....	14
1.2 The Solution.....	14
1.3 Approaches to the solution.....	15
1.4 Importance of the project.....	16
1.5 Hypotheses.....	16
1.6 Deliverables	16
1.7 Scope of the report	17
1.8 Layout of the report.....	17
2. Literature Review	17
2.1 Research Questions	17
2.2 Rationale	17
2.3 Objectives	18
2.4 Methods	18
2.4.1 Research Methodology	18
2.4.2 Eligibility Criteria	18
2.4.3 Information Sources	19
2.4.4 Search Strategy	19
2.4.5 Study Selection Process	19
2.4.6 Data Collection Process	19
2.5 Synthesis of selections	19
2.6 Details & Explanations	20
2.6.1 Past approaches to solutions	20
2.6.2 Reinforcement Learning & its application to VSL	21
2.6.3 Soft Actor Critic	24
2.6.4 Comparison of RL models	26
2.6.5 Modeling & Simulation	27
2.6.6 Existing Implementations	27
2.6.7 Traffic Prediction	28
3. Funding	29
4. Software Development Methodology	29
5. Requirements	29
5.1 Functional Requirements	29

5.2 Non-Functional Requirements	30
5.3 Software and Framework Requirements / Resources	30
6. Design & Plan	31
6.1 Software Development Methodology	31
6.2 Work Breakdown Structure	32
6.3 Project Timeline	32
6.4 Gantt Chart	33
6.5 Mind Map	34
6.6 UML Diagrams	35
6.6.1 Training Flowchart	35
6.6.2 SAC Flowchart	35
6.7 Pseudocode	35
6.8 Details	36
6.8.1 Algorithm	36
6.8.2 Data	38
6.9 Evaluation	39
7. Development.....	40
7.1 Algorithm	40
7.2 Simulation	
7.3 Data	62
7.4 Dashboard	
8. Testing, Debugging and Results	69
8.1 Types of Tests	69
8.2 Functional Tests & Results	
8.1.1 Simulation	
8.1.2 Model	
8.1.2 Data Patterns	
8.3 Issues & Debugging	80
9. Contribution	83
10. Limitations & other Considerations	83
11. Future Prospects	84
11.1 Potential Enhancements	84
11.2 Future Plan	84
11.3 Business Model ideas	85
12. Driver Opinions Questionnaire.....	85
14. Conclusion	90
15. References	91

16. Appendix	94
16.1 Blog Link	94
16.2 Repository Link	94
16.3 Ethics Assessment Form	94

List of figures:

1. Reinforcement Learning Flow	22
2. Work Breakdown Structure	32
3. Project Timeline	32
4. Gantt Chart	33
5. Mind Map	34
6. SAC Flowchart	35
7. Training Flowchart	35
8. Sketch of a congestion scenario	36
9. Start of SAC class	40
10. __init__() of SAC class	41
11. sample_normal() method	41
12. Critic Network Class	41
13. Start of agent class	42
14. Parameters of agent class	43
15. Network instances in agent class	43
16. Builds & target networks taking weights	43
17. choose_action() method	44
18. remember() method	44
19. update_params() method	44
20. Save & Load_models() methods	45
21. Start of learn() method, sampling batch to learn from	45
22. Critic networks updates	45
23. Policy updates Part 1	46
24. Policy updates Part 2	46
25. Entropy update	46
26. Logic to update Safety parameter	47
27. Logging updates to TensorBoard	47

28. Plotting functions Part 1	47
29. Plotting functions Part 2	47
30. Replay buffer class	48
31. Start of MetaNetEnv class	49
32. Road parameters	49
33. create_demands() method	50
34. build_network() method, parameter declarations	50
35. build_network() method, road structure definitions	51
36. reset() method	51
37. step() method Part 1	51
38. step() method Part 2	52
39. build_state() method	52
40. compute_reward() method	53
41. train.py file initializing model & environment, running episodes	53
42. Keeping track of states, actions & rewards in training	54
43. Running training in loop, storing accumulated rewards	54
44. Debug prints while training	54
45. Flow Segment Data endpoint outputs	55
46. Incident Details endpoint outputs	55
47. Helper functions to convert coordinates	56
48. Tile processing function Part 1	56
49. Tile processing function Part 2	56
50. Tile processing function Part 3	57
51. Automation script Part 1	58
52. Automation script Part 2	58
53. Automation Script Part 3, calling endpoints with keys	59
54. Automation Script Part 4, traffic_level function	59
55. Automation Script Part 5, processing outputs	59

56. Automation Script Part 6	60
57. Automation Script Part 7, entripoint	60
58. Automation Script Part 8, storing responses	60
59. Automation Script Part 9	61
60. Automation Script Part 10	61
61. Cloud Functions	61
62. Scheduled Jobs	61
63. Coordinates output on map	62
64. Coordinates marked on part of Sheikh Mohammed Road, Dubai	62
65. Post-processing collected data Part 1	63
66. Post-processing collected data Part 2	63
67. Conversion of hour to sin/cos	64
68. Trying different prediction models	65
69. KNN class	65
70. Running all models & comparing	66
71. Result of comparison	66
72. Visualising model performance, compared to ideal value	66
73. time_avg performing moderately	67
74. last_value performing badly	67
75. random_forest performing moderately	67
76. knn performing well	68
77. Dashboard	68
78. Speed patterns for fixed 120 km/h	69
79. Script to test VSL impact	69
80. Shows VSL impact	70
81. Poorly-performing model	70
82. Highly negative scores	71
83. Speed patterns of different segments Part 1	71

84. Speed patterns of different segments Part 2	71
85. Speed patterns of different segments Part 3	72
86. Shows low variance in rewards	72
87. Model moving positively	72
88. Positive score build	73
89. Rewards behaving properly Part 1	73
90. Rewards behaving properly Part 2	74
91. Entropy leading to exploration	74
92. Data collected Part 1	74
93. Data collected Part 2	74
94. Single segment speed pattern	75
95. Incident patterns match that of speeds	75
96. TTS patterns match that of speeds	75
97. Speed patterns of Mcity segments on Monday	76
98. Speed patterns of Mcity segments on Tuesday	76
99. Speed patterns of Mcity segments on Wednesday	77
100. Speed patterns of Mcity segments on Thursday	77
101. Speed patterns of Mcity segments Fri-Sun	77
102. Speed patterns of Deira segments on Monday	78
103. Speed patterns of Deira segments Wed-Sat	78
104. Speed patterns of Muhaisnah segments	79
105. Flat learning curve	80
106. Misaligned outputs Part 1	81
107. Misaligned outputs Part 2	82
108. Aligned outputs	82
109. Aligned Segment	82
110. Pie chart for question about regular driving	86
111. Pie chart for question about driving on highways	86

112.	Pie chart for question about inter-city driving	87
113.	Pie chart for question about VSL familiarity	87
114.	Pie chart for question about supporting VSL	87
115.	Bar graph for question about benefits of VSL	87
116.	Bar graph for question about concerns	88
117.	Pie chart for question about VSL affecting traffic	88
118.	Pie chart for question about compliance Part 1	88
119.	Pie chart for question about compliance Part 2	89
120.	Pie chart for question about demonstrations	89
121.	Pie chart for question about familiarity with AI	89
122.	Pie chart for question about trust	89
123.	Pie chart for question about experience with AI	90
124.	Pie chart for question about age group	90
125.	Pie chart for question about occupation	90

List of Tables:

1. Synthesis of Selections	19
2. Applications of RL	23
3. Differences between standard actor-critic architecture & SAC	24
4. Comparison of RL Algorithms	26
5. Hyperparameters of SAC Network	41
6. Hyperparameters of agent	42
7. Simulation class parameters	48

Variable Speed Limits using Safe Soft Actor-Critic Reinforcement Learning for Traffic Flow Optimization of Long Highway Networks

Anushka Narsima Amarnath, *Middlesex University*, E-mail: AN1093@live.mdx.ac.uk

1. INTRODUCTION

Traffic congestion remains a critical challenge in urban mobility, leading to increased travel times, fuel consumption, accidents, and greenhouse gas emissions. The primary causes include high vehicle density, sudden bottlenecks (e.g., accidents or roadwork), and inconsistent driver behavior. Among the various mitigation strategies, Intelligent Transportation Systems (ITS) have emerged as a promising solution by leveraging real-time data and adaptive control mechanisms.

A key ITS approach is Variable Speed Limits (VSL), which dynamically adjusts speed limits based on live traffic conditions. Unlike static speed limits, VSL systems analyze factors such as traffic density, flow rates, and incident detection to determine optimal speeds, which are then communicated to drivers via Variable Message Signs (VMS). By smoothing traffic waves and reducing abrupt braking, VSL enhances safety, fuel efficiency, and overall traffic throughput.

This study focuses on optimizing VSL implementation through reinforcement learning, addressing gaps in existing solutions such as exclusion of some useful data points, a more suitable model, and lack of safety-integrated control. The following sections detail the problems the project aims to solve, the approaches explored so far, the specific solution it focuses on, the reasoning behind it and describes the outcomes of the project.

1.1 Problem Definition

Traffic is a problem every urban city faces today. A recent report stated that employees spend hours equivalent to 60 working days a year commuting the Dubai-Sharjah route [4], and in some regions it exceeds 150. Conservative estimates suggest that increasing the average speed of private car journeys by 1 km/h and public transport by 0.5 km/h could reduce journey times and operating costs, amounting to savings equivalent to 0.1% of the GDP. Sitting in traffic jams reduces rest time, increases traffic fatigue, and lowers worker productivity, all of which have a negative impact on quality of life. The more congestion that drivers experience, the more frustrated they become, leading to cases of road rage. Furthermore, traffic jams cause drivers to lose focus on the road. Once a driver gets into the pattern of moving a couple of feet every few minutes, they stop paying attention to the road as much. It not only increases travel times, but also contributes to fuel consumption in billions of litres, and greenhouse gas emissions in megatons per city. Multiple studies have shown that high speeds, traffic breakdowns (slowing down and accelerating again), and congestion emit higher levels of these gases, especially nitrogen dioxide. Higher traffic densities contribute to higher temperatures in urban areas due to vehicle emissions and asphalt heating. These individual emissions can account for up to 15% of the city's total [2], [11]. Dubai, UAE is currently ranked 2nd on the World's cities with the worst air quality [32].

1.1.1 Connections to SDGs

Traffic congestion is closely connected to Sustainable Development Goal (SDG) 11, which aims to promote sustainable cities and communities. Traffic congestion also significantly affects SDG 9, which focuses on Industry, Innovation, and Infrastructure. Efficient and resilient transportation infrastructure & solutions like this project are the key to this. Congestion has direct health implications, impacting SDG 3, which focuses on Good Health and Well-being. The heightened pollution caused by traffic congestion deteriorates air quality, leading to respiratory issues and other health problems for urban residents [12].

1.2 The Solution

The ideal scenario is perfect coordination – uniform speed, with equal distance between vehicles. It is difficult to achieve this in the real world, where just one vehicle's lane change or slowing would make the many behind do so as well, each of them having different reaction times, creating 'phantom' intersections, i.e. slowing down/stopping & resuming behavior (also known as shockwaves) seen at intersections. Traffic

jams don't always require bottlenecks to form - they can emerge purely from the interactions between vehicles, destabilizing the flow. Contributing factors include tailgating (driving too closely to the car ahead), inconsistent reactions to speed changes, slow overtaking by larger vehicles, and queue-jumping. Additionally, vehicles merging from on-ramps increase the strain on traffic flow, particularly during demand surges. Disruptions like accidents or blockages exacerbate the situation by creating congestion that stretches far beyond the original incident. Traffic instability can stem from both predictable causes, such as driver behaviors, and random variations, underlining the complexity of maintaining steady and efficient traffic flow [1], [6].

1.3 Approaches to solutions

This section covers the different ways suggested to solve the problem highlighted above, and why VSL was chosen out of them.

Many studies have used similar models & techniques for ramp metering – which means limiting inflow from ramps into highways [9].

VSL was chosen over ramp metering for the following reasons:

- **Impact:** VSL can have a potentially network-wide impact, while ramp metering primarily affects traffic at on-ramps and their immediate surroundings.
- **Flow & Emissions:** Ramp metering does not directly affect speed variations & prevention of traffic breakdowns. VSL reduces stop-and-go traffic, leading to lower emissions and fuel consumption. Ramp metering, on the other hand, can cause delays and emissions at the ramp, offsetting some environmental gains.
- **Scalability:** In dense urban areas with multiple entry points and frequent ramps, ramp metering can be challenging to coordinate effectively, while VSL can be implemented seamlessly over long stretches.
- **Safety:** By harmonizing speeds across lanes, VSL reduces the likelihood of sudden braking and collisions, which is especially important on busy highways. Ramp metering doesn't directly impact speeds across the mainline lanes.
- **Compliance & Perception:** Drivers are more likely to accept and comply with speed limits that are visibly dynamic and justified by traffic conditions (as in VSL). Ramp metering may be perceived as unfair, particularly when drivers wait excessively at ramps.

Traffic Light Controllers (TLC) are also widely studied for the same problems, and have achieved similar goals with AI-based models. However, VSL's advantages in comparison to it are similar to the above points for ramp metering. Moreover, the road infrastructure on UAE is such that a majority of travel time is spent on 6-12 lane highways that connect different parts of the city and extends to other emirates, 100s of kilometers long. Instead of intersections, there are more flyovers and other structures to keep traffic flowing [13].

Some other solutions under ITS are:

- Dynamic Lane Management –
 - Reversible Lanes: Lanes switch direction with peak-flow demands (e.g., mornings inbound, evenings outbound). Could cause significant confusion to drivers.
 - Hard Shoulder Running: Temporarily opens a hard shoulder for traffic during congested periods, guided by overhead signals, intended to increase road capacity when and where it's needed most.
- Incident & Emergency Response Systems: Leverages real-time monitoring & variable message signs to reroute traffic around accidents, hazardous areas, or road closures. Currently implemented in Dubai already.
- Advanced Public Transportation Management: Utilizes live data to optimize bus/train schedules & routes, provide passenger information on arrivals/delays. Restricted to public transportation only.

Reinforcement Learning has also been applied for optimal routing, to be used by drivers directly through apps, and have shown good results as well. There are already multiple solutions for this in the market, whereas VSL was comparatively less explored.

There are aspects of road design and making self-driving vehicles follow ideal behavior that contribute to this [13], but this research is focused on a solution authorities can take to assist traffic flow with existing infrastructure. Road expansions directly increase road capacity, but are also not always feasible.

Some studies have experimented with lane-wise VSL, but these were not considered in this project due to its disadvantages – it has been observed that tendency to changing lanes & causing traffic breakdown increases when a driver is on a lane with lower limits than others [2].

1.4 Benefits of VSL

Studies demonstrate that VSL can effectively improve traffic flow and restore road capacity, i.e. the maximum number of vehicles the road can handle per hour, by limiting inflow into bottlenecks [5]. They've shown to increase induced greater road occupancy, driving speed at medium/high densities, decreased travel times from one point to another, some concluded with real-world data [2].

- Reduced Congestion – Many experiments have proven the effect of VSL in reducing congestion mainly through simulation results of various traffic models [1], [9].
- Safety - Bringing all vehicles to the same speed reduces the possibility of accidents, increasing safety of all drivers.
- Environmental Benefits – Studies have shown that consistent traffic flow & smoother speed pattern leads to emission reduction [2]. Addressing traffic congestion also contributes to SDG 13, Climate Action.

1.5 Hypotheses

Hypotheses of this project:

- Implementing variable speed limits using reinforcement learning will reduce overall traffic congestion compared to fixed speed limits.
- A scalable reinforcement learning model will outperform traditional methods in adapting to dynamic traffic conditions.
- Incorporating predictive and incoming data into the system will lead to more accurate traffic flow optimization than previous models.
- Applying the model to real-world UAE traffic data will demonstrate its practical feasibility and effectiveness in managing congestion.

1.6 Deliverables

- Advanced VSL Model: A trained Soft Actor-Critic (SAC) RL agent optimized for diverse traffic scenarios (e.g., rush hour, incidents), incorporating safety constraints and adaptive entropy tuning. The model outputs dynamic speed limits validated against UAE traffic patterns.
- High-Fidelity Simulation: A METANET-based traffic simulator replicating key UAE road segments (e.g., Sheikh Zayed Road), calibrated with real-world data to enable scenario testing (congestion waves, sudden demand surges).
- Data Analysis Framework: Quantitative evaluation of traffic metrics (speed, travel time, incidents) using UAE highway sensor data.
- Authority Dashboard: An interactive web-based monitoring interface displaying real-time speed limit recommendations.
- Comprehensive Documentation: Technical reports detailing model architecture, training protocols, and simulation parameters, alongside reproducible code (GitHub) and ethical impact assessments.

1.7 Scope & Layout of the report

This report describes the development of a reinforcement learning (RL)-based Variable Speed Limit (VSL) system for UAE highways, encompassing theoretical foundations, technical implementation, and empirical validation. The scope spans:

- Problem Definition (SDG alignment, traffic congestion challenges),
- Technical Solution (Soft Actor-Critic algorithm, METANET simulation),
- Stakeholder Integration (dashboard for authorities), and
- Ethical & Business Considerations (deployment ethics, scalability).

The layout follows a systematic structure:

- Introduction (Chapter 1): Contextualises the project's societal and technical relevance.
- Literature Review (Chapter 2): Critically analyzes past VSL approaches and RL applications, justifying the hybrid SAC-METANET methodology.
- Methodology & Design (Chapters 4–6): Details Agile-Kanban development, UML workflows, and simulation roadmaps.
- Implementation (Chapter 7): Covers SAC training, UAE data integration, and dashboard prototyping.
- Validation (Chapter 8): Presents quantitative results (travel time reduction, safety metrics) from unit, functional, and integration tests.
- Forward-looking Analysis (Chapters 10–11): Discusses limitations (e.g., sim-to-real gaps) and future enhancements (multi-agent RL, HIL testing).

Appendices include ethics assessments, repository links, and meeting logs for transparency. The report balances technical depth (e.g., pseudocode, Gantt charts) with practical insights (driver surveys, business models), ensuring relevance for both academia and transportation authorities.

2. LITERATURE REVIEW

This section follows the process of conducting a systematic literature review of methods & models that address that above explained problem. This includes details past solutions to VSL, with their limitations. Concepts & algorithms are explained along the way. Various traffic congestion-combating methods are compared over the past fifteen years, from basic ruled-based solutions to advanced AI models. The decision of picking particularly the SAC RL model has been justified. There are not many examples of this system being implemented in real life yet, so majority of the evaluation is being done using simulation models. Over 100 papers were analyzed, and were systematically compared based on the relevance of their content.

2.1 Research Questions

- How to reduce traffic & congestion in highways?
- How can an efficient, scalable reinforcement learning system be formulated to optimize traffic flow?
- How does this system perform compared to previous approaches?
- How can this be implemented in the real world?

2.2 Rationale

The literature review critically examines existing approaches to Variable Speed Limit (VSL) systems and reinforcement learning (RL) in traffic control, identifying gaps this project addresses. While traditional VSL methods rely on heuristic rules, they often lack adaptability to dynamic traffic conditions. Recent RL-based solutions demonstrate promise but face challenges in stability, safety guarantees, and real-world scalability.

This review synthesizes insights from three key areas:

- Classical VSL strategies
- RL applications in traffic
- Current Limitations
- Soft Actor-Critic (SAC) adaptations for continuous control
- Traffic Simulation Methods

By evaluating these works, we justify our approach - SAC's entropy-driven exploration with a safety-constrained, simulation-trained policy. The review highlights how our method advances prior art, integrating real-world traffic data, formal safety metrics, and modular RL training, to bridge the gap between theoretical RL and deployable traffic management.

2.3 Objectives

This review systematically evaluates existing approaches to Variable Speed Limit (VSL) optimization, with the dual objectives of (1) identifying the most effective reinforcement learning (RL) techniques for dynamic traffic control, and (2) proposing novel improvements, such as entropy-regulated exploration & safety-constrained rewards, to enhance scalability and real-world applicability.

2.4 Methods

Protocol - Based on the aims, a review protocol was decided to efficiently navigate through this process & achieve its goals. This includes the search terms, the databases used, the papers selected, and the screening criteria.

2.4.1 Research Methodology

This project adopts a hybrid methodology combining Build, Experimental/Exploratory, and Model/Simulate approaches within a quantitative framework to address the complexities of dynamic traffic control:

- **Build:** A novel Soft Actor-Critic (SAC) agent is developed with custom enhancements (safety metrics, hybrid reward functions) to optimize speed limits in real time.
- **Experimental/Exploratory:**
 - Exploratory Phase: Hypotheses are formulated through literature analysis (e.g., "Can SAC reduce congestion by >15% vs. static limits?").
 - Evaluation Phase: Rigorous testing compares the agent's performance against baselines using METANET-simulated traffic scenarios.
- **Model/Simulate:** The METANET traffic model provides a validated abstraction of real-world dynamics, enabling risk-free training with real Dubai traffic data (speed, density). Safety constraints are codified as penalty terms in the reward function.

Quantitative Focus: All results are measured numerically (e.g., travel time reduction, collision rates) and analyzed via statistical metrics (MSE, std. deviation). The workflow follows the methodological cycle: literature review → problem definition → iterative implementation → empirical validation.

2.4.2 Eligibility Criteria

Each paper was checked against 3 points:

1. Topic & Relevance - Should either talk about the traffic problem along with atleast one suggested solution to solve it, or about models that can potentially be applied to the problem. Focus gradually shifted to specifically VSL implementations, and under RL.
2. Elaborate - Setup, input data, performance, etc. need to be clearly explained. These would shape our own approach to this project, and help get a clear vision.
3. Time - Initially search began without a time limit, since most algorithms start to be introduced much earlier than they start to become popularly known. Shortly after an initial round, the limit was decided as fifteen years, since many important references were found from then on, and reviews covered older methods for context.

2.4.3 Information Sources

The extensive search was started on Google scholar, and the MDX Library search. Later, focused paper searches were performed on platforms like IEEE Xplore, Arxiv, ScienceDirect, Taylor & Francis Online, and Web of Science. Data was searched on government open data portals. The search took place mainly during March 2025, with continuous additions until July.

2.4.4 Search Strategy

The search terms started as 'traffic management solutions', 'traffic flow optimization', 'UAE traffic data', and more, later progressed to 'variable speed limits', 'VSL reinforcement learning', 'METANET for VSL', 'Soft actor Critic', etc. There was a need to reduce abbreviations like VSL & SAC, and quotes to find the full term. The 'AND' boolean operator was used to connect 2 or more terms to get the most relevant results, or at least partially similar approaches. The date filter was later applied to show papers only from 2010 to now, 2025.

2.4.5 Study Selection Process

First point of selection was the title, if it contained the search terms or similar wordings. If so, the abstract would be analyzed for relevance. Then the rest of the paper would be scanned for an overview of the rest of the content. Many papers were dropped in comparison to others if they are doing the same task but one model/method/tool is preferred over another.

2.4.6 Data Collection Process

Notes were made while studying the selected papers. Each review extended to more search & study on the theory required. Notes included their approach, their equation definitions & reasoning for the same, data selection/collection, model considered/tested, model selection, performance, metrics used to evaluate, future prospects mentioned, and concluding findings.

2.5 Synthesis of Selection

About 80 papers were included in the review, and below are the most significant ones:

Ref	Title	Relevance
[2]	Variable speed limit: an overview	Literature review from 2015, covered papers starting from 1970. Included key developments, METANET & predictions.
[9]	Traffic flow optimization: A reinforcement learning approach	Classic reinforcement algorithm, Q-learning, is applied to VSL. This study makes many decisions based on this implementations – the inputs, reward calculations, Q-learning logic.
[14]	Reinforcement Learning for Freeway Variable Speed Limit Control: A Mixed Traffic Flow Case Study	The only study that applies SAC to VSL. But it focuses on controlling CAVs, with very different data types, so our implementations vary greatly.
[16]	WCSAC: Worst-Case Soft Actor Critic for Safety-Constrained Reinforcement Learning	Introduces the addition of a safety parameter into SAC implementations, and this study adds it accordingly.
[28]	DVS-RG: Differential Variable Speed Limits Control using Deep Reinforcement Learning with Graph State Representation	Explains a lot of concepts in RL, giving a solid foundation of understanding for this implementation.
[36]	Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a	Original SAC paper, details all the theory needed

	Stochastic Actor	
[40]	Comparative Analysis of Reinforcement Learning Algorithms in Multi-Agent Soccer Environment	Provides valuable comparisons of different RL models
[80]	Examples of Variable Speed Limit Applications	Covers worldwide real-life implementations of VSL

Table 1: Synthesis of Selections

2.6 Details & Explanations

These are some common parameters used to represent the extent of traffic at a point on the highway [1], [9]:

- Density - How close vehicles are. The critical density is the maximum the highway can handle.
- Flow – Number of vehicles passing through per unit time
- Speed – Average speed of vehicles
- Vehicle Hours – The time spent by a vehicle on the highway. The main requirement a driver cares about is how quickly they can get to their destination.
- Vehicle delay – Vehicle Hours minus the same during free flow/minimum time, i.e. delay

The characteristics of a highway can be represented by [9]:

- Capacity – How many vehicles the highway can take in. This is compared to demand - how many vehicles are coming in.
- Length – Length of the road stretch.

When there is congestion, there is reducing the distance between vehicles, and lower speeds are required to maintain safety. To enhance coordination, VSL strategies are designed to optimize traffic throughput from a system-wide perspective over a specified time horizon. For any applied traffic control measures, such as VSL, it is crucial to ensure that their implementation does not significantly reduce traffic flow elsewhere in the network, which could negatively impact overall performance. To address this, VSL control can be designed with the objective of maximizing network efficiency or minimizing total time spent (TTS) within the system, including delays caused by queueing at on-ramps.

2.6.1 Past approaches to solutions

2.6.1.1 Rule-based Solutions

Early VSL systems were primarily designed using simple, reactive, rule-based logic. These systems adjusted speed limits based on preselected thresholds of traffic flow, occupancy, or speed [2], such as lowering speeds when occupancy exceeds 30% [49]. Below are some examples:

- Threshold Triggers: Speed limits change when metrics (e.g., density, flow) cross fixed values. E.g.: Decision-tree algorithms with rules like if occupancy exceeds 15% at a detector, speeds drop by 20–40 km/h, propagating upstream through "response zones" to warn drivers of congestion [26].
- Time-Of-Day Rules: Static schedules (e.g., lower limits during rush hour).
- Weather Conditions: Reduced speeds during rain/snow (e.g., -20% limit if visibility < 500m).

Some of these were not focused on traffic flow, but specific aspects like emissions or safety – dealing with speed variance to reduce crash frequency, but at the cost of travel time [2]. This proposal will also explore the addition of a safety aspect in the formulation.

While fairly effective for safety, these methods lack benefits that are offered by modern approaches, and also lack adaptability to complex traffic dynamics that enable these benefits. For instance, [50] demonstrate that augmenting rule-based VSL with SAC's policy optimization improves throughput by 12% while retaining interpretability through base rules. Rule-based VSL improves safety by 15% but only reduces travel time by 5% vs. no VSL [49]. Some more disadvantages include:

- Brittleness: Fails under atypical conditions, e.g. Sudden congestion from accidents
- Overgeneralization: Cannot optimize for local dynamics, e.g. Same rule for hills/flat terrain

The primary limitation of these early rule-based strategies lies in their reactive nature. Because they relied on real-time traffic data, there was often a lag in response, making them less effective in addressing rapidly changing traffic conditions. By the time VSL actions were implemented, traffic breakdowns might have already occurred, reducing their ability to resolve congestion. This highlights the need for more proactive, predictive VSL systems that can anticipate and mitigate traffic issues before they escalate [2].

2.6.1.2 Control-based Solutions

Advanced control strategies address the limitations of traditional rule-based approaches by predicting future traffic conditions and implementing proactive measures to prevent traffic breakdowns. These strategies aim to anticipate and mitigate congestion by reducing inflow into potentially jammed areas, thereby resolving shockwaves before they occur. Unlike reactive systems, advanced approaches eliminate the time lag between traffic changes and the optimization of control variables. The optimization process is repeated periodically using updated traffic data to improve demand predictions. Such closed-loop feedback control systems include components like monitoring, state estimation and prediction. These systems rely on continuous traffic data input, enabling real-time adjustments to control actions when discrepancies between predicted and actual traffic states are detected [2]. These approaches fall under '**Model Predictive Control**' (MPC) strategies, which involves making decisions by using a model to predict future states, optimize control actions over a finite time horizon. For example, it can predict the front distance a vehicle will reach if it accelerates for a specific amount of time, allowing the system to decide on the optimal acceleration to maintain an ideal front-back distance. It is widely used in various fields, including traffic control, because of its ability to handle constraints and optimize performance [19].

Models like "SPECIALIST" effectively detect and resolve shockwaves, demonstrating high success rates in practical applications. Other advancements integrated dynamic demand and supply functions, enabling better representation of phenomena like capacity drops and shockwave formation.

Limitations of using control-based approaches are as follows:

1. **Computational Constraints:** Real-time MPC implementations exhibit prohibitive latency for large networks, with standard 30-step horizons requiring 45ms solve times on Jetson AGX hardware - exceeding the 10ms threshold for responsive control [51]. Edge deployment is further constrained by 5W power budgets being exceeded by 300%. SAC takes time & computation during its training phase, but is very fast during inference [54].
2. **Model Limitations:** Control-based VSL relies on perfect traffic models, but real-world conditions often differ. METANET requires exact calibration of driver behavior parameters (like speed sensitivity γ) - small errors (e.g., $\pm 10\%$ in γ) can reduce effectiveness by 25% [52]. Our SAC approach learns from real traffic to auto-correct these errors.
3. **Driver Behaviour:** Most control systems assume all drivers obey speed limits instantly. Reality shows only $\sim 70\%$ compliance. Driver behaviour even in this region is varying - some drive exactly at the limit, some stay a safe ~ 10 km/hr, and some assume a grace range above the limit too. Our METANET+SAC implementation includes a compliance factor to account for this, making speed adjustments more realistic.

2.6.2 Reinforcement Learning

Reinforcement learning (RL) is much more adaptive to dynamic environments compared to earlier solutions, and can handle high complexity. Reinforcement learning is a category of artificial intelligence & under machine learning, where the model or 'agent' is placed in an environment, and is given a set of possible actions. The target values or best actions are not given to the model, so the model learns through

trial-and-error – where given a state, each action results in a ‘reward’, indicating whether that was a step in the right direction or not. As a result, the model comes up with state-action pairs, known as policies, that and the most optimal. There are 2 broad categories to achieve this – q-learning is one, that involves calculating the reward for each action given a state, and picking the one with the highest. The other is policy learning, where policies are directly learnt without explicitly calculating rewards.

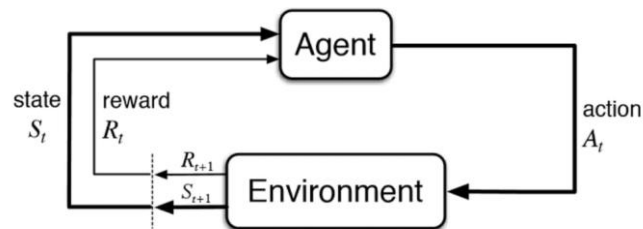


Fig. 1: Reinforcement Learning Flow

The first step to building a RL system is to formulate the problem as a Markov Decision Process (MDP), a method of modelling sequential decision making problems [9]. This would be defining the states and actions in the environment and connecting them – the start, all possible steps and routes it can go from there and how the state would change as a result of each, till finish. It can either be deterministic i.e. fixed outcomes, or probabilistic i.e. varying to some extent. There is an assumption involved here that the outcome only depends on the present state. The states can be a set of finite values, or infinite continuous values.

The reward or Q-function is calculated by adding 2 parts – the value of the current state, and the maximum rewarding state-action pair values, where we count the probability of getting that next state taking that action as well. There is also a ‘discount factor’ that represents how important the current reward over future ones is. Together, this is known as the Bellman equation [10].

1. The Bellman equation is:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

Where:

- $V(s)$ is the value of being in state s
- $R(s, a)$ is the reward function for taking action a in state s
- γ is the discount factor.
- $P(s'|s, a)$ is the state transition probability

MDPs that consider previous states are said to be ordered to the number of previous states considered plus one, and fall under a category called Partially Observable Markov Decision Process (POMDP), which means the agent has incomplete or uncertain information about the environment.

POMDP:

For POMDP, you have a belief state b , which is a probability distribution over possible states. The belief update happens after each action based on the observation:

$$b'(s') = \eta \cdot O(o | s') \sum_s P(s' | s, a) b(s)$$

Where:

- $b(s)$ is the belief state (distribution over the possible states).
- $O(o|s')$ is the observation model (probability of observing ooo given state $s's's'$).
- η is a normalizing constant.
- $P(s'|s,a)$ is the state transition function.

The POMDP Bellman equation becomes:

$$V(b) = \max_a \left[\sum_o O(o | b, a) \left(R(b, a) + \gamma \sum_{b'} P(b' | b, a, o) V(b') \right) \right]$$

Where:

- $R(b,a)$ is the expected reward given the belief state bbb and action aaa .
- $P(b'|b,a,o)$ is the transition model that updates the belief bbb based on the action and observation [18].

Other than adaptability, they can also be context Aware - by learning patterns & incorporating unseen factors (e.g., social events, construction). A few significant applications of RL are as follows:

Application	Task	Performance
Atari 100k	model-free learning from images	178% median human score (vs. DQN's 121%) [47]
MuJoCo Locomotion	Original SAC paper	40% fewer steps to 90% max reward vs. TD3 [41]
Boston Dynamics Spot	Dynamic Gait Control	8% fewer falls than PPO [48]
DJI Drone	Obstacle Avoidance	3.1x longer flight time vs. TD3

Table 2: Applications of RL

An important study extensively reviewed for this project [9] uses reinforcement learning for VSL. For the MDP in [9], the highway is divided into sections where each need to be assigned a VSL. Each section will have a state that contains variables for its current density & speed, and the initial state is set to be the standard maximum limit. Action spaces were explicitly defined based on regulations. They were made dependent & restricted on the current state, to avoid drastic shifts in output speeds. However, this approach could over-constrain the system, potentially leading to suboptimal policies by ruling out useful actions in some scenarios.

[9] uses Q-learning, which comes under model-free RL. This means it does not 'model the full environment', and just populates the state-action table with rewards. It does not directly pick up on all the options it could take and where it would lead it (transition probabilities), which further steps will be presented after this step is taken, etc. It is simple, and preferred when the environment is fixed with fixed rewards, but involves more exploration. [9] justifies the use of Q-learning due to initially unknown rewards & probabilities. Function approximation was considered a viable choice due to large or continuous state spaces, and they decided on artificial neural networks for this task. However, below is a better solution that also deals with these issues, while offering significant advantages.

The reward function in [9] is defined based on the vehicle hours, with bigger punishments (negative rewards) for longer hours. There is a threshold defined such that no punishment is given when the output speed is above it. Reward functions balance safety (e.g., minimising emergency braking), mobility (travel time), and sustainability (emissions) [28]. This study also considers traffic predictions that many others don't, and demonstrates better yields with it. MARVEL's parameter sharing allowed coordination across a 17-mile stretch, reducing collisions by 63.4% on highways [29]. The state description was extended to include previous speeds as well. Additional inputs were not considered due to lack of availability of the data.

[25] and many others address the struggle observed in RL to remember older training data. The below algorithm is highly sample-efficient, meaning it learns from lesser interactions, therefore not extending the memory to the point of forgetfulness.

2.6.3 Soft Actor-Critic

Deep reinforcement learning is the convergence of RL and deep learning, leveraging artificial neural networks (ANNs) for RL tasks. This combination enables the extraction of complex, high-dimensional nonlinear relationships from input data, making it particularly effective for decision-making in dynamic environments.

Soft Actor Critic (SAC) employs a stochastic actor-critic architecture, where the "actor" decides actions (basically a policy approximator), and the "critic" evaluates them, featuring separate policy and value function networks. The key differences between standard Actor-Critic (AC) and Soft Actor-Critic (SAC) are:

Feature	Actor-Critic (AC)	Soft Actor-Critic (SAC)
Exploration	Fixed stochastic/deterministic	Stochastic with learned distribution
Entropy Regularization	Optional	Always included
Action Output	Raw actions (deterministic) or categorical (discrete)	Mean + Std Dev for Gaussian distribution
Critic Networks	1 Q-function	2 Q-functions (min Q-value)
Temperature (α)	No	Yes (learned or fixed)

Table 3: Differences between standard actor-critic architecture & SAC

So basically, AC can also be modified to be applied on continuous spaces, but SAC handle that better since it was specifically designed for that purpose, along with many other improvements, like better exploration and entropy-based stability.

The SAC algorithm is an advanced off-policy RL method i.e. it can learn from previously collected data/interactions rather than requiring fresh interactions with the environment for every update. It is based on the maximum entropy framework, which encourages policies that are both high-performing and exploratory, reducing the risk of overly rigid or suboptimal strategies [15]. This basically means that the networks try to maximize both the reward & entropy. It benefits from entropy regularization (which keeps the policy from becoming too deterministic too quickly, encouraging better long-term exploration) to improve stability and sample efficiency (learning faster with fewer interactions). The temperature parameter in SAC allows the agent to balance exploration and exploitation.

SAC often uses two Q-networks (a "clipped double Q-trick") to mitigate overestimation bias, a common issue in Q-learning. While SAC primarily focuses on learning Q-functions (action-values), the concept of a "soft value function" is implicitly derived from the soft Q-function. The objective function for both the actor and critic in SAC incorporates an entropy regularization term, which influences the learned "value" by encouraging exploration.

In the original SAC paper, a separate state-value network was also trained. This network's objective was to approximate the expected value of a state under the current policy, including the entropy bonus [36]. Many modern implementations of SAC simplify the architecture by removing the separate network, while still achieving good performance, since Q-networks are already computes the concept.

SAC operates within a maximum entropy reinforcement learning framework. This means its objective is not just to find a policy that maximizes the cumulative reward, but also to find a policy that is as stochastic (random) as possible while still achieving a high reward. This encourages exploration and helps prevent the policy from converging prematurely to a suboptimal deterministic policy. This is done by the temperature parameter (α), which plays a crucial role in several areas of machine learning, the basic idea being controlling the randomness of a probability distribution. A well-tuned α allows the agent to explore effectively at the beginning of training and then gradually converge to a more exploitative (but still somewhat stochastic) policy as it learns [33]. A significant innovation in SAC is the concept of "automatic entropy tuning." Instead of manually setting α as a fixed hyperparameter, it can be learned and adjusted during training through a separate optimization process. This allows the algorithm to dynamically adapt the

exploration-exploitation trade-off, often leading to more stable and robust learning across different tasks [34][35].

The SAC algorithm is well-suited for optimizing VSL due to its efficiency, stability, and adaptability in complex traffic environments. SAC is a highly sample-efficient reinforcement learning method, meaning it can learn effective speed control strategies with fewer interactions - an essential advantage given the high computational cost of traffic simulations. Balancing exploration and exploitation, it is suitable in uncertain environments, where driver behavior, congestion levels, and external factors (e.g., weather, road incidents) fluctuate.

SAC is recognised for its effectiveness in continuous action spaces, which aligns well with the requirements of traffic control and driving environments. Research demonstrates SAC's ability to handle multiple objectives, such as speed, safety, comfort, and efficiency simultaneously. It has been implemented to handle many tasks in autonomous driving simulations [30]. The MARLIN framework applies SAC to congestion control in computer networks, maximizing entropy and return. While this is not a direct VSL application, the underlying principles are widely applicable: SAC's maximum entropy approach enables robust policy learning in environments with high variability and uncertainty, which is also a challenge in VSL systems [31]. One prior study has applied SAC to VSL with a focus on mixed traffic environments that include Connected Autonomous Vehicles (CAVs) [14]. These approaches utilise image-based traffic observations, and deep reinforcement learning models to adjust speed limits in response to real-time traffic flow. However, our project diverges from this by applying SAC to standard VSL scenarios (without relying on CAVs). [27] uses actor-critic, but they employ a lane-wise approach, which is highly unstable & might have the opposite effect of disruptive flow due to a higher tendency of changing lanes.

Below are key challenges while using SAC, and the development decisions taken to avoid them:

1. Sim-to-Reality Gap

Problem:

- SAC trained in simulation fails in the real world due to:
- Model inaccuracies (e.g., simulated friction $\neq$ real turf friction)
- Sensor noise (e.g., camera latency, IMU drift)

SAC-Specific Fix:

- Domain Randomisation: Inject noise into simulation training (e.g., randomise ball mass, motor delays). E.g.: Randomising ground friction in training reduces real-world errors by 45% [44].
- Hybrid Replay Buffers: Mix simulated and real-world data during training.

Project Modification: Noise has been included in the speed predictions while initially training of synthetic data.

2. Hardware Latency

Problem: Real robots have Actuator delays & Computation lag.

SAC-Specific Fix:

- Action Interpolation: Predict future states to compensate for latency. Cuts tracking error by 60% on quadrupeds [46].
- Faster Control Loops: Deploy quantized SAC models (e.g., TensorRT).

Project Modification: Not currently relevant to this project, it is only software-based. Even when displays on actual boards come in, limits change only once in 5 minutes (unless there are incidents).

3. Safety Violations

Problem: SAC's exploration can cause -

- Collisions (e.g., kicking obstacles)
- Joint overloading (e.g., exceeding motor torque limits)

SAC-Specific Fix:

- Action Clipping: Hard-code safety bounds in the actor network.
- Entropy Thresholding: Reduce exploration noise near obstacles. Resulted in zero collisions in 100 trials [45].

Project Modification: Safety aspect added to evaluate actions.

Traditional reinforcement learning (RL)-based methods optimize policies solely for reward maximization, which can lead to unsafe actions, such as abrupt decelerations that could lead to collisions. To address this, the Worst-Case Soft Actor-Critic (WCSAC) framework introduces a safety mechanism to mitigate the risks associated with unsafe decisions. The inclusion of a safety critic, operating parallel to the reward critic, enables the system to estimate the risk of unsafe actions. Safety is modeled using the Conditional Value-at-Risk (CVaR), a metric that minimizes the impact of worst-case traffic conditions. This risk-aware approach ensures policies prioritize safety in addition to efficiency. The adaptive nature of WCSAC allows for dynamic adjustment of the trade-off between safety and performance, enabling more reliable policy decisions under uncertain and variable traffic conditions. This method highlights the potential of integrating risk-aware metrics to generate safer and more stable outputs in RL-based systems, particularly in safety-critical domains like VSL control.

2.6.4 Comparison of RL Algorithms

Analyses evaluated the suitability of various reinforcement learning algorithms for training autonomous agents in competitive multi-agent soccer simulations, focusing on their ability to learn both cooperative and adversarial behaviors. While the application wasn't similar to this study, the comparison & capabilities of each algorithm that was described was valuable.

Algorithm	Strengths	Limitations	Suitability
Q-Learning	Simple implementation	Poor scalability with multiple agents	Basic discrete action scenarios
DQN	Handles high-dimensional states	Instability in dynamic environments	Single-agent skill learning
PPO	Stable policy updates	Requires careful hyperparameter tuning	Continuous control fundamentals
MADDPG	Centralized training framework	High computational requirements	Team coordination and strategy
SAC	Automatic entropy tuning	Complex implementation	Continuous precision movements

Table 4: Comparison of RL Algorithms

SAC demonstrated particular effectiveness in learning precise continuous control actions essential for individual player skills, with its entropy-maximizing objective proving valuable for maintaining exploration in competitive scenarios. It showed similar pattern to PPO with fast convergence. The only downsides mentioned are complexity of implementation, and memory requirements, both of which we have already tackled through implementation & simple input data. Future research directions mentioned included investigating SAC variants with centralized critics for multi-agent scenarios - maybe combining SAC's exploration advantages with MADDPG's team coordination capabilities [39][40].

Empirical studies consistently rank SAC among the most sample-efficient continuous control algorithms. Though its replay buffer incurs higher memory costs (~4GB/1M transitions), recent work demonstrates effective compression techniques retain 90% performance at 60% memory overhead [42]. In multi-agent settings, decentralized SAC variants rival specialized methods like QMIX in cooperative scenarios [43], making it a versatile baseline. SAC's entropy maximization & automatic tuning enabled 3.1x more state coverage than DDPG in maze navigation. Many efforts were focused on Memory/Runtime Tradeoffs since although it is sample-efficient, the length of each sample & double Q networks were still heavy. Some

suggestions include prioritized sampling [42], & parameter sharing. Many studies applied FIFO prioritisation, keeping only first x samples in memory.

SAC has demonstrated remarkable real-world viability, achieving high precision in robotic insertion tasks while maintaining safety via hardware-aware action clipping. Though memory-intensive, recent edge-computing adaptations like Edge-SAC [46] enable 100Hz control on embedded devices through network quantization. Its sample efficiency shines in physical systems, equiring 6-8x fewer trials than PPO (e.g. 50 trials to learn door-opening vs. PPO's 300+), making it ideal for costly real-world training.

2.6.5 Modeling & Simulation

A significant amount of research has been conducted in the traffic flow domain, focusing on modeling and optimizing highway traffic flow. These can be classified into two categories: microscopic and macroscopic. Macroscopic prediction models remain the primary choice for VSL studies due to their ability to effectively capture aggregate traffic dynamics and simplify implementation across large-scale networks. While microscopic models show promise in representing individual driver behaviors with greater granularity, they are computationally demanding, and introduce complexities in modeling lane-changing behavior and multilane road networks.

METANET-based models have been widely used to simulate & test transportation networks, and the effect of applying various strategies to control it. With both synthetic & real-world data, these macroscopic models demonstrated their capability to accurately describe the dynamic behaviour of traffic and reproduce all relevant phenomena, highlighting METANET's effectiveness as a tool for traffic prediction and management [2]. They use averaged parameters that represent the traffic as a whole, which is acceptable for this use-case. We can set the duration time of the simulation and the step size at which models are applied to observe reactions.

There have been many modifications made to the original METANET to suit specific use-cases within traffic modelling, such as support for multiple lanes, vehicle types, etc. [9] also uses METANET, with a few changes. They consider flow, density & speed, with constants defining the length of the highway and no. of lanes, and check traffic volumes of on-ramps & off-ramps. Traffic near specific on-ramps compared to other lanes is a common sight everyday in the UAE. They divide the highway into sections, and observe the incoming & outgoing traffic volumes of both in each, and queue length of the ramp. They keep the maximum density (when speed nears 0) & ideal free flow speed as reference values. A step size is defined such that VSLs outputs are calculated only when the METANET step is a multiple of the size.

2.6.6 Existing Implementations of VSL

United States

- Tennessee, Interstate 24:
 - 17-mile corridor near Nashville equipped with 67 VSL controllers.
 - Multi-agent reinforcement learning (MARL) system launched in March 2024, making over 35 million speed decisions in six months.
 - Results: 14% better accuracy in congestion warnings, 26% reduction in crash rates, and 50% drop in secondary crashes compared to non-RL benchmarks [75].
- East Coast, Interstate 95: Service reliability in corridor operations [76].
- Washington State: I-5 and Snoqualmie Pass feature long-standing VSL deployments since the 1990s with significant crash reductions and smoother traffic flow.
- Colorado: In mountainous corridor, incorporating weather sensors & radar to address rapidly changing conditions. Documented declines in crash rates, especially for trucks.
- Others: California, Oregon, Washington, Nevada, Colorado, Minnesota, Texas, Utah [81].

Europe

- Stockholm, Sweden: Empirical evidence links VSL to better speed harmonization and reduced congestion and accidents, particularly when implemented as mandatory [78].

- Netherlands: VSL systems using shockwave theory algorithms implemented for congestion mitigation, resulting in incident reductions and more predictable travel times.
- United Kingdom: M25 and other motorways utilize VSL in “smart motorway” schemes to preempt or respond to congestion and incidents, reducing crashes and increasing capacity.
- Finland, Germany, France: VSL applied on major highways, often in regions with extreme and variable weather. Notable impacts include measurable reductions in average speed and improved safety during adverse road conditions.

Canada & Australia

- Toronto, Canada: Urban freeway pilot using VSL demonstrated significant reduction in crash potential with a minor trade-off in travel time.
- British Columbia: Mountain highways equipped with VSL systems to adjust speeds during snow, ice, or fog. Achieved better compliance and decreased severe collisions.
- Australia: Cities and construction zones to dynamically respond to changing traffic patterns [81].

2.6.7 Traffic Speed Prediction

Traffic speed prediction has been widely studied in intelligent transportation systems (ITS), often relying on large datasets and complex models to achieve high accuracy. However, when working with limited historical data and focusing on traffic control rather than forecasting, simpler models can often be more effective and interpretable.

- Baseline and Time-Averaging Methods: such as time-of-day averaging or persistence (using the last known speed as the next prediction) are frequently used in practice. These methods perform surprisingly well for short-term predictions, particularly in roads with regular daily patterns. As noted by [71], such approaches form strong baselines against which to compare more complex methods.
- Regression Models:
 - Linear Regression: Not suitable, since the pattern was visibly non-linear during analysis.
 - Decision Trees/Random Forests: Can capture non-linearities and interactions, but may overfit with very little data. Not suitable.
- Time Series Models: Autoregressive models like ARIMA have been used to capture temporal patterns in speed data. Studies demonstrate that AR models can provide good performance on freeway speed prediction but require a significant amount of clean, continuous data to train effectively, which is often not available.
- Machine Learning Models: such as Support Vector Regression (SVR) and Random Forests have shown promise in speed prediction due to their ability to model non-linear relationships. However, overfitting can be a major concern when applying such models to small datasets. Moreover, their "black-box" nature can hinder interpretability, important for applications where transparency in decision-making is required.

Why KNN is a Suitable Choice

K-Nearest Neighbors (KNN) offers a good balance between simplicity and performance for short-term traffic prediction in limited-data settings. It is a non-parametric, instance-based model that relies directly on historical observations to make predictions [70]. Several recent studies support its use in low-data traffic scenarios [72], highlighting its utility in small urban datasets, due to its adaptive nature and low training complexity. [74] found that KNN provided competitive performance in freeway traffic prediction when combined with effective feature scaling and distance weighting.

Limitations and Tradeoffs

While KNN is interpretable and works well for periodic patterns, its memory requirement grows with the dataset size, and it struggles with extrapolation i.e. predicting for time combinations not present in historical

data. In our case, the dataset was small and consistent across days, making KNN a practical and lightweight solution.

3. Funding

I, Anushka Narsima, verify that I have no affiliations with or have any involvement in any organization with any financial or non-financial in the topic discussed in this systematic review.

4. Requirements

The first step of following a PM methodology would be requirement analysis. Defining the expectations of the end-result in a systematic manner helps strategically approach development - prioritizing the most important aspects, completing steps that move us closer to the result, minimize issues & miscommunication, and ensure a good quality product.

4.1 Functional Requirements

This project is to build a model that outputs variable speed limits. Here are the specifics:

1. Learning-Based VSL Agent

The RL-based controller is the core intelligence of the system. It learns optimal speed limits based on observed traffic conditions and its past experiences. The following behaviors are required:

- **Adaptive Learning Capability**
The agent must learn by **interacting with the simulation environment**, observing its actions' effects, and refining its policy over time. This is a core characteristic of reinforcement learning and will be evaluated by improvement in key metrics like Total Time Spent (TTS).
- **Logical and Interpretable Behavior**
The model should produce speed limit decisions that align with **human intuition and traffic logic**. For instance, reducing limits under high congestion and increasing them in free-flow conditions. Outputs should be reasonable and interpretable, reinforcing trust in the system.
- **Safety-Aware Decision Making**
The system should include a **safety constraint mechanism**, e.g. using Conditional Value at Risk (CVaR) or a safety critic. The goal is to minimize unsafe traffic states such as rapid deceleration or erratic speed variations that could lead to accidents.
- **Traffic Efficiency Optimization**
The primary metric for evaluating success is **Total Time Spent (TTS)**. The agent should learn to reduce congestion and improve flow over time, achieving lower TTS values compared to baseline or no-control scenarios.
- Inputs:
 - Current, previous & predicted speeds
 - Previous Limit
 - Safety Parameter
- Output: Variable Speed Limit

To show working outside of the command line, a web application will be built, useful as a dashboard for the controlling authorities. Here is what's expected of it:

- Should integrate google maps
- Should mark points at which model is trained on data & current limit

4.2 Non-Functional Requirements

- Safety - Especially for a system intended to be deployed on the real roads of a city with millions of people commuting daily, safety is the most important & non-negotiable point that has to be given upmost priority. This project introduces that into the VSL application.
- Reliability - These highways are used round-the-clock and the system will have to support that. Data for the full day has to be collected to extend the model for this.
- Performance - Needs to update on time & show real-time values on the dashboard.
- Scalability - Needs to be designed to be applied into many different roads, and show all on the dashboard.

4.3 Software and Framework Requirements / tools

Model:

- TensorFlow – Main deep learning python framework used to define the networks & their behaviours.
- Numpy – Often used with tensors to reshape, defining arrays, etc.
- Gym – Reinforcement learning framework that creates the action space.
- Matplotlib – For visualization & plotting.

Simulation

- sym-metnet – Only open-source implementation of METANET available
- casadi – Used with above framework, for differential calculations used to simulate traffic.
- Numpy
- Matplotlib

Data Collection & Analysis

- TomTom API – Provided real-time speed & incident data
- Azure Maps API – Provided traffic levels
- Google Cloud Platform – Used for automation & hosting
- Scikitlearn – For prediction models
- Pandas – Working with collected data on excel sheets
- matplotlib

Dashboard

- JavaScript – Imported maps, handled HTTP requests
- HTML & CSS – Front-end
- GCP – Maps API & hosting

Others

- VS Code & Google Colab was used for coding
- Excel sheets to store data
- Canva for diagrams & slides

- Next.js for blog
- MS word for documentation
- Git & GitHub for version control & storage / showcasing

5. Design & Plan

The aim of this project is to build an updated, scalable & efficient VSL system, trained & tested on real world data. This section has detailed the plan ahead to implement it.

5.1 Software Development Methodology

Project management is the key to successful & timely completion of a project. The chose of model was done after thinking about the following points –

- Iterative Development – Essential for refining the RL model (e.g., starting with a basic actor-critic framework, then incrementally adding soft updates, entropy, and safety metrics).
- Project progress needed to be tracked, on a weekly basis, to ensure timely planning to achieve successful completion.
- Modular Testing – The simulation (METANET) and RL components were developed and validated in isolation before integration.
- Flexible Task Management – There were a list of tasks that needed to be completed is each section & stage. Kanban visualized dependencies & tracked progress without rigid sprint deadlines.

Implementation Workflow:

- Scrum Cycles:
 - One/Two-week sprints for RL model & enhancements (e.g., reward function tuning).
 - Twice-a-week standups to address roadblocks (e.g., convergence issues in SAC).
 - Weekly retrospective meetings to reflect on approach & modify if required, prioritise, etc.
- Kanban Board Columns: Backlog, In Progress, Testing, Done.

The project adopted a hybrid Agile-Kanban approach, combining the iterative structure of Scrum with the task-focused flow of Kanban. Agile (Scrum) ensured rapid iterations for the RL component, where requirements evolved with experimental results. Kanban provided transparency for parallel tasks (e.g., simulation debugging vs. RL training), avoiding bottlenecks.

The hybrid approach addressed key challenges:

- Novelty & Low Experience: Early sprints focused on "proof of concept" (e.g., a working actor-critic baseline). Later sprints incorporated advanced features (entropy regularization) after initial validation.
- Simulation Dependencies: Kanban tracked parallel tasks, prerequisites, etc.

Any SDE model will involve the stages of initiation, planning, design, development, & testing. So these will be the following sections of this report.

5.2 Work Breakdown Structure

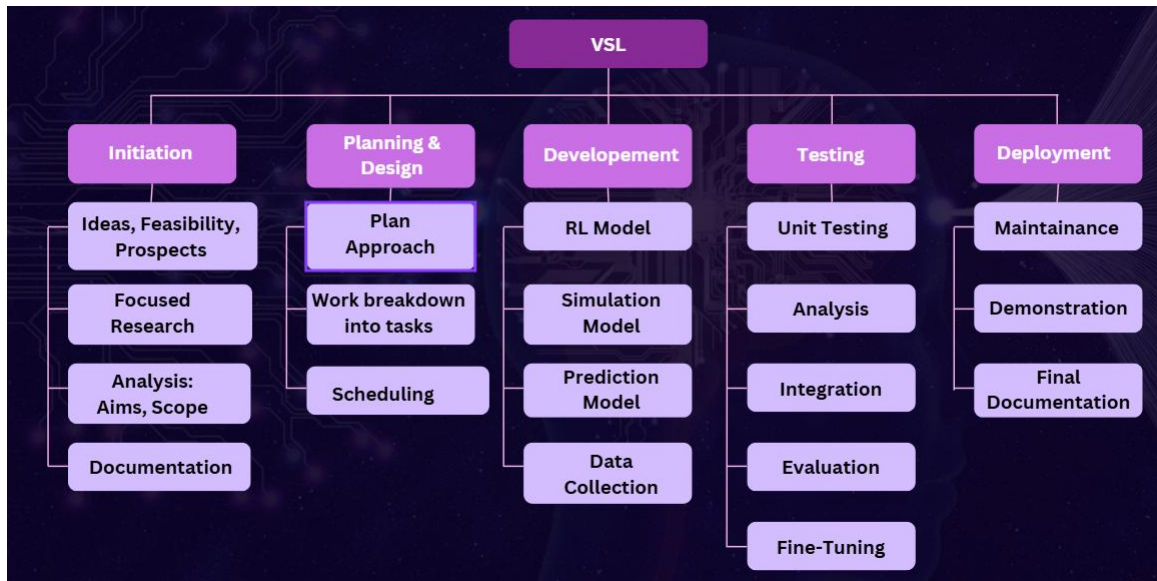


Fig. 2: Work Breakdown Structure

5.3 Project Timeline

Task	Start Date	End Date	Duration (Weeks)
Project Documentation	03/01/2025	25/7/2025	20
Supervisor Meetings	03/01/2025	25/7/2025	20
Blog Updates	03/01/2025	25/7/2025	20
1. Project Initiation	03/01/2025	03/12/2025	2
1.1 Research topic & pick idea	03/01/2025	03/04/2025	0.6
1.2 Define aims, scope, etc.	03/01/2025	03/04/2025	0.6
1.3 Consider feasibility	03/01/2025	03/04/2025	0.6
1.4 Research implementation & decide approach	03/04/2025	03/12/2025	1
1.5 Propose & discuss	03/04/2025	03/12/2025	1
1.6 Evaluate ethics	03/12/2025	03/12/2025	0.2
2. Design	03/12/2025	31/3/2025	2
2.1 Literature Review	03/12/2025	31/3/2025	2
2.2 Requirement Analysis	03/12/2025	15/3/2025	0.4
2.3 Design model architecture	15/3/2025	31/3/2025	2
2.4 Decide methods & schedule	15/3/2025	31/3/2025	2
3. Development	04/01/2025	31/6/2025	12
3.1 Build simple actor-critic mechanism	04/01/2025	15/4/2025	2
3.2 Base simulation script	20/4/2025	31/4/2025	1.5
3.3 Setup data collection	04/07/2025	31/6/2025	11
3.4 Run model with simulation	05/01/2025	05/10/2025	1.5

3.5 Add on soft updates, etc.	05/10/2025	31/6/2025	7
3.6 Customise simulation	05/01/2025	30/5/2025	4
3.7 Train model to perform well	05/10/2025	31/6/2025	7
3.8 Setup prediction model	20/5/2025	30/5/2025	1.5
3.9 Add all inputs to state	20/5/2025	30/5/2025	1.5
3.10 Plug-in real data	20/5/2025	30/5/2025	1.5
3.11 Dashboard dev	25/5/2025	30/5/2025	1.5
4. Testing	05/01/2025	30/6/25	8.5
4.1 Test base simulation script	05/01/2025	05/07/2025	1
4.2 Test API	05/01/2025	15/5/2025	2
4.3 Test SAC model with base sim	05/07/2025	30/6/25	7.5
4.4 Data Analysis	05/01/2025	05/07/2025	1
4.5 Evaluate model performance	05/07/2025	30/6/25	7.5
4.6 Debug & fine-tune	05/07/2025	30/6/25	7.5
5. Deployment	07/01/2025	25/7/2025	3
5.1 Host dashboard & model on GCP	07/01/2025	07/05/2025	0.6
5.2 Demo successful inference	18/7/2025	18/7/2025	0.2
5.3 Presentation & Viva	18/7/2025	18/7/2025	0.2
5.4 Final Submission	25/7/2025	25/7/2025	0.2

Table 5: Project Timeline

5.4 Gantt Chart

	Task	Assigned To	Start	End	Dur	%	2025				
							Mar	Apr	May	Jun	Jul
	Variable Speed Limits	⊖	3/1/25	7/25/25	105						
1	Project Documentation		3/1/25	7/25/25	105						
2	Supervisor Meetings		3/1/25	7/25/25	105						
3	Blog Updates		3/1/25	7/25/25	105						
4	Project Initiation	⊖	3/1/25	3/12/25	8						
4.1	Research topic & pick idea		3/1/25	3/4/25	2						
4.2	Define aims, scope. etc.		3/1/25	3/4/25	2						
4.3	Consider feasibility		3/1/25	3/4/25	2						
4.4	Research implementations & decide approach		3/4/25	3/12/25	7						
4.5	Propose & Discuss		3/4/25	3/12/25	7						

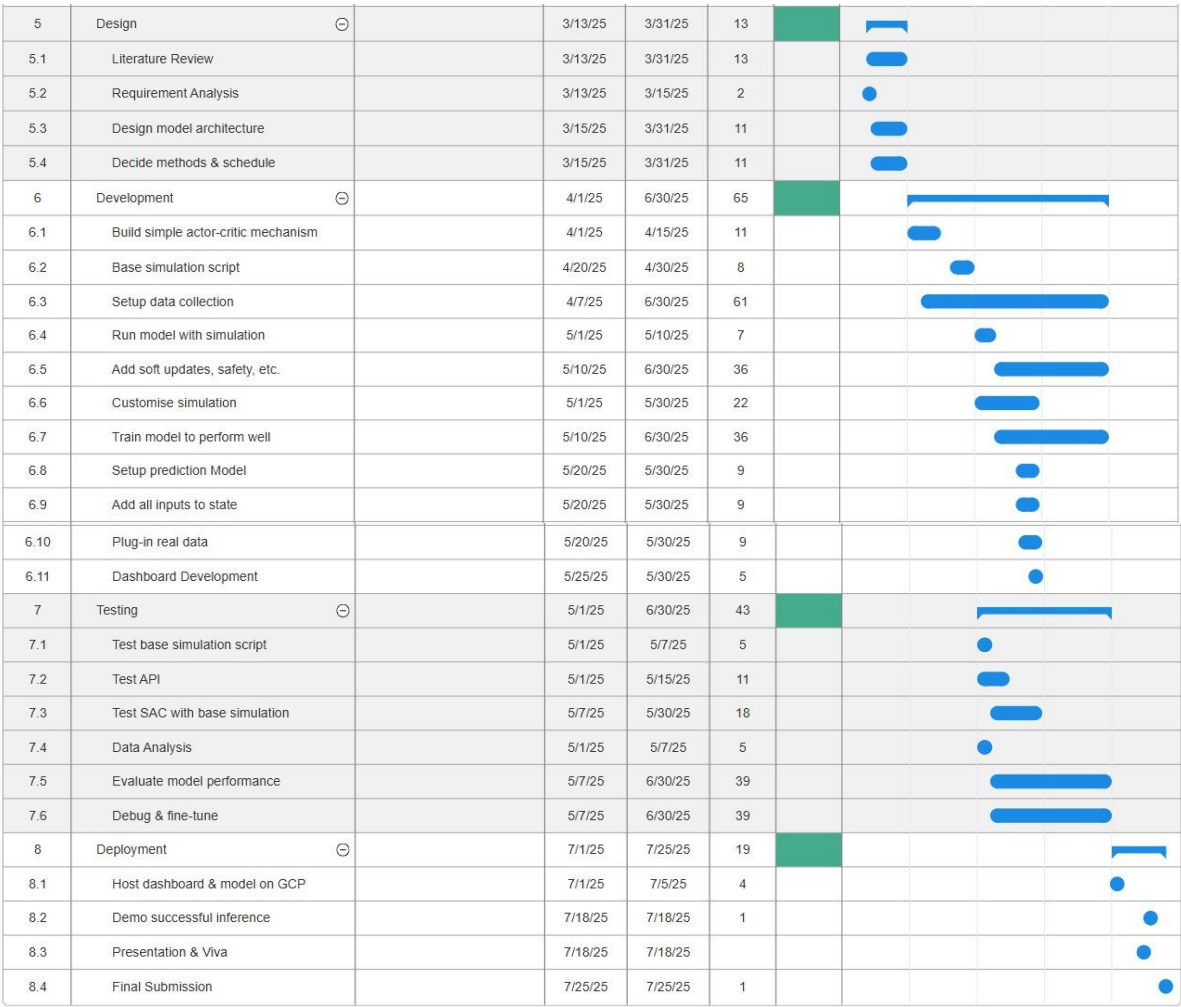


Fig. 3: Gantt Chart

5.5 Mind Map

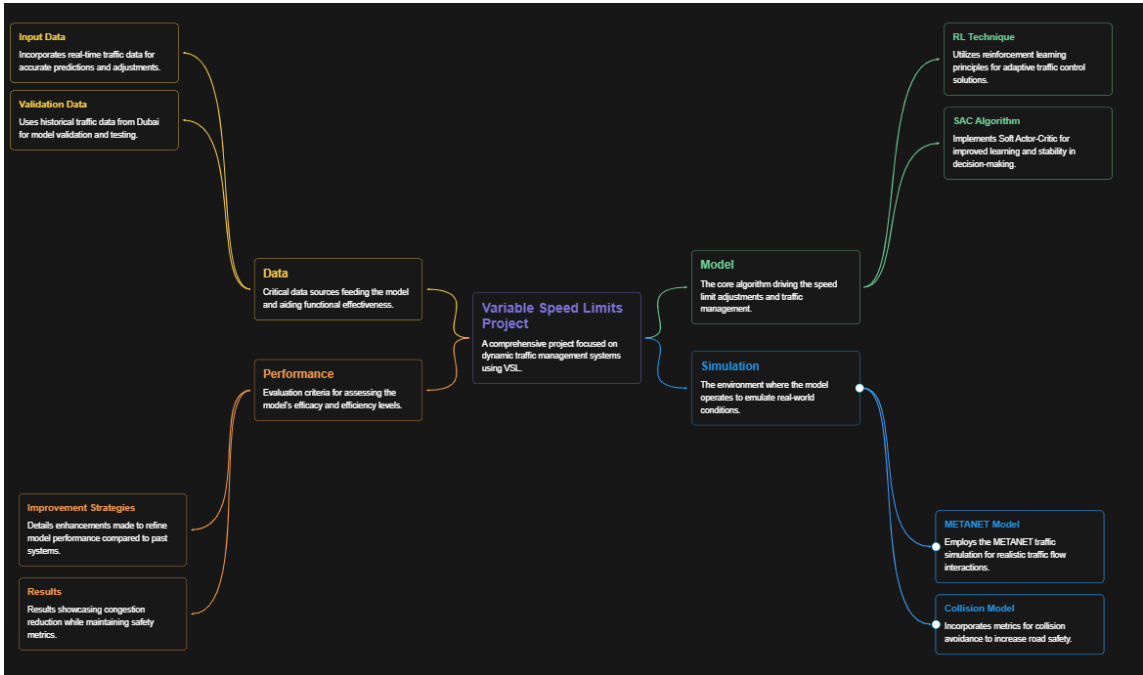


Fig. 4: Mind Map

5.6 UML Diagrams

5.6.1 SAC Flowchart

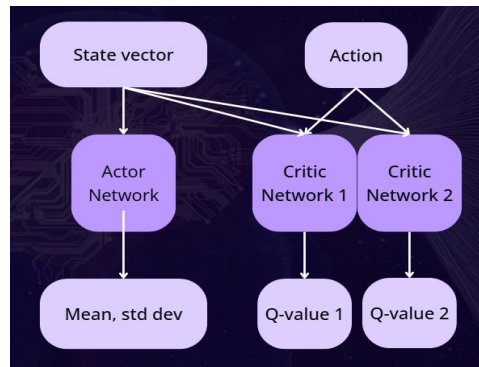


Fig. 5: SAC Flowchart

5.6.2 Training Flowchart

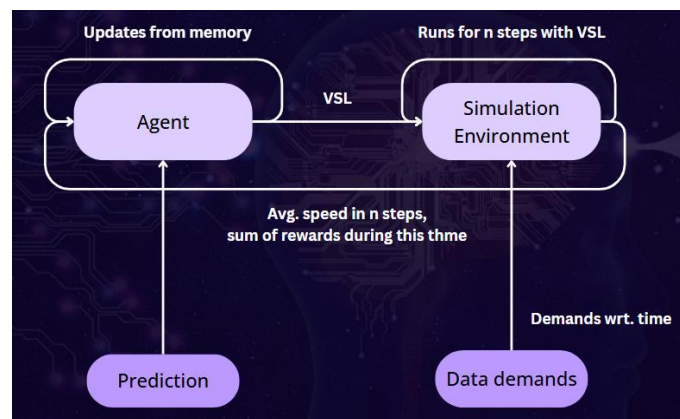


Fig. 6: Training Flowchart

5.7 Pseudocode

SAC Training Loop:

1. Sample action $a \sim \pi(\cdot|s)$
2. Execute in environment, store (s,a,r,s') in buffer
3. For each batch:
 - a. Update critics via $\min(Q1,Q2) + \text{entropy}$
 - b. Update actor to maximize $Q + \alpha H(\pi)$
 - c. Adjust α to meet target entropy
 - d. Soft-update target networks

Pseudocode for training:

- 1) Initialize actor critic network
- 2) Repeat for large number of episodes

- 3) Reset the environment, score, terminal flag
- 4) While state is not terminal
- 5) Select action according to actor network
- 6) Take action and receive reward and new state
- 7) Plot scores over time for evidence of learning

6.8 Details

6.8.1 Algorithm

Let's start with getting the intuition. Looking at Fig. 1, we are looking at a highway, and need to assign speed limits based on current data. There is congestion on the second half, where there is an on-ramp with incoming vehicles. What should the VSLs be? A good approach would be to gradually reduce speeds closer to the average speed of the congested area. So first will be a little lesser than the standard limit, second will be even lesser, and so on. When should they be issued? Best if before the congestion happens, and avoid the buildup as much as possible. After this area, speeds can gradually rise, and this should happen as density shows signs of decreasing. This is how we want the model to work.

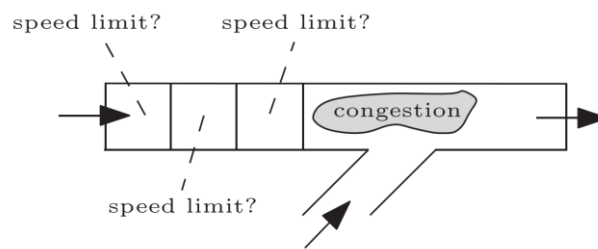


Fig. 7: Sketch of a congestion scenario

An MDP was formulated in [9], but predicted values were added with the current values together in the state descriptions. Predictions are not ground truth, so these should be considered partially observable, and the problem should be formulated as a POMDP.

Let's begin by defining the problem as a POMDP - Includes states S , actions A , transition probabilities P , rewards R , discount factor (γ).

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

Usually, the outputs/actions of an actor are probabilities for each of the action set, where SoftMax function is applied such that all add up to 1. In this case, since the output of the actor network needs to be a speed (a continuous value) instead of a probability distribution over discrete actions, the actor network is designed to output parameters of a probability distribution over continuous actions, a Gaussian distribution. [9] has categories for different limits at intervals of 10, but a distribution was chosen to make the results more interpretable.

The actor network will output a mean (μ) and standard deviation (σ) for a Gaussian distribution over the continuous action space. The speed limit action is sampled from the Gaussian distribution:

$$a = \mu + \sigma \cdot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, 1)$$

The sampled action is then clipped to ensure it remains within a valid range, the minimum and maximum allowed speed limits for the road. Speed limits have a specific range (e.g., [30 km/h, 120 km/h]), so the activation function can be:

- **Tanh:** Scales output to $[-1, 1]$, then must be rescaled to range.
- **Sigmoid:** Scales output to $[0, 1]$, rescale similarly.

Since both the **mean** (μ) and **standard deviation** (σ) are unconstrained and the output distribution is Gaussian, the loss function is derived from the **negative log-likelihood (NLL)** of the Gaussian distribution, not just least squares. The loss penalizes incorrect predictions, relative to the predicted uncertainty (σ). This allows the model to adjust both the mean and uncertainty dynamically. The NLL for a Gaussian distribution is:

$$\text{Loss} = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - \mu)^2}{2\sigma^2}$$

Policy: Negative of the uncertainty of the policy

$$\mathcal{H}(\pi(\cdot|s)) = \mathbb{E}_{a \sim \pi} [-\log \pi(a|s)]$$

Actor / Policy Loss: Includes uncertainty $\log(\pi)$, alpha (α) encourages entropy, penalize low Q-values, safety term added. This is written like below to minimize negative entropy & negative Q-value, where a higher $\log(\pi)$ means more exploration and a higher Q-value means more exploitation.

$$\mathcal{L}_{\text{actor}} = \mathbb{E} [\alpha \log \pi(a|s) - Q(s, a) + \lambda Q_{\text{safety}}(s, a)]$$

Value Function: Based on current state. Includes reward r * discount factor (γ), which gets smaller for rewards further in the future.

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

Generic Q / Critic Function: Based on current state & action. Includes reward r * discount factor (γ), which gets smaller for rewards further in the future.

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Soft Q / Critic Function: With soft updates using entropy value (α).

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right]$$

Q Loss:

$$\mathcal{L}_Q = \frac{1}{2} \mathbb{E} \left[(Q(s, a) - (r + \gamma (Q(s', a') - \alpha \log \pi(a'|s'))))^2 \right]$$

Entropy -

Entropy Loss: Uncertainty of current policy + target entropy, * current entropy

$$\mathcal{L}_\alpha = \mathbb{E}_{a \sim \pi} [-\alpha \log \pi(a|s) - \alpha \mathcal{H}_{\text{target}}]$$

Q Value:

$$Q_{\text{safety}}(s, a) = \mathbb{E}[\text{safety cost} \mid s, a]$$

Safety Loss:

$$\mathcal{L}_{\text{CVaR}} = \frac{1}{\alpha N} \sum_{i=1}^{\alpha N} Q_{\text{safety}}^{(i)}(s, a)$$

Instead of learning from just the most recent step (which could be highly correlated), we store lots of past experiences, and later sample a random batch from this buffer to train the agent. This breaks correlation between consecutive steps and improves learning stability. This is especially important in off-policy algorithms like DQN or SAC [22].

Probabilities can be very small numbers. Multiplying many small probabilities together (e.g., in a long sequence of actions) can lead to "numerical underflow" where the result becomes effectively zero on a computer. Taking the logarithm converts multiplications into additions ($\log(A \cdot B) = \log A + \log B$), which are numerically more stable [37].

Reward Design

To guide learning, we define a reward function that penalizes **Total Time Spent (TTS)** on the road and in on-ramp queues. It is computed by:

- Summing the product of traffic density with link length and lane count.
- Adding the time vehicles spend waiting on on-ramps.
- The result is scaled and negated, so **lower TTS yields higher reward**.

This aligns with the real-world goal of minimizing congestion and improving throughput.

6.8.2 Simulation Framework

For this study, we adopt a macroscopic modelling approach due to its efficiency in capturing aggregate traffic dynamics and suitability for large-scale simulations. Given the focus on optimizing VSL at a network-wide level, macroscopic models provide a balance between accuracy and computational feasibility. We utilize the METANET traffic model, a widely used macroscopic framework for simulating freeway traffic flow and evaluating control strategies.

To tailor the METANET implementation for our study, we specify structures that enhance its applicability to the UAE traffic environment. Our model divides the highway into multiple segments, monitoring traffic flow, density, and speed at each. We incorporate on-ramp and off-ramp interactions, as these significantly affect congestion in urban highways. The model references critical density thresholds, distinguishing between free-flow, congested, and near-gridlock conditions, allowing for adaptive speed limit interventions.

6.8.3 Data

Two options were considered for collecting real-time (& historic) traffic data: TomTom, and Azure.

TomTom provided high-resolution traffic insights, including traffic flow and incident reports (accidents, roadblocks, construction delays), with detailed coverage of the UAE. By leveraging TomTom's RESTful API endpoints, traffic data could be queried and integrated into the simulation, allowing for data-driven calibration of speed limits and reinforcement learning models.

Here are the detailed API endpoints relevant to this project:

- **Traffic API** – Real-time
 - **Flow Segment Data** – Provides current speed, also free flow speed, current & free flow travel times, and shape of road segment.
 - **Vector Flow Tiles** – Includes traffic level between 0 & 1.
 - **Incident Details** – Describes incidents on bounded area requested.
- **Traffic Stats API** - Historical, returns data over date & time range
 - **Route Analysis Service** – Data over a series of points. Useful in case segments made by TomTom itself are not suitable. Gives:
 - speed limit, Harmonic average speed, medianSpeed, averageSpeed, standardDeviationSpeed, speedPercentiles
 - travelTimeStandardDeviation, sampleSize, averageTravelTime, medianTravelTime, travelTimeRatio
 - **Area analysis Service** – Similar, takes different format of coordinates.
- **Maps API** – Helps with visualization.

The Azure Maps Traffic service is a comprehensive set of web services built for developers, enabling the creation of dynamic web and mobile applications that incorporate real-time traffic information. This service allows you to either visualize current traffic conditions directly on maps or integrate the data to calculate smarter routes that account for live driving conditions. Endpoints are similar to TomTom:

- Traffic Flow Segment
- Traffic Flow Tile
- Traffic Incidents Detail

6.9 Evaluation

To assess the effectiveness of the SAC algorithm in optimizing Variable Speed Limits (VSL), a multi-faceted evaluation approach will be employed. The performance of the learned policies will be analysed through traffic simulations on the METANET model, using real-world data from Dubai's road network, collected via the APIs [23].

1. Performance Metrics:

The evaluation should primarily focus on the following key metrics:

- **Total Time Spent (TTS):** Measures the mean time taken by vehicles to traverse the network.
- **Traffic Throughput:** Represents the number of vehicles passing a given point per unit time.
- **Speed Variability:** Analyses fluctuations in vehicle speeds to assess traffic stability.
- **Congestion Reduction:** Assesses the impact of VSL on bottleneck formations.
- **Safety Metric:** Evaluates potential accident risks based on speed harmonization and rapid deceleration rates.

2. Baseline Comparisons:

To determine the efficacy of the SAC-based VSL control, results should be compared against:

- **No Control (Fixed Speed Limits):** A scenario where speed limits remain constant.
- **Rule-Based VSL:** A traditional approach that adjusts speed limits based on predefined congestion thresholds.
- **Alternative RL Approaches:** Potential comparisons with other RL-based VSL methods.

3. Simulation Scenarios:

Multiple traffic scenarios should be tested to ensure robustness, including:

- **Peak vs. Off-Peak Traffic:** Evaluating performance under varying congestion levels.
- **Incident-Induced Congestion:** Testing how SAC-VSL adapts to sudden disruptions.
- **On-Ramp & Off-Ramp Interactions:** Assessing how VSL influences merging and diverging traffic flows.

By leveraging data-driven simulations and comparative analysis, this evaluation framework aims to establish the practical feasibility of reinforcement learning-based VSL control in large-scale urban freeway networks.

7. DEVELOPMENT

7.1 Algorithm Codes & Explanations

The logic was divided into a few base files, making it usable for various applications:

- Network.py – Contained main network classes – Actor & Critic networks.
- Agent.py – agent class, that made instances of the networks, defined the main learning function.
- Sim_env.py – Simulation class, used framework to define road structure & its behaviour methods.
- Train.py – initializes agent & environment, and runs training in a loop, main file to run.

The network.py module implements the core neural network architecture for the Soft Actor-Critic (SAC) algorithm). It consists of 2 classes, the actor network & the critic network. Some important points:

- Feature extractor: consists of two fully connected layers, for both actor and critic. The reason why actor & critic isn't a single network branching out, is because the actor just needs the inputs and gives an output, whereas the critic needs both actor + outputs to 'critic' it, i.e. q-values are calculated for state-action pairs.
- Activation function for inner layers: LeakyReLU mitigates dying neurons compared to ReLU, and since negative values are involved in the calculations
- Optimizer: Adam is the most popular & reliable option, which was evident in earlier-attempted projects. Still, SGD was also tested during debugging.
- Boundaries: Limiting output by giving min & max values
- Actor/Policy outputs: a mean & standard deviation. Mean has tanh applied as activation function, and is rescaled to sit within bounds. Softplus is applied to std dev.
- Sampling: Sample_normal() builds a distribution with actor outputs mean & standard deviation, and takes samples from that, along with calculating log probabilities ie. how likely were the other actions.
- Model saving & loading methods allow saving checkpoints during training.

```
26 class SACNetwork(keras.Model):
27     def __init__(self, min_s, max_s, # n_actions,
28                 l1_dims=32, l2_dims=64, # No. of neurons for full connected layers 1 & 2
29                 name='sacn', cp_dir='models', alpha=0.01): # Checkpoint Directory
30         # Alpha used to be 0.0001, trying 0.01 now
31
32         super(SACNetwork, self).__init__()
33         # self.n_actions = n_actions
34         self.name = name
35         self.alpha = alpha # Learning rate for the optimizer
36         self.cp_dir = cp_dir
37         self.cp_file = os.path.join(self.cp_dir, name+'_ac'+'.weights.h5')
38         self.min_s = min_s
39         self.max_s = max_s
40         self.l1_dims = l1_dims
41         self.l2_dims = l2_dims
42         self.noise = 1e-6
```

Fig. 8: Start of SAC class

```

44         # Two optimizers for actor and value outputs
45         self.v_optimizer = Adam(learning_rate=alpha)
46         self.a_optimizer = Adam(learning_rate=alpha)
47
48         self.l1 = Dense(self.l1_dims)
49         self.l1_activation = LeakyReLU(alpha=0.01)
50         self.l2 = Dense(self.l2_dims)
51         self.l2_activation = LeakyReLU(alpha=0.01)
52         # Tried just relu, trying leakyrelu now
53
54         # Value function
55         self.value_f = Dense(1, activation=None)
56
57         # Policy function, Output will be mean & standard deviation to represent distribution
58         self.mean = Dense(1, activation='tanh') # tanh gives [-1, 1]
59         self.scaled_mean = Lambda(lambda x: min_s + (max_s - min_s) * (x + 1) / 2) # Scale to [min, max]
60         self.std_dev = Dense(1, activation='softplus') # Softplus ensures positive std dev
61         # Build network

```

Fig. 9: __init__() of SAC class

```

def sample_normal(self, state, reparameterize=True):
    _, mean, std_dev = self.call(state)
    probabilities = tfp.distributions.Normal(mean, std_dev) # Create distribution
    actions = probabilities.sample() # Sample action
    # action = tf.math.tanh(actions) * self.max_s # Apply tanh to squash action into range [-1,
    # then scale to max speed
    # mean is already scaled by scaled_mean, just clip
    action = tf.clip_by_value(actions, self.min_s, self.max_s)

    # Compute log probability of sampled actions (used for entropy regularization and actor loss)
    log_probs = probabilities.log_prob(actions)
    # Apply correction to log_probs to account for the tanh squashing (change-of-variable correction)
    # This prevents underestimating the true log-probability due to nonlinearity of tanh
    log_arg = tf.clip_by_value(1 - tf.math.pow(action, 2) + self.noise, self.noise, 1.0)
    log_probs -= tf.math.log(log_arg)

    # Sum over all action dimensions (usually only 1 here, but done for generality)
    log_probs = tf.math.reduce_sum(log_probs, axis=1, keepdims=True)

    return action, log_probs

```

Fig. 10: sample_normal() method

```

109 class CriticNetwork(keras.Model):
110     def __init__(self, fc1_dims=16, fc2_dims=32, name='critic', cp_dir='tmp', alpha=0.01):
111         super(CriticNetwork, self).__init__()
112         self.cp_file = os.path.join(cp_dir, name+'_critic')
113
114         # Initialize optimizer
115         self.optimizer = SGD(learning_rate=alpha)
116
117         # Same structure as SACNetwork, but with state and action inputs
118         self.fc1 = Dense(fc1_dims)
119         self.fc1_activation = LeakyReLU(alpha=0.01)
120         self.fc2 = Dense(fc2_dims)
121         self.fc2_activation = LeakyReLU(alpha=0.01)
122         self.q = Dense(1, activation=None) # Q-value output
123
124     def call(self, state_action):
125         state, action = state_action
126         # Concatenate state and action
127         x = tf.concat([state, action], axis=-1)
128         x = self.fc1_activation(self.fc1(x))
129         x = self.fc2_activation(self.fc2(x))
130         return self.q(x)

```

Fig. 11: Critic Network Class

Justification of hyperparameters can be found in the table below.

Parameter	Value	Rationale
Learning rate (α)	0.01	Higher than typical (0.0001) to accelerate convergence in traffic dynamics, since current inputs have moderate variance & not-so-complex scenarios

Noise clip	1e-6	Prevents numerical instability in log-prob calculations
Layer Sizes	32/64 neurons	Balancing complexity and computational efficiency

Table 6: Hyperparameters of SAC Network

Next, the agent.py initialises the network pieces from the previous file, and defines the behaviours of the SAC algorithm. Key points are:

- Twin Q-Networks – Stabilises training and reduces overestimation bias.
- Entropy Maximisation / Soft Updates – Adds entropy parameter to loss calculations to balance exploitation & exploration. Also automatically tunes this value for optimal trade-off.
- Learning calculations – explained in detail below.

Initially, the first network consisted of a shared feature extractor, branching into actor & value outputs, and a separate critic network, only one instead of two. This was intended to reduce computation. But that resulted in unstable outputs, and it was learned that value networks are no longer implemented in newer, more optimised networks.

Hyperparameters:

Parameter	Value	Rationale
Gamma	0.99	Discount factor, makes future rewards less significant compared to closer
Tau	0.005	Controls soft update to target networks
Target_entropy	-1.0	For entropy tuning

Table 7: Hyperparameters of agent

```
import tensorflow as tf
from tensorflow.keras.optimizers import Adam, SGD
import tensorflow_probability as tfp
import gym
import numpy as np
from network import SACNetwork, CriticNetwork
from replay_buffer import ReplayBuffer
import matplotlib.pyplot as plt
import os

tfd = tfp.distributions
tf.random.set_seed(42)

class ACAgent:
    def __init__(self, min_s, max_s, input_dims, r_scale, # n_actions,
                 env=None, tau = 0.005,
                 alpha=0.001, gamma=0.99): # Alpha used to be 0.0001, trying 0.01 now
        # Convert input_dims to integer if it's a sequence
        if isinstance(input_dims, (list, tuple)):
            input_dims = input_dims[0]

        # self.alpha = alpha # Alpha is default learning rate
        self.lr = 1e-4 # learning rate (used in optimizer)
        self.alpha_optimizer = Adam(learning_rate=self.lr, clipnorm=1.0)

        self.log_alpha = tf.Variable(0.0, trainable=True) # for entropy tuning
```

Fig. 12: Start of agent class

```

self.target_entropy = -1.0 # for 1D continuous action

self.gamma = gamma # Gamma is default discount factor
self.tau = tau # Soft update parameter
self.entropy_coef = 0.1 # Default entropy coefficient

buffer_input_shape = (input_dims,) if isinstance(input_dims, int) else input_dims # Handle both int and
self.memory = ReplayBuffer(max_size=100000, input_shape=buffer_input_shape)
self.batch_size = 30
self.reward_scale = 0.01
self.min_s = min_s
self.max_s = max_s

# self.n_actions = n_actions
self.action = None
self.action_space = gym.spaces.Box(low=np.array([min_s]), high=np.array([max_s]), dtype=np.float32)

```

Fig. 13: Parameters of agent class

```

self.ac = SACNetwork(min_s, max_s) # n_actions removed
self.ac.compile(optimizer=self.alpha_optimizer)

# Critic 1 and its target
self.critic1 = CriticNetwork()
self.critic1_target = CriticNetwork()
self.critic1.compile(optimizer=Adam(learning_rate=self.lr, clipnorm=1.0))
self.critic1_target.compile(optimizer=Adam(learning_rate=self.lr, clipnorm=1.0))
self.critic1_target.set_weights(self.critic1.get_weights())

# Critic 2 and its target
self.critic2 = CriticNetwork()
self.critic2_target = CriticNetwork()
self.critic2.compile(optimizer=Adam(learning_rate=self.lr, clipnorm=1.0))
self.critic2_target.compile(optimizer=Adam(learning_rate=self.lr, clipnorm=1.0))
self.critic2_target.set_weights(self.critic2.get_weights())

# Target value network for soft updates
self.target_value = SACNetwork(min_s, max_s)
self.target_value.compile(optimizer=Adam(learning_rate=self.lr, clipnorm=1.0))

```

Fig. 14: Network instances in agent class

```

# Build networks explicitly; should trigger build
test_state = tf.random.normal((1, input_dims))
test_action = tf.random.normal((1, 1))
_ = self.ac(test_state)
_ = self.target_value(test_state)
_ = self.critic1([test_state, test_action])
_ = self.critic2([test_state, test_action])

# Initialize target network weights to match main network
self.target_value.set_weights(self.ac.get_weights())
self.target_weights = self.target_value.get_weights()

self.r_scale = r_scale
self.update_params(tau=1) # update network parameters

self.summary_writer = tf.summary.create_file_writer("logs/")
self.total_steps = 0 # Count how many times learn() is called

```

Fig. 15: Builds & target networks taking weights

Below are the supporting methods:

- Choose_action() – Calls actor network & passes action

```
def choose_action(self, observation, evaluate=False):
    # Selects an action based on the current observation
    state = tf.convert_to_tensor([observation], dtype=tf.float32)
    mean, std = self.ac.get_policy_params(state)

    # Sample from the normal distribution
    dist = tfd.Normal(loc=mean, scale=std)
    action = mean if evaluate else dist.sample()

    action_value = float(action.numpy().item()) # Convert to Python
    return np.clip(action_value, self.min_s, self.max_s)
```

Fig. 16: choose_action() method

- Remember() – Stores state & action into replay buffer (explained next)

```
def remember(self, state, action, reward, n_state, done):
    # Store transition in replay buffer
    self.memory.store_transition(state, action, reward, n_state, done)
```

Fig. 17: remember() method

- Update_params() – Soft updates to the target network. Tau, the discount factor, controls this. The target networks are updated with only a part of the difference from the main networks, taking smaller steps towards the optimal values without convergence & instability.

```
def update_params(self, tau=None):
    # Soft update of target network parameters
    if tau is None:
        tau = self.tau

    # Get all weights from both networks
    main_weights = self.ac.get_weights()
    target_weights = self.target_value.get_weights()

    # Soft update: new_target = tau * main + (1-tau) * old_target
    updated_weights = []
    for w_main, w_target in zip(main_weights, target_weights):
        updated_weight = tau * w_main + (1 - tau) * w_target
        updated_weights.append(updated_weight)

    # Set the updated weights
    self.target_value.set_weights(updated_weights)

    for target, main in zip(self.critic1_target.weights, self.critic1.weights):
        target.assign(self.tau * main + (1 - self.tau) * target)

    for target, main in zip(self.critic2_target.weights, self.critic2.weights):
        target.assign(self.tau * main + (1 - self.tau) * target)
```

Fig. 18: update_params() method

- Save & load model methods


```

def save_models(self):
    print('Saving model.....')
    # self.ac.save_weights(self.ac.cp_file)
    path = self.ac.cp_dir
    os.makedirs(path, exist_ok=True)
    self.ac.save_weights(path + '/actor.weights.h5')
    self.critic1.save_weights(path + '/critic1.weights.h5')
    self.critic2.save_weights(path + '/critic2.weights.h5')
    self.critic1_target.save_weights(path + '/critic1_target.weights.h5')
    self.critic2_target.save_weights(path + '/critic2_target.weights.h5')
    print("✅ Models saved!")

def load_models(self):
    print('Loading model.....')
    # self.ac.load_weights(self.ac.cp_file)
    path = self.ac.cp_dir
    self.ac.load_weights(path + '/actor.h5')
    self.critic1.load_weights(path + '/critic1.h5')
    self.critic2.load_weights(path + '/critic2.h5')
    self.critic1_target.load_weights(path + '/critic1_target.h5')
    self.critic2_target.load_weights(path + '/critic2_target.h5')
    print("✅ Models loaded!")

```

Fig. 19: Save & Load_models() methods

- Learn() – the main method of the agent class that makes the networks learn. Logic explained in parts below.

```

def learn(self):
    # Check if enough samples are available in memory
    if self.memory.mem_cntr < self.batch_size:
        return

    # Sample batch
    states, actions, rewards, next_states, dones = self.memory.sample_buffer(self.batch_size)

    # Convert to tensors
    states = tf.convert_to_tensor(states, dtype=tf.float32)
    actions = tf.convert_to_tensor(actions, dtype=tf.float32)
    rewards = tf.convert_to_tensor(rewards, dtype=tf.float32)
    next_states = tf.convert_to_tensor(next_states, dtype=tf.float32)
    dones = tf.convert_to_tensor(dones, dtype=tf.float32)

```

Fig. 20: Start of learn() method, sampling batch to learn from

- Critic Updates – Calls both Q networks & takes minimum of the two, helps with stability. Entropy parameter added here to encourage exploration by penalizing low-entropy policies.

```

with tf.GradientTape(persistent=True) as tape:
    # Q1 and Q2 targets
    next_action, next_log_prob = self.ac.sample_normal(next_states)
    q1_next = self.critic1_target([next_states, next_action])
    q2_next = self.critic2_target([next_states, next_action])
    min_q_next = tf.minimum(q1_next, q2_next)

    # Target Q value using entropy
    q_target = rewards + self.gamma * (1 - dones) * (
        tf.squeeze(min_q_next) - self.alpha * tf.squeeze(next_log_prob)
    )

    # Current Q-values for critic 1
    q1_pred = tf.squeeze(self.critic1([states, tf.expand_dims(actions, -1)]), 1)
    critic1_loss = 0.5 * tf.reduce_mean((q1_pred - q_target)**2)
    # print("Q-values:", q1_pred.numpy()[0:5])

    # Current Q-values for critic 2
    q2_pred = tf.squeeze(self.critic2([states, tf.expand_dims(actions, -1)]), 1)
    critic2_loss = 0.5 * tf.reduce_mean((q2_pred - q_target)**2)

    # Gradients
    grad1 = tape.gradient(critic1_loss, self.critic1.trainable_variables)
    grad2 = tape.gradient(critic2_loss, self.critic2.trainable_variables)

    self.critic1.optimizer.apply_gradients(zip(grad1, self.critic1.trainable_variables))
    self.critic2.optimizer.apply_gradients(zip(grad2, self.critic2.trainable_variables))

```

Fig. 21: Critic networks updates

- Actor/Policy Updates – Maximises q-values, while maintaining high entropy.

```

with tf.GradientTape() as tape:
    # Sample new actions
    new_actions, log_probs = self.ac.sample_normal(states)

    # Get Q estimates
    q1 = tf.squeeze(self.critic1([states, new_actions]))
    q2 = tf.squeeze(self.critic2([states, new_actions]))
    q_values = tf.minimum(q1, q2)

    # Add safety critic to get safety risk of new_action
    safety_risk = self.ac.get_safety_q(tf.concat([states, actions], axis=1))

    # New total loss
    actor_loss = tf.reduce_mean(self.alpha * log_probs - q_values + self.lambda_safety * safety_risk)
    # print(f"Policy loss: {actor_loss.numpy():.6f}")

    # Define policy-specific variables (exclude value function)
    policy_variables = (
        self.ac.l1.trainable_variables +
        self.ac.l2.trainable_variables +
        self.ac.mean.trainable_variables +
        self.ac.std_dev.trainable_variables
    )

    policy_grads = tape.gradient(actor_loss, policy_variables)

```

Fig. 22: Policy updates Part 1

```

270     # Check if gradients are valid and apply them
271     # Clip gradients to prevent exploding gradients
272     if policy_grads and all(g is not None for g in policy_grads):
273         policy_grads = tf.clip_by_global_norm(policy_grads, 1.0)[0]
274         # print("Avg policy/actor grad norm:", np.mean([tf.norm(g).numpy() for g in policy_grads]))
275         # print("Before:", policy_variables[0].numpy().flatten()[:5])
276         self.ac.a_optimizer.apply_gradients(zip(policy_grads, policy_variables))
277         # print("After:", policy_variables[0].numpy().flatten()[:5])
278     else:
279         # print("Warning: Policy gradients are None!")
280         # Debug: print which gradients are None
281         for var, grad in zip(policy_variables, policy_grads):
282             if grad is None:
283                 print(f"No gradient for: {var.name}")
284
285     # 4. Update target networks
286     self.update_params()

```

Fig. 23: Policy updates Part 2

Entropy Value Tuning: Automatically adjusts to keep policy entropy near target_entropy.

```

# Automatic Entropy Tuning
with tf.GradientTape() as tape:
    alpha_loss = -tf.reduce_mean(
        self.log_alpha * tf.stop_gradient(log_probs + self.target_entropy)
    )
    alpha_grads = tape.gradient(alpha_loss, [self.log_alpha])
    self.alpha_optimizer.apply_gradients(zip(alpha_grads, [self.log_alpha]))

```

Fig. 24: Entropy update

Safety Update: Calculate safety. Use equation, apply gradients to Q_safety. Calculate unsafe actions, and update lambda to be bigger is many unsafe, and smaller otherwise.

```

# 1. Compute safety Q-values
q_safety_pred = self.ac.get_q_safety(tf.concat([states, actions], axis=1))
safety_costs = tf.squeeze(q_safety_pred, 1) # <-- this is now available globally in learn()

# 2. CVaR risk computation
with tf.GradientTape() as tape:
    sorted_costs = tf.sort(safety_costs)
    cvar_index = int(self.cvar_alpha * self.batch_size)
    cvar_loss = tf.reduce_mean(sorted_costs[-cvar_index:])

# Backprop
safety_variables = self.ac.safety_q.trainable_variables
grads = tape.gradient(cvar_loss, safety_variables)
self.ac.safety_q_optimizer.apply_gradients(zip(grads, safety_variables))

# Last block
unsafe_actions = tf.reduce_sum(tf.cast(safety_costs > self.safety_threshold, tf.float32))
too_many_unsafe = unsafe_actions > (0.3 * self.batch_size)

if too_many_unsafe:
    self.lambda_safety += 0.01
else:
    self.lambda_safety -= 0.01

self.lambda_safety = tf.clip_by_value(self.lambda_safety, 0.1, 10.0)

```

Fig. 25: Logic to update Safety parameter

Tensorboard was then used to observe the plot of parameter updates throughout the training process. The plots could be viewed on localhost after running the training script, and starting the local board by typing 'tensorboard --logdir runs/' in cmd.

```

with self.summary_writer.as_default():
    tf.summary.scalar('Critic1 Loss', critic1_loss, step=self.total_steps)
    tf.summary.scalar('Critic2 Loss', critic2_loss, step=self.total_steps)
    tf.summary.scalar('Actor Loss', actor_loss, step=self.total_steps)
    tf.summary.scalar('Reward', tf.reduce_mean(rewards), step=self.total_steps)
    tf.summary.scalar('Alpha (Entropy Coeff)', self.alpha, step=self.total_steps)

self.total_steps += 1

```

Fig. 26: Logging updates to TensorBoard

A few functions were defined in utils.py to plot training progress:

```

import numpy as np
import matplotlib.pyplot as plt

from scipy import stats

def plot_learning_curve(x, scores, figure_file):
    running_avg = np.zeros(len(scores))
    for i in range(len(running_avg)):
        running_avg[i] = np.mean(scores[max(0, i-100):(i+1)])
    plt.plot(x, running_avg)
    plt.title('Running average of previous 100 scores')
    plt.savefig(figure_file)

def plot_results(history):
    plt.figure(figsize=(10,5))
    plt.plot(history['train'], label='Training', marker='o')
    plt.plot(np.linspace(0, len(history['train'])-1, len(history['eval'])),
             history['eval'], 'r-', label='Evaluation', marker='o')
    plt.xlabel('Episode')
    plt.ylabel('Score')
    plt.legend()
    plt.savefig('training_progress.png')
    plt.show()

```

Fig. 27 & 28: Plotting functions Part 1

```

def plot_speeds_across_episodes(speed_records):
    # Convert to 2D arrays: one per episode
    episode_speeds = [np.vstack(ep) for ep in speed_records]
    n_segments = len(episode_speeds[0][0])
    # n_eps = len(episode_speeds)

    for seg in range(n_segments): # 6 segments
        plt.figure()
        for i, ep_log in enumerate(episode_speeds):
            speeds = [step_v[seg] for step_v in ep_log] # Collect
            plt.plot(speeds, label=f"Ep {i}")
        plt.title(f"Segment {seg+1} Speeds Across Valid Episodes")
        plt.xlabel("Timestep")
        plt.ylabel("Speed (km/h)")
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()

```

Fig. 29: Plotting functions Part 2

The replay_buffer file contained the class with methods to store state-action arrays & to sample batches.

```
class ReplayBuffer:
    def __init__(self, max_size, input_shape): #: #, n_actions
        self.input_shape = (input_shape,) if isinstance(input_shape, int) else input_shape
        self.mem_size = max_size
        self.mem_cntr = 0
        self.state_mem = np.zeros((self.mem_size, *input_shape))
        self.n_state_mem = np.zeros((self.mem_size, *input_shape))
        self.action_mem = np.zeros((self.mem_size)) #: #, n_actions
        self.reward_mem = np.zeros(self.mem_size)
        self.terminal_mem = np.zeros(self.mem_size, dtype=bool)

    def store_transition(self, state, action, reward, n_state, done):
        assert state.shape == self.state_mem[0].shape, "Mismatch in state shape"
        index = self.mem_cntr % self.mem_size
        self.state_mem[index] = state
        self.n_state_mem[index] = n_state
        self.reward_mem[index] = reward
        self.action_mem[index] = action
        self.terminal_mem[index] = done
        self.mem_cntr += 1

    def sample_buffer(self, batch_size):
        max_mem = min(self.mem_cntr, self.mem_size)
        batch = np.random.choice(max_mem, batch_size)
        states = self.state_mem[batch]
        n_states = self.n_state_mem[batch]
        actions = self.action_mem[batch]
```

Fig. 30: Replay buffer class

Now let's talk about the sim_env.py script, that holds the Metanet environment class. Key points are:

- I/O definitions
- Road structure definitions with vsl applied
- Definition of time period of the whole episode, of each step, logic for each step
- Modelling traffic patterns
- Gives rewards to the agent, indicating good & bad actions

It includes:

- Network creation from **nodes, links, ramps, and destinations**
- Simulation using **CasADi** for efficient math
- Traffic flow dynamics based on:
 - Speed
 - Density
 - Queue
 - On-ramp merging

Parameters:

Parameter	Meaning	Typical Range
rho_max	Max jam density (veh/km/lane)	140–200
rho_crit	Density where flow is max (before congestion)	~30–50
v_free	Free-flow speed (km/h)	90–120
a	Driver sensitivity (how quickly they respond)	1.5–2.0
tau	Reaction time (h)	~5–20 sec = 0.0014–0.005
kappa	Anticipation (how far ahead drivers look)	10–60
eta	Merge priority (higher = more aggressive merging)	50–80
delta	Smoothing for speed updates	0.01–0.1

Table 8: Simulation class parameters

So we start with defining the length of an ‘episode’ – the whole duration of the simulation setup. A ‘step’ would be a small part of the episode, the smallest unit that can be run. The number of segments being defined & the number of those on which VSL will be applied, is defined in the beginning to ensure consistent shapes throughout the script, so that changing the road structure etc. would require minimal changes. The initial current & previous speeds & VSL is defined with initial values. As the steps start taking place, these will be updated based on defined traffic pattern & outcomes of the steps.

```
import numpy as np
import casadi as cs
from sym_metanet import engines, Network, Node, Link, Destination, MainstreamOrigin, MeteredOnR
import sym_metanet
from data_demands import generate_demands # Import demand generation function

np.random.seed(42)

# MetaNetEnv: A traffic simulation environment using the MetaNet framework
class MetaNetEnv:
    def __init__(self, reward_scale=0.01):
        # Simulation parameters
        self.T = 10 / 3600 # Time step duration (10 seconds in hours)
        self.Tfin = 2.5 # Total simulation time in hours
        self.timesteps = int(self.Tfin / self.T) # Total number of steps to simulate = 900
        self.reward_scale = reward_scale # Scaling factor for reward calculation
        self.time = 0

        self.vsl_count = 2 # Number of VSL segments
        self.n_segments = 6 # Total number of segments in the network

        # Two VSL segments → two speed limits (Initial)
        self.current_action = np.array([80.0] * self.vsl_count)
        self.prev_action = np.array([80.0] * self.vsl_count)
        self.prev_v_vals = np.array([80.0] * self.n_segments)
```

Fig. 31: Start of MetaNetEnv class

The state dimensions is defined to be passed to the agent, telling the networks the dimensions of inputs to expect. Action range sets lower & upper bounds for the output. After some basic parameters for the road, demands are called, & the environment is reset & ready to start a new episode.

```
self.STATE_DIM = (self.vsl_count * 5 + 1) # 2 current VSLs + 2 previous VSLs +
self.ACTION_RANGE = [60, 120]

# Road parameters
self.L = 1 # Link length
self.lanes = 2 # Number of lanes

# Synthetic parameters
self.free_flow_speed = 120 # Max speed (used to normalize)
self.jam_density = 180 # Max density at which flow stops
self.critical_density = 33.5 # Density where flow is maximized

# Demand profile over time (vehicles entering network); inflow at points 01 & 02
self.demands = self.create_demands()

# Build the traffic network and compile the simulation function
self.build_network()

# Initialize state variables
self.reset()
```

Fig. 32: Road parameters

Below are the methods governing the behaviour of the environment:

- Create_demands() – Defines traffic demand patterns. Initially, synthetic sine waves were defined, and later, we derived demands by transforming the speed data collected.

```
def create_demands(self):
    # Simulates realistic dual-peak demand across simulation period

    # Create time-varying demand arrays using interpolation
    time = np.arange(0, self.Tfin, self.T)
    d1 = 2000 + 1500 * np.sin(2*np.pi*time/0.5)**2 # Mainstream; sinusoidal inflow at O1
    d2 = 500 + 300 * np.sin(2*np.pi*(time-0.25)/0.5) # On-ramp; sinusoidal inflow at O2

    return np.stack((d1, d2), axis=1) # Combine into a 2D array (900, 2)
```

Fig. 33: create_demands() method

- Build_network() – where road structure is defined. We start with basic parameters like the length & number of lanes, free flow speed, density, etc. We also define expected compliance of drivers to the speed limit, and their reaction time.

Entities in the below road network:

- Nodes: Connectors between road segments (N1, N2, N3)
- Origins & Destinations:
 - O1 (main origin), O2 (ramp origin)
 - D1 (destination)
- Links:
 - L1: Mainline with 4 segments, segments 2 and 3 have VSL control
 - L2: Downstream link with 2 segments

Road Network Layout:

The simulation models a simple highway segment using two links (L1 and L2) connected via nodes (N1, N2, N3). Vehicles enter from a **mainstream origin (O1)** and a **metered on-ramp (O2)** and exit at a **destination (D1)**. The network is defined symbolically and compiled using the CasADi framework.

```
def build_network(self):
    # Road and model parameters; Network constants from METANET literature
    # L = 1 # Link length (km)
    # lanes = 2 # Number of lanes per link
    rho_max = 180 # Maximum density (veh/km/lane)
    rho_crit = 33.5 # Critical density (veh/km/lane)
    v_free = 102 # Free-flow speed (km/h)
    a = 1.867 # Model parameter (driver sensitivity)
    C = (4000, 2000) # Max Capacity for origins (veh/h); O1, O2

    # MetaNet calibration parameters
    tau = 18 / 3600 # Relaxation time (h); Driver reaction delay
    kappa = 40 # Anticipation factor for downstream conditions
    eta = 60 # Merging priority (on-ramp vs mainline)
    delta = 0.0122 # Smoothing parameter

    # Define network structure using nodes and links
    N1 = Node(name="N1")
    N2 = Node(name="N2")
    N3 = Node(name="N3")

    O1 = MainstreamOrigin[cs.SX](name="O1")
    O2 = MeteredOnRamp[cs.SX](C[1], name="O2")
    D1 = Destination[cs.SX](name="D1")
```

Fig. 34: build_network() method, parameter declarations

```

96     # Define links with segment counts
97     # L1 = Link[cs.SX](4, lanes, L, rho_max, rho_crit, v_free, a, name="L1")
98     L1 = LinkWithVsl[cs.SX](4, self.lanes, self.L, rho_max, rho_crit, v_free, a,
99                             segments_with_vsl={2, 3}, alpha=0.1, name="L1")
100     L2 = Link[cs.SX](2, self.lanes, self.L, rho_max, rho_crit, v_free, a, name="L2")
101
102     # Define network layout (paths from origins to destinations)
103     self.net = (
104         Network(name="A1")
105         .add_path(origin=O1, path=(N1, L1, N2, L2, N3), destination=D1)
106         .add_origin(O2, N2)
107     )
108
109     # Compile the symbolic MetaNet model into a CasADi function
110     sym_metanet.engines.use("casadi", sym_type="SX")
111     self.net.is_valid(raises=True)
112     # self.net.step(T=self.T, tau=tau, eta=eta, kappa=kappa, delta=delta)
113     self.net.step(T=self.T, tau=tau, eta=eta,
114                  kappa=kappa, delta=delta,
115                  init_conditions={O1: {"v_ctrl": v_free}}))

```

Fig. 35: build_network() method, road structure definitions

- Reset() – resets variables to initial values.

```

def reset(self):
    # Reset simulation time
    self.time = 0
    self.current_action = np.array([120.0] * self.vsl_count)
    self.action = np.array([120.0, 120.0] * self.vsl_count)
    self.prev_v_vals = np.array([80.0] * self.n_segments)

    self.rho = cs.DM([22, 22, 22.5, 24, 30, 32]) # Need to change when n_segments changes
    self.v = cs.DM([80, 80, 78, 72.5, 66, 62])
    self.w = cs.DM([0, 0])

    return self._build_state()

```

Fig. 36: reset() method

- Step() – Runs one step with vsl input & gives output at the end. Stores previous values, before starting the step run. Prepares applied vsl, speeds, densities, current demands. Calls function to run step, and assigns new values after changes in the environment. Reward is calculated for every step.

```

184 def step(self, action):
185     # Expecting a list/array with 2 values (for segments 2 and 3)
186     if isinstance(action, (float, int)):
187         action = np.array([action, action])
188     elif isinstance(action, list):
189         action = np.array(action)
190     assert action.shape == (self.vsl_count,), f"Expected action shape ({self.vsl_count}), got {action.shape}"
191
192     self.prev_action = self.current_action
193     self.current_action = action
194     self.prev_v_vals = np.array(self.v).flatten()
195
196     v_ctrl1_O1 = cs.DM(action.reshape(-1, 1)) # (2, 1)
197     r_O2 = cs.DM([[1.0]])
198     u = cs.vertcat(v_ctrl1_O1, r_O2) # Now shape (3, 1)

```

Fig. 37: step() method Part 1

```

200 # Format inputs for MetaNet function
201 # print('Rho shape:', np.array(self.rho).shape, 'V shape:', np.array(self.v).shape, 'W shape:', np.array(self.w).shape)
202 x = cs.vertcat(self.rho, self.v, self.w) # State: densities, speeds, queues
203 d = cs.DM(self.demands[self.time]) # Demand: from O1 and O2
204 # print("Expected u shape:", self.F.size1_in(1))
205 # Run MetaNet step
206 x_next, _ = self.F(x, u, d) # Discard q_all (second output) for now
207
208 # Split next state
209 self.rho = x_next[0:6] # Must be changed when n_segment changes
210 self.v = x_next[6:12]
211 self.w = x_next[12:14]
212
213 # Construct new state, compute reward
214 next_state = self._build_state()
215 reward = self._compute_reward()
216
217 self.time += 1
218 done = self.time >= self.timesteps
219
220 return next_state, reward, done, {}

```

Fig. 38: step() method Part 2

- Build_state() – Normalises inputs & constructs current state vector.

State includes:

- Normalized current VSLs (2 values)
- Normalized previous VSLs (2 values)
- Current speeds at segments 2 & 3 (2 values)
- Previous speeds at those segments (2 values)
- Slightly noisy predicted future speeds (2 values)
- Incident flag (currently 0.01)

Total: **11-dimensional state vector.**

```

223 def _build_state(self):
224     try:
225         # Convert and flatten all inputs
226         current_vsl = np.asarray(self.current_action / 120).flatten()
227         prev_vsl = np.asarray(self.prev_action / 120).flatten()
228         current_speeds = np.asarray(self.v[2:4]).flatten() / self.free_flow_speed
229         prev_speeds = np.asarray(self.prev_v_vals[2:4]).flatten() / self.free_flow_speed
230         pred_speeds = current_speeds + np.random.normal(0, 1.0, size=current_speeds.shape).flatten() / self.free_flow_speed
231         incident = np.array([0.01]).flatten()
232         # densities = np.asarray(self.rho).flatten() / self.jam_density
233
234         # Verify shapes
235         assert current_vsl.shape == (self.vsl_count,), f"Bad current action shape: {current_vsl.shape}"
236         assert prev_vsl.shape == (self.vsl_count,), f"Bad prev action shape: {prev_vsl.shape}"
237         assert current_speeds.shape == (self.vsl_count,), f"Bad current speeds shape: {current_speeds.shape}"
238         assert prev_speeds.shape == (self.vsl_count,), f"Bad prev speeds shape: {prev_speeds.shape}"
239         assert pred_speeds.shape == (self.vsl_count,), f"Bad pred speeds shape: {pred_speeds.shape}"
240         assert incident.shape == (1,), f"Bad incident shape: {incident.shape}"
241
242         state = np.concatenate([
243             current_vsl,
244             prev_vsl,
245             current_speeds,
246             prev_speeds,
247             pred_speeds,
248             incident
249         ]).astype(np.float32)
250     except:
251         return state

```

Fig. 39: build_state() method

- Compute_reward() – Calculates reward for the step. Main aspect being total time spent by a vehicle of that modelled road stretch, & added higher speed encouragement.


```

262     def _compute_reward(self, baseline=False):
263         # Encourage lower total time spent (TTS) on road + in ramp queues
264         rho = np.clip(np.array(self.rho).flatten(), 0, self.jam_density)
265         w = np.clip(np.array(self.w).flatten(), 0, 1000) # Arbitrary large queue limit
266
267         # Calculate veh-hours for highway and on-ramp queues, with numerical safeguards
268         highway = np.sum(rho * self.L * self.lanes) * self.T
269         queue = np.sum(w) * self.T
270
271         # Combined and scaled
272         total_hours = highway + queue
273         # reward = -np.tanh(total_hours * self.reward_scale)
274         # reward = -total_hours * self.reward_scale # Scale reward
275         # reward = total_hours * self.reward_scale
276         reward = -total_hours
277         # reward = total_hours
278         # reward = (-total_hours + 2)
279         # reward = (-total_hours + 1)
280         # reward = (-total_hours + 1) * 10
281         # reward = (-total_hours + 0.5)
282         # reward = (-total_hours + 0.5) * 10 # Negative TTS as reward
283         # reward = 1/total_hours
284
285         # Encourage higher VSLs when traffic is not congested
286         vsl_term = np.mean(self.current_action[0]) / self.free_flow_speed # Normalize to [0,1]
287         reward += vsl_term * 1
288
289         # NaN check and fallback

```

Fig. 40: compute_reward() method

- Train.py –It initializes the environment and agent, runs training over multiple episodes, logs actions, speeds, and rewards, and plots the results.

After initiation, number of episodes to run is defined. The outer loop is for the episodes, and the inner one is for steps of an episode. Within it, the agent takes steps. Since reactions of an applied VSL won't be visible in one step of a few seconds, we accumulate the rewards & store the sum when step % control interval = 0. In each step, model outputs a limit & that is applied to the simulation step (between learn() runs, this value stays almost same). We log actions, speeds & rewards.

<pre> # Initialize environment and agent env = MetaNetEnv() agent = ACAgent(min_s=60, # Minimum speed limit (km/h) max_s=120, # Maximum speed limit (km/h) input_dims=env.STATE_DIM, # State dimensions r_scale=0.01, # Reward scaling factor tau=0.005, # Soft update parameter alpha=3e-4, # Learning rate gamma=0.99 # Discount factor) n_eps = 18 # Number of episodes to train control_interval = 30*10 # Steps between speed limit changes history = {'train': [], 'eval': []} agent.speed_logs = [[] for _ in range(n_eps)] # One empty list </pre>	<pre> # Main training loop for ep in range(n_eps): state = env.reset() done = False score = 0 step = 0 current_action = 120 vsl_30steps = [] reward_30steps = 0 # Warmup network # _ = agent.ac(tf.conve nan_detected = False scores = [] # Store sc </pre>
--	---

Fig. 41 & 42: train.py file initializing model & environment, running episodes

```

vsl_30steps.append(action)
next_state, reward, done, _ = env.step(action) # Take step
score += reward # Update score
reward_30steps += reward
scores.append(score) # Store score for this episode

# Debug prints to observe values
if step % 50 == 0:
    print(
        # f"Step {step}: VSL {action:.1f}, Reward {reward:.2f}, "
        # f"Speeds {np.array(env.v).flatten()[::2]}"
        f"Step {step}: "
        f"VSL {action}, "
        f"VSL_prev: {env.prev_action[0]}, "
        f"Reward {reward:.2f}, "
        f"Speeds: {np.round(np.array(env.v[2:4]).flatten(), 1)}, "
        f"Prev Speeds: {np.round(env.prev_v_vals[2:4].flatten(), 1)}, "
        # f"Pred Speeds: {np.round((np.array(env.v[0:2]).flatten() + np.random.
        # f"Incidents: {[0.01]*env.n_segments}"
    )

# NaN detection
if np.isnan(reward) or np.isnan(action):
    print(f"NaN detected at ep {ep}, step {step}!")
    nan_detected = True
    break

```

Fig. 43: Keeping track of states, actions & rewards in training

```

# Loops through steps in the episode
while not done:
    # Take action every 'control_interval' steps
    if step % control_interval == 0 and step != 0:
        # Store transition into agent memory
        avg_speed = sum(vsl_30steps) / len(vsl_30steps)
        print('Updating memory - avg speed: ', avg_speed, '    Reward for 30 steps: ', reward_30steps)
        agent.memory.store_transition(
            np.array(state, dtype=np.float32),
            avg_speed,
            reward_30steps,
            np.array(next_state, dtype=np.float32),
            done
        )
        agent.learn()
        reward_30steps = 0
        vsl_30steps = []
        action = agent.choose_action(state)
        # print(f"Action chosen: {action:.1f} at step {step}")
        prev_action = current_action
        current_action = action
    else:
        action = current_action # Use the last action for the next steps

```

Fig. 44: Running training in loop, storing accumulated rewards

```

# Update state and step count; move on to next step
state = next_state
step += 1

agent.speed_logs[ep].append(np.array(env.v).flatten())

# End of episode
# Learning and evaluation
if not nan_detected:
    agent.learn()
    history['train'].append(score)
    # print('Score list:', scores) # Debug print

# Debug: print mean and std of policy parameters
state_tensor = tf.convert_to_tensor([state], dtype=tf.float32)
mean, std = agent.ac.get_policy_params(state_tensor)
print(f"Mean: {mean.numpy().item():.2f}, Std: {std.numpy().item():.2f}") # Debug print

# print(state, action, score, current_action)

```

Fig. 45: Debug prints while training

Points were picked up from Google Maps to obtain coordinates of desired roads in an area, and the following website was used to visualize the points from the api response on the map & get total distance of the tile - <https://www.cachesleuth.com/distancemultiplepoints.html>

Data Collection

Three endpoints were needed to get data for one segment:

- Speed-related data from TomTom Flow Segment Data Endpoint
- Incidents from TomTom Incident Details Endpoint
- Traffic levels from Azure's Get Traffic Flow Tile Endpoint

All of these required latitude/longitude pairs, along with a 'zoom level' - to determine the radius around the given points to cover. This value (1-20) did not affect the results much, but we kept it as 15, which was said to cover ~2.5km.

The first speeds endpoint returns current speed, free flow speed, current & free flow travel time, confidence of the data, and the coordinates of the start & end of the road segment. The intended length was 1.5km, but upon testing different coordinates, the lengths were inconsistent, ranging from 100m to 10km. Reasonable segments were put together.

```
# Flow Segment Data - speed, TTS, coords
zoom = 15
xcor, ycor = 25.231400, 55.40425
url = f'https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/{zoom}/json?key={api_key}&point={xcor},{ycor}'
response = requests.get(url)
response.text

'{"flowSegmentData":{"frc":"FRC1","currentSpeed":86,"freeFlowSpeed":86,"currentTravelTime":12,"freeFlowTravelTime":12,"confidence":1,"roadClosure":false,"coordinates":{"coordinate":
[{"latitude":25.229934973001065,"longitude":55.404397369223744}, {"latitude":25.230723530384278,"longitude":55.404278010922525}, {"latitude":25.230899211173920,"longitude":55.404257894
354913}, {"latitude":25.232053906084850,"longitude":55.404117078381603}, {"latitude":25.232602399385829,"longitude":55.404086232977932}], "version": "4"}}
```

Fig. 46: Flow Segment Data endpoint outputs

Next is incident details, which requires a bbox of 2 pairs of coordinates, so the first & last of the output of the speed response was used. It returned an array of incidents, for each mentioned was the magnitude of delay, time delay if available, type of incident (collision, road repairs, etc.), along with a list of coordinates of the affected areas.

```
# Incident Details - Intensity
lower_left_x, lower_left_y = 25.019163, 55.159927 # 1km past start
upper_right_x, upper_right_y = 25.042877, 55.197986 # 1km past end
fields = {'incidents': {'type': 'Feature', 'properties': {'iconCategory': 8, 'magnitudeOfDelay': 4, 'delay': null, 'probabilityOfOccurrence': 'certain'}, 'geometry': {'type': 'LineString', 'coordinates':
[[[55.1784145541, 25.0348672757], [55.1785660990, 25.0349101842], [55.1787364192, 25.0349906091], [55.1790864475, 25.0351985442], [55.1794753678, 25.0354305531], [55.1799045213, 25.0357094948],
[55.1801955409, 25.0358891775], [55.1803846367, 25.0360166109], [55.1806971140, 25.0362284932], [55.1807936735, 25.0362566682]]]}}, {'type': 'Feature', 'properties': {'iconCategory': 6, 'magnitud
OfDelay': 1, 'delay': 66, 'probabilityOfOccurrence': 'certain'}, 'geometry': {'type': 'LineString', 'coordinates': [[55.1960728772, 25.0485934344], [55.1962217398, 25.0481254414], [55.1968440123,
5.0469210896], [55.1970840700, 25.0465616801], [55.1971189387, 25.0464812628], [55.1972020872, 25.0462813962], [55.1972785301, 25.0461017287], [55.1973241277, 25.0459260096], [55.1973402210, 25
0457543150], [55.1973455854, 25.0456416996], [55.1973013289, 25.0454579304], [55.1971578307, 25.044... ]}}
```

Fig. 47: Incident Details endpoint outputs

For the traffic levels, the tiles endpoint in TomTom was not giving desired results, so Azure was used. There was pre & post-processing required to bring the responses back into the correct format. Several functions were defined to deal with this:

- Coordinates – Unlike lat/long, this endpoint divided the world map into 'tiles', so it had a separate system of x/y values. So two functions were defined, one to turn input data into x & y before the call, and one after to convert them back.

```
[ ] import math
import requests
import mapbox_vector_tile
import io
import traceback

# --- 1. Lat/Lon to Tile X, Y (unchanged - this gets the tile index) ---
def lat_lon_to_tile(lat, lon, zoom):
    n = 2 ** zoom
    xtile = int(n * ((lon + 180) / 360))
    ytile = int(n * (1 - (math.log(math.tan(math.radians(lat)) + 1 / math.cos(math.radians(lat))) / math.pi)) / 2)
    return xtile, ytile

# --- 2. GLOBAL Pixel Coordinates to Lat/Lon (unchanged - this handles the final projection) ---
def pixel_to_lat_lon(pixel_x, pixel_y, zoom):
    total_pixels_at_zoom = 256 * (2 ** zoom)
    norm_x = pixel_x / total_pixels_at_zoom
    norm_y = pixel_y / total_pixels_at_zoom
    lon_deg = norm_x * 360.0 - 180.0
    n_val = math.pi - (2.0 * math.pi * norm_y)
    lat_rad = math.atan(0.5 * (math.exp(n_val) - math.exp(-n_val)))
    lat_deg = math.degrees(lat_rad)
    return lat_deg, lon_deg
```

Fig. 48: Helper functions to convert coordinates

- Format Conversion & alignment – The response itself comes in Protocol Buffer (pbf) format (json was not an available choice), so this was decoded using the mapbox_vector_tile library. The 'geometry' attribute of the data indicated the area covered by that tile. Many formatting steps were written, addressing every type of geometry the coordinates could correspond to.

```
# --- 3. Function to GET and PARSE PBF and CONVERT GEOMETRY (NEW REVISION with extent scaling) ---
def get_and_process_traffic_tile_final_v2(subscription_key, lat, lon, zoom):
    xtile, ytile = lat_lon_to_tile(lat, lon, zoom)

    print(f"Requesting PBF tile for Lat: {lat}, Lon: {lon}, Zoom: {zoom}")
    print(f"Calculated Tile X: {xtile}, Y: {ytile}")

    url = f"https://atlas.microsoft.com/traffic/flow/tile/pbf?api-version=1.0&style=relative&zoom={zoom}&x={xtile}&y={ytile}&subscription-key={subscription_key}"
    response = requests.get(url)

    if response.status_code == 200:
        pbf_data = response.content
        features_with_traffic_data = []
        try:
            decoded_data = mapbox_vector_tile.decode(pbf_data)

            if 'Traffic flow' in decoded_data:
                traffic_layer = decoded_data['Traffic flow']

                # --- CRUCIAL: Get the extent from the layer definition ---
                # Your debug output showed: {'Traffic flow': {'extent': 4096, ...}}
                extent = traffic_layer.get('extent', 4096) # Default to 4096 if not found for some reason

                for feature_idx, feature in enumerate(traffic_layer['features']):
                    properties = feature['properties']
                    geometry_obj = feature['geometry']

                    converted_geometry = []

                    if not isinstance(geometry_obj, dict) or 'type' not in geometry_obj or 'coordinates' not in geometry_obj:
                        print(f"Warning: Malformed geometry object for feature {feature_idx}: {geometry_obj}")
                        continue
```

Fig. 49: Tile processing function Part 1

```
for feature_idx, feature in enumerate(traffic_layer['features']):
    properties = feature['properties']
    geometry_obj = feature['geometry']

    converted_geometry = []

    if not isinstance(geometry_obj, dict) or 'type' not in geometry_obj or 'coordinates' not in geometry_obj:
        print(f"Warning: Malformed geometry object for feature {feature_idx}: {geometry_obj}")
        continue

    geo_type = geometry_obj['type']
    geo_coords_raw = geometry_obj['coordinates'] # These are the [2382, 3822] type coords

    def convert_single_point(pbf_x, pbf_y):
        # Step 1: Scale from extent-based to 256x256 tile pixels
        pixel_in_tile_x = (pbf_x / extent) * 256
        pixel_in_tile_y = (pbf_y / extent) * 256

        # Step 2: Convert to global pixel coordinates
        global_px = (xtile * 256) + pixel_in_tile_x
        global_py = (ytile * 256) + pixel_in_tile_y

        # Step 3: Convert global pixel to Lat/Lon
        return pixel_to_lat_lon(global_px, global_py, zoom)

    if geo_type == 'Point':
        px_raw, py_raw = geo_coords_raw
        conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
        converted_geometry.append((conv_lon, conv_lat))
```

Fig. 50: Tile processing function Part 2

```

if geo_type == 'Point':
    px_raw, py_raw = geo_coords_raw
    conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
    converted_geometry.append([conv_lon, conv_lat])
elif geo_type == 'LineString':
    line_coords = []
    for px_raw, py_raw in geo_coords_raw:
        conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
        line_coords.append([conv_lon, conv_lat])
    converted_geometry.append(line_coords)
elif geo_type == 'Polygon':
    polygon_rings = []
    for ring in geo_coords_raw:
        ring_coords = []
        for px_raw, py_raw in ring:
            conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
            ring_coords.append([conv_lon, conv_lat])
        polygon_rings.append(ring_coords)
    converted_geometry.append(polygon_rings)
elif geo_type == 'MultiPoint':
    for px_raw, py_raw in geo_coords_raw:
        conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
        converted_geometry.append([conv_lon, conv_lat])
elif geo_type == 'MultilineString':
    multi_line_coords = []
    for line in geo_coords_raw:
        single_line_coords = []
        for px_raw, py_raw in line:
            conv_lat, conv_lon = convert_single_point(px_raw, py_raw)
            single_line_coords.append([conv_lon, conv_lat])
        multi_line_coords.append(single_line_coords)

```

Fig. 51: Tile processing function Part 3

During this step, 3 areas were decided to be included – E311 near Motor City, E311 from Mirdif, and Business Bay Bridge towards Deira, covering ~5km each. Each area had 2-3 segments, their structures were noted & modelled as follows:

- MCity: Part of Sheikh Mohammed Bin Zayed Road near Motor City, Dubai, UAE. It is a large 8-lane road with 3 segments considered, each ~1.5 km long.
 - Segment 1:
 - outgoing ramp - beginning of segment 1
 - incoming ramp - end of segment 1
 - Segment 2:
 - outgoing ramp - 500m into segment 2
 - incoming ramp - end of segment 2
 - Segment 3:
 - outgoing ramp - 500m into segment 3
- Deira - 2 segments considered, the first segment is 3km long, and second is 5km long. Uniform lengths & segments were not possible due to data API limits.
 - Segment 1:
 - outgoing ramp - 1.7km
 - incoming ramp - 2.2km
 - incoming ramp - 2.3km
 - division - 2.5km (could be modelled as a larger outgoing ramp)
 - Segment 2:
 - incoming ramp - 1km
 - outgoing ramp - 1.2km
 - incoming ramp - 2.1km
 - outgoing ramp - 2.7km
 - incoming ramp - 3.5km
 - incoming ramp - 3.5km
 - division - 3.9km (could be modelled as a larger outgoing ramp)
 - join - 4km (could be modelled as a larger incoming ramp)
 - outgoing ramp - 4.4km
 - incoming ramp - 4.8km

- MohdRoad – Part of Sheikh Mohammed Bin Zayed Road near Mirdif & Muhaisnah.
 - Segment 1:
 - outgoing ramp - beginning of segment 1
 - incoming ramp - end of segment 1
 - Segment 2:
 - outgoing - 700m from start of segment 2
 - incoming - 1.8km
 - outgoing - 3.1km
 - outgoing - 3.7km
 - Segment 3:
 - outgoing - Beginning of segment 3
 - incoming - 1km
 - incoming - 1.1km

This data needed to be run every time we needed a sample, and it was to be collected every 1 minute during certain windows of the day. These steps were restructured into a script that was created as a function on Google Cloud, and 2 jobs were created on Scheduler to call them every minute, and store the data in a Google Sheet. All the required coordinates were defined there, and the API keys were defined outside the script in the environment, for privacy. It was organized to request & pass on only required info, minimizing RAM usage.

```

traffic-collector-function > main.py > get_traffic_flow_for_bbox_and_filter
31 TQNTQM_KEY3 = os.getenv("TQNTQM_KEY3")
32
33 ZOOM_LEVEL = 15
34
35 LOCATIONS_CONFIG = { # Each location covers 5km
36     "city": { # Motor City - with 3 segments
37         "incident_bbox": [25.017388, 55.158913, 25.040493174160975, 55.187995404747312],
38         "segments": [
39             {"name": "Seg1", "coords": [25.017388, 55.158913]},
40             {"name": "Seg2", "coords": [25.027700, 55.168197]},
41             {"name": "Seg3", "coords": [25.040389, 55.187802]}
42         ],
43         "fixed_tile_bbox": {
44             "min_lon": 55.154870123405658,
45             "min_lat": 25.011286592755695,
46             "max_lon": 55.163085729620207,
47             "max_lat": 25.023282797416908
48         },
49         "main_road_geometry": [
50             [55.154870123405658, 25.011286592755695],
51             [55.155727, 25.012557],
52             [55.156151, 25.013203],
53             [55.156566, 25.013889],
54             [55.157008, 25.014568],
55             [55.157380, 25.015111],
56             [55.157639, 25.015466],
57             [55.157961, 25.016034],
58             [55.158188, 25.016277],

```

Fig. 52: Automation script Part 1

```

286 # --- Dynamic Header Generation ---
287 def generate_sheet_headers():
288     headers = ['timestamp']
289
290     # Headers for Flow Segment Data (Speed, Travel Time, Confidence)
291     segment_data_types = [
292         'CurrentSpeed', 'FreeFlowSpeed', 'CurrentTravelTime', 'FreeFlowTravelTime', 'FlowsSegmentConfidence'
293     ]
294
295     # Headers for Incident Data
296     incident_data_types = [
297         'Incident_MagnitudeDelayList', 'Incident_DelayList', 'Incident_ProbabilityList', 'Incident_GeometryList'
298     ]
299
300     for loc_name in LOCATIONS_CONFIG:
301         # Add headers for aggregated segment data (one column per location)
302         for data_type in segment_data_types:
303             headers.append(f"{loc_name}_{data_type}")
304
305         # Add headers for incident data (one column per location)
306         for data_type in incident_data_types:
307             headers.append(f"{loc_name}_{data_type}")
308             headers.append(f"{loc_name}_trafficLevelList")
309     return headers
310
311 SHEET_HEADERS = generate_sheet_headers()

```

Fig. 53: Automation Script Part 2

```

313 def get_flow_segment_data(xcor, ycor):
314     url = f'https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/{ZOOM_LEVEL}/json?key={TOMTOM_KEY}&point={xcor},{ycor}'
315     response = requests.get(url)
316     time.sleep(0.2)
317     if response.status_code != 200:
318         url = f'https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/{ZOOM_LEVEL}/json?key={TOMTOM_KEY2}&point={xcor},{ycor}'
319         response = requests.get(url)
320         time.sleep(0.2)
321     if response.status_code != 200:
322         url = f'https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/{ZOOM_LEVEL}/json?key={TOMTOM_KEY3}&point={xcor},{ycor}'
323         response = requests.get(url)
324         time.sleep(0.2)
325     return response
326
327 def get_incident_details(lower_left_x, lower_left_y, upper_right_x, upper_right_y):
328     fields = '{incidents(type,geometry(type,coordinates),properties(iconCategory,magnitudeOfDelay,delay,probabilityOfOccurrence))}'
329     url = f'https://api.tomtom.com/traffic/services/5/incidentDetails?key={TOMTOM_KEY}&bbox={lower_left_x},{lower_left_y},{upper_right_x},{upper_right_y}'
330     response = requests.get(url)
331     time.sleep(0.2)
332     return response
333
334 # --- 1. Lat/Lon to Tile X, Y ---
335 def lat_lon_to_tile(lat, lon, zoom):
336     n = 2 ** zoom

```

Fig. 54: Automation Script Part 3, calling endpoints with keys

```

425 # --- MASTER FUNCTION: GET AND PROCESS TRAFFIC FOR A GIVEN BOUNDING BOX AND FILTER ---
426 # Now returns only the list of traffic levels
427 def get_traffic_flow_for_bbox_and_filter(min_lon, min_lat, max_lon, max_lat, zoom,
428                                         azure_key, main_road_geojson_linestring_coords,
429                                         buffer_degrees=0.00005, delay_per_tile=0.05):
430     print(f"Calculating tiles for BBOX: [{min_lon},{min_lat},{max_lon},{max_lat}] at Zoom {zoom}...")
431     tiles_to_query = list(mercantile.tiles(min_lon, min_lat, max_lon, max_lat, zooms=[zoom]))
432     print(f"Found {len(tiles_to_query)} tiles to query.")
433
434     all_filtered_traffic_levels = []
435
436     # --- Prepare the main road geometry for efficient filtering ---
437     prepared_main_road = None
438     try:
439         main_road_line_shapely = LineString(main_road_geojson_linestring_coords)
440         if buffer_degrees > 0:
441             main_road_line_shapely = main_road_line_shapely.buffer(buffer_degrees)
442             print(f"Main road geometry buffered by {buffer_degrees} degrees for filtering.")
443
444         prepared_main_road = prep(main_road_line_shapely)
445         print(f"Main road geometry prepared for efficient spatial filtering (length: {main_road_line_shapely.length:.4f} degrees)")
446     except Exception as e:
447         print(f"ERROR: Could not create Shapely LineString for main road. Check your MAIN_DIAGONAL_ROAD_GEOMETRY format: {e}")
448
449     if not prepared_main_road:

```

Fig. 55: Automation Script Part 4, traffic_level function

```

452 for i, tile in enumerate(tiles_to_query):
453     xtile, ytile, current_zoom = tile
454     print(f"--- Processing Tile {i+1}/{len(tiles_to_query)}: x={xtile}, y={ytile}, z={current_zoom} ---")
455
456     tile_features_for_filter = _get_and_process_single_traffic_tile(xtile, ytile, current_zoom, azure_key)
457
458     if tile_features_for_filter:
459         if prepared_main_road: # Only attempt spatial filtering if main road is prepared
460             for feature_data in tile_features_for_filter:
461                 feature_geometry_raw = feature_data['Geometry']
462                 original_type = feature_data['Properties'].get('original_pbf_geo_type')
463
464                 shapely_feature = None
465                 try:
466                     if original_type == 'LineString':
467                         if feature_geometry_raw and isinstance(feature_geometry_raw[0], list):
468                             shapely_feature = LineString(feature_geometry_raw[0])
469                         else:
470                             shapely_feature = LineString(feature_geometry_raw)
471                     elif original_type == 'MultiLineString':
472                         shapely_feature = MultiLineString(feature_geometry_raw)
473
474                     if shapely_feature and prepared_main_road.intersects(shapely_feature):
475                         traffic_level = feature_data['Properties'].get('traffic_level')
476                         if traffic_level is not None:

```

Fig. 56: Automation Script Part 5, processing outputs


```

493 # --- Helper function to convert collected features to GeoJSON FeatureCollection ---
494 def features_to_geojson_feature_collection(features):
495     geojson_features = []
496     for feature in features:
497         props = feature['Properties']
498         geom_raw = feature['Geometry']
499         original_type = props.get('original_pbf_geo_type')
500
501         geojson_geom = {}
502         if original_type == 'Point':
503             geojson_geom = {'type': 'Point', 'coordinates': geom_raw[0]}
504         elif original_type == 'LineString':
505             geojson_geom = {'type': 'LineString', 'coordinates': geom_raw[0]}
506         elif original_type == 'MultiLineString':
507             geojson_geom = {'type': 'MultiLineString', 'coordinates': geom_raw}
508         elif original_type == 'Polygon':
509             geojson_geom = {'type': 'Polygon', 'coordinates': geom_raw[0]}
510         elif original_type == 'MultiPoint':
511             geojson_geom = {'type': 'MultiPoint', 'coordinates': geom_raw}
512         elif original_type == 'MultiPolygon':
513             geojson_geom = {'type': 'MultiPolygon', 'coordinates': geom_raw}
514         else:
515             if geom_raw and isinstance(geom_raw, list) and geom_raw:
516                 if isinstance(geom_raw[0], list) and len(geom_raw[0]) >= 2 and isinstance(geom_raw[0][0], (int, float)):
517                     geojson_geom = {'type': 'LineString', 'coordinates': geom_raw[0]}
518                 elif isinstance(geom_raw[0], list) and isinstance(geom_raw[0][0], list) and len(geom_raw[0][0]) >= 2 and isinstan

```

Fig. 57: Automation Script Part 6

```

538 # --- Helper function to convert a LineString list to GeoJSON Feature ---
539 def linestring_to_geojson_feature(coords, properties=None):
540     if properties is None:
541         properties = {}
542     return {
543         "type": "Feature",
544         "properties": properties,
545         "geometry": {
546             "type": "LineString",
547             "coordinates": coords
548         }
549     }
550
551 # --- Cloud Function Entry Point ---
552
553 def traffic_data_collector(request):
554     """
555     Google Cloud Function that collects traffic data and appends it to a Google Sheet.
556     This function is triggered by an HTTP request (e.g., from Cloud Scheduler).
557     """
558     print(f"Function triggered at: {datetime.datetime.now()}")
559
560     # 1. Initialize Google Sheets API client
561     # The Cloud Function's default service account will be used for authentication.
562     # Ensure this service account has 'Editor' access to your Google Sheet.
563     try:

```

Fig. 58: Automation Script Part 7, endpoint

```

try:
    # Get current timestamp in Dubai timezone
    dubai_now = datetime.datetime.now(pytz.timezone('Asia/Dubai'))
    timestamp_str = dubai_now.strftime('%Y-%m-%d %H:%M:%S')
    all_results['timestamp'] = timestamp_str

    for loc_name, config in LOCATIONS_CONFIG.items():
        # Initialize lists to aggregate segment data if multiple segments exist
        loc_current_speeds = []
        loc_free_flow_speeds = []
        loc_current_travel_times = []
        loc_free_flow_travel_times = []
        loc_confidences = []

        # 1. Collect Flow Segment Data for each segment
        for segment in config["segments"]:
            try:
                # Make API call, then parse response and check for errors
                flow_response = get_flow_segment_data(segment['coords'][0], segment['coords'][1])
                flow_response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
                flow_segment_data = flow_response.json() # Parse JSON

                # Ensure 'flowSegmentData' key exists as per TomTom API structure
                speed_data = flow_segment_data.get('flowSegmentData', {})

```

Fig. 59: Automation Script Part 8, storing responses


```

# Aggregate segment data into comma-separated strings for this location
all_results[f"{loc_name}_CurrentSpeed"] = ", ".join(loc_current_speeds)
all_results[f"{loc_name}_FreeFlowSpeed"] = ", ".join(loc_free_flow_speeds)
all_results[f"{loc_name}_CurrentTravelTime"] = ", ".join(loc_current_travel_times)
all_results[f"{loc_name}_FreeFlowTravelTime"] = ", ".join(loc_free_flow_travel_times)
all_results[f"{loc_name}_FlowsSegmentConfidence"] = ", ".join(loc_confidences)

# 2. Collect Incident Data for the location's bounding box
incident_bbox = config["incident_bbox"]
mag_delay_values = []
delay_values = []
prob_values = []
geometry_values = []

try:
    incident_response = get_incident_details(
        incident_bbox[0], incident_bbox[1], incident_bbox[2], incident_bbox[3]
    )
    incident_response.raise_for_status() # Raise HTTPError for bad responses
    incident_data_json = incident_response.json() # Parse JSON

    # TomTom incident response typically has a top-level 'incidents' array
    incidents_list = incident_data_json.get('incidents', [])

    for incident in incidents_list:
        properties = incident.get('properties', {})

```

Fig. 60: Automation Script Part 9

```

553 def traffic_data_collector(request):
554     all_results[f"{loc_name}_IncidentProbabilityList"] = ", ".join(prob_values)
555     all_results[f"{loc_name}_IncidentGeometryList"] = " | ".join(geometry_values) # Use a clear separator for
556
557     BUFFER_SIZE_DEGREES = 0.00005 # Adjust this buffer as needed for proximity
558     # Call the master function
559     traffic_levels = get_traffic_flow_for_bbox_and_filter(
560         min_lon=config["fixed_tile_bbox"]["min_lon"],
561         min_lat=config["fixed_tile_bbox"]["min_lat"],
562         max_lon=config["fixed_tile_bbox"]["max_lon"],
563         max_lat=config["fixed_tile_bbox"]["max_lat"],
564         zoom=ZOOM_LEVEL,
565         azure_key=AZURE_PKEY,
566         main_road_geojson_linestring_coords=config['main_road_geometry'],
567         buffer_degrees=BUFFER_SIZE_DEGREES,
568         delay_per_tile=0.05
569     )
570     all_results[f"{loc_name}_trafficLevelList"] = ", ".join(map(str, traffic_levels))
571
572     # 3. Prepare the final row_values list based on the SHEET_HEADERS order
573     final_row_values = []
574     for header in SHEET_HEADERS:
575         # Standardize header to match all_results key format: lowercase location prefix
576         # key_in_all_results = header.replace(header.split('_')[0], header.split('_')[0].lower(), 1)
577
578         final_row_values.append(all_results.get(header, 'N/A'))

```

Fig. 61: Automation Script Part 10

Filter	Name	Deployment type	Req/sec	Region	Authentication	Ingress	Last deployed	Deployed by	Recommendation
☐	flow-points-collector-v1	Function	0	us-central1	Public access	All	7 days ago	Cloud Run functions	Security
☐	traffic-data-collector1	Function	0	us-central1	Public access	All	Jun 24, 2025	Cloud Run functions	Security

Fig. 62: Cloud Functions

Filter	Name	Status of last execution	Region	State	Description	Frequency	Target	Last run	Next run	Last updated	Actions
☐	collect-traffic-4pm-8pm	Has not run yet	us-central1	Paused		* 16-20 *** (Asia/Dubai)	URL: https://us-central1-vsl-data-463507.cloudfunctions.net/flow_points_collector_v1	Jul 22, 2025, 4:36:00 PM		Jul 22, 2025, 4:37:05 PM	
☐	collect-traffic-5am-9am	Has not run yet	us-central1	Paused		* 6-12 *** (Asia/Dubai)	URL: https://us-central1-vsl-data-463507.cloudfunctions.net/flow_points_collector_v1	Jul 17, 2025, 12:37:00 PM		Jul 17, 2025, 12:37:30 PM	

Fig. 63: Scheduled Jobs

Here are the points from the responses visualised:

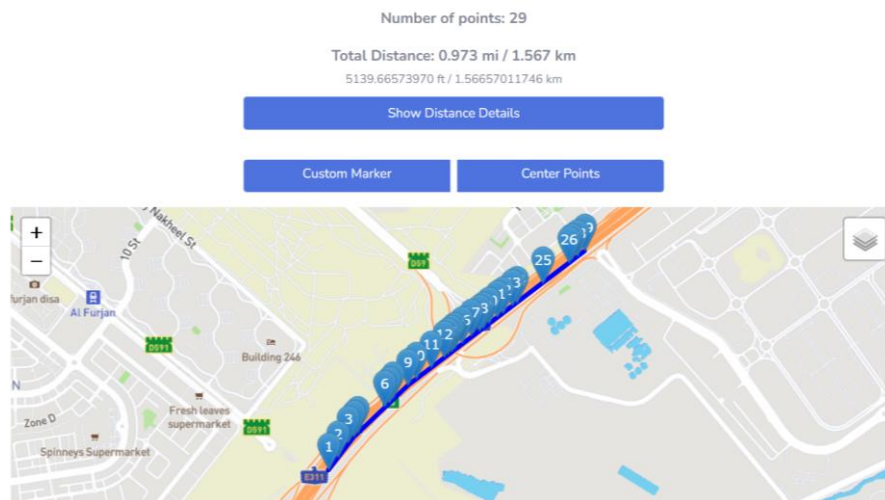


Fig. 64: Coordinates output on map

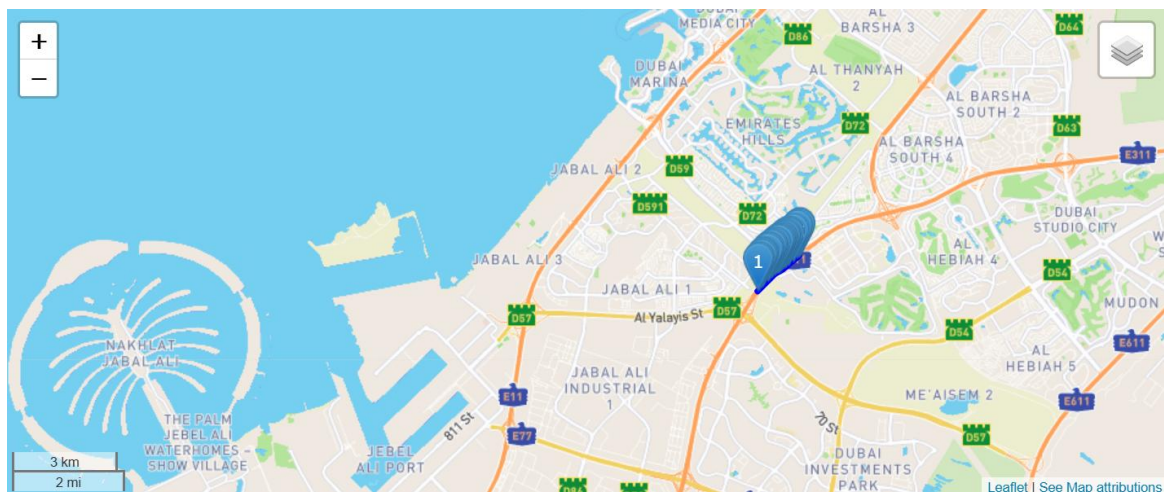


Fig. 65: Coordinates marked on part of Sheikh Mohammed Road, Dubai

Data analysis

Data was collected for 1-2 weeks, the aim being to cover atleast one round of all week days during the set windows – 5am to 9am in the morning, and 4pm to 8pm in the evening, capturing peak hours & around it. The data we had included speeds & timestamp. Feature engineering was done to extract more useful/indicative features from these:

- Time of day (as hour & minute)
- Day of week
- Weekday or weekend

Data Cleaning: Checked for missing or anomalous values due to API limits or outages. Multiple keys were given to the automated model to check for incase of an unsuccessful request. Still, limits were hit a number of times.

Plot speeds vs. time for each segment to identify:

1. Clear daily patterns
2. Differences between weekdays/weekends
3. Outliers or sensor errors

Practical Example Workflow

- Prepare your dataset with columns: [minute, day, speed, speed_t-1, speed_t-2, ...].
- Split data: For training, testing & validation
- Fit a few different models.
- Compare predictions to actual speeds using MAE/RMSE.

```
from matplotlib.dates import DateFormatter

df = pd.read_excel('final_vsl_data.xlsx')[1:]
df = df[['Timestamp', 'Mcity_CurrentSpeed', 'Mohd_CurrentSpeed', 'Deira_CurrentSpeed']]
df['Mcity_S1'] = df['Mcity_CurrentSpeed'].str.split(',').str[0]
df['Mcity_S2'] = df['Mcity_CurrentSpeed'].str.split(',').str[1]
df['Mcity_S3'] = df['Mcity_CurrentSpeed'].str.split(',').str[2]
df = df[df['Mcity_S1'] != 'Error']
df = df.drop('Mcity_CurrentSpeed', axis=1)

df = df[df['Deira_CurrentSpeed'].apply(lambda x: isinstance(x, str))]
df['Deira_S1'] = df['Deira_CurrentSpeed'].str.split(',').str[0]
df['Deira_S2'] = df['Deira_CurrentSpeed'].str.split(',').str[1]
df = df.drop('Deira_CurrentSpeed', axis=1)

morning_window = ('05:00', '09:00') # 5-9 AM
evening_window = ('16:00', '20:00') # 4-8 PM
time_condition = (
    df['Timestamp'].dt.time.between(pd.to_datetime(morning_window[0]).time(),
                                    pd.to_datetime(morning_window[1]).time()) |
    df['Timestamp'].dt.time.between(pd.to_datetime(evening_window[0]).time(),
                                    pd.to_datetime(evening_window[1]).time())
)
df = df[time_condition].copy()

df['hour'] = df['Timestamp'].dt.hour
df['minute'] = df['Timestamp'].dt.minute
df['day_of_week'] = df['Timestamp'].dt.dayofweek # Monday=0
df['is_weekend'] = df['day_of_week'].isin([5,6]).astype(int)
df['date'] = df['Timestamp'].dt.date

df.head()
```

Fig. 66: Post-processing collected data Part 1

```
def process_speed_string(cell_value):
    if isinstance(cell_value, str):
        if '-' in cell_value:
            return [cell_value[5:7], cell_value[8:10], cell_value[12:14]]
        parts = [p.strip() for p in cell_value.split(',') if p.strip()]
        if len(parts) == 1:
            return ['Error', parts[0], 'Error']
        elif len(parts) == 0:
            return ['Error', 'Error', 'Error']
        else:
            while len(parts) < 3:
                parts.append('Error')
            return parts[:3] # Ensure we only return 3 elements
    else:
        print(f"Unexpected type: {type(cell_value)}, data: {cell_value}")
        return [np.nan, np.nan, np.nan]

df['Mohd_CurrentSpeed'] = df['Mohd_CurrentSpeed'].astype(str)
split_speeds = df['Mohd_CurrentSpeed'].apply(process_speed_string)
df[['Mohd_S1', 'Mohd_S2', 'Mohd_S3']] = pd.DataFrame(split_speeds.tolist(), index=df.index)
df = df[df['Mohd_S1'] != 'Error']
df = df[df['Mohd_S2'] != 'Error']
df = df[df['Mohd_S3'] != 'Error']
df = df.drop('Mohd_CurrentSpeed', axis=1)

# Convert all speed columns in one go
speed_cols = ['Mcity_S1', 'Mcity_S2', 'Mcity_S3', 'Deira_S1', 'Deira_S2', 'Mohd_S1', 'Mohd_S2', 'Mohd_S3']
df[speed_cols] = df[speed_cols].apply(pd.to_numeric, errors='coerce')
df = df.dropna()
```

Some values with comma-separated speed values got interpreted as dates, required extra processing:

```
def process_speed_string(cell_value):
    if isinstance(cell_value, str):
        # 1. Handle date strings
        # Keep consistent with prior logic for identifying date strings
        if '-' in cell_value:
            # print(f"Date value: {cell_value}")
            return [cell_value[5:7], cell_value[8:10], cell_value[2:4]] # Or ['Date', 'Date', 'Date'] if you prefer strings

        # 2. Process comma-separated values
        # Split and strip whitespace, filtering out any empty strings from the split
        # (e.g., splitting "109," or ",109" won't produce empty parts)
        parts = [p.strip() for p in cell_value.split(',') if p.strip()]

        # 3. Apply specific padding logic based on the number of parts
        if len(parts) == 1:
            # User wants: Error, num, Error
            return ['Error', parts[0], 'Error']
        elif len(parts) == 0:
            # Empty string or string with only commas/whitespace
            return ['Error', 'Error', 'Error']
        else:
            # For 2 or more parts (e.g., '111, 109', '111, 109, 83', 'Error, 109, Error')
            # Pad with 'Error' if less than 3 parts, then ensure only 3 are returned
            while len(parts) < 3:
                parts.append('Error')
            return parts[:3] # Ensure we only return 3 elements

    else:
        # Handle non-string types (e.g., actual numbers, None, np.nan)
        print(f"Unexpected type: {type(cell_value)}, data: {cell_value}")
        return [np.nan, np.nan, np.nan]
```

Fig. 67: Post-processing collected data Part 2

Prediction Model

The intention was to build a basic prediction model that outputs expected speed on let's say 5pm on a Monday. Many sophisticated models take in much more data like weather, incidents, etc., but speed prediction is not a focus of this project, so only a basic version was implemented.

Model Selection: As mentioned in the literature review, **k-nearest neighbours** was considered most suitable due to interpretable results with a small dataset.

Considerations while implementing KNN:

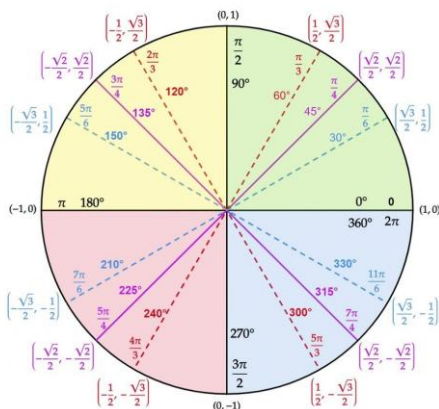
- **Feature Scaling:** KNN uses distance metrics, so features at different scales (e.g., hour_angle vs. minute) distort neighbors.
- **Hyperparameters:** Kept as 5, not too susceptible to noise, nor over smoothening & missing local patterns. Weighted by distance, standard Euclidean distance metric.

Other Considerations:

- **Data augmentation:** Add slight noise (± 1 -2 mph) to create synthetic samples.
- **Model Simplicity:** With limited data, simpler models (linear regression, AR models) often outperform complex ones due to lower risk of overfitting.

We used sine and cosine transformations for the hour feature to properly represent cyclical time patterns in the traffic speed prediction model. The Problem with Raw Hours (0-23): 23:59 and 00:01 would appear as “far apart” (distance of 23.97), but they’re actually adjacent times with similar traffic patterns. This discontinuity harms model performance. By transforming hours into these two features, we create a smooth circular representation where values like 23:59 and 00:01 have nearly identical coordinates – all hours are equally spaced around a unit circle [69].

The Unit Circle Chart



```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import TimeSeriesSplit

df['hour_sin'] = np.sin(2 * np.pi * df['Timestamp'].dt.hour/24)
df['hour_cos'] = np.cos(2 * np.pi * df['Timestamp'].dt.hour/24)

target_col = 'Mcity_s1'

y = df[target_col]
X = df[['hour_sin', 'hour_cos', 'minute', 'day_of_week', 'is_weekend']]
```

Fig. 68 & 69: Conversion of hour to sin/cos

Evaluated Models:

- Time-Average (Baseline)
- Last-Value (Persistence)
- Random Forest
- KNN (Primary Model)

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler

models = {}

# 1. Time-based average (simple baseline)
class TimeAverageModel:
    def fit(self, X, y):
        self.avg_speeds = y.groupby([X['hour_sin'], X['hour_cos']]).median()

    def predict(self, X):
        return X.apply(lambda row: self.avg_speeds.get((row['hour_sin'], row['hour_cos']), y.median()), axis=1)

models['time_avg'] = TimeAverageModel()

# 2. Last Value (persistence model)
class LastValueModel:
    def fit(self, X, y):
        self.last_speed = y.iloc[-1]

    def predict(self, X):
        return np.full(len(X), self.last_speed)

models['last_value'] = LastValueModel()

# 3. Random Forest
models['random_forest'] = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
```

Fig. 70: Trying different prediction models

```

# 4. K - Nearest Neighbours
class KNNTIMEModel:
    def __init__(self):
        self.model = KNeighborsRegressor(
            n_neighbors=5,
            weights='distance',
            metric='euclidean'
        )
        self.scaler = StandardScaler()
        self.knn_features = ['hour_angle', 'minute', 'day_of_week']
        self.fallback = None

    def fit(self, X, y):
        # Store fallback value (median)
        self.fallback = y.median()

        # Create hour_angle if not present
        Xx = X.copy()
        if 'hour_angle' not in Xx.columns:
            Xx['hour_angle'] = np.arctan2(Xx['hour_sin'], Xx['hour_cos'])

        # Scale features
        X_knn = Xx[self.knn_features]
        self.scaler.fit(X_knn)
        X_scaled = self.scaler.transform(X_knn)

        # Train model
        self.model.fit(X_scaled, y)
        return self

    def predict(self, X):

```

```

        def predict(self, X):
            # Create hour_angle if needed
            Xx = X.copy()
            if 'hour_angle' not in Xx.columns:
                Xx['hour_angle'] = np.arctan2(Xx['hour_sin'], Xx['hour_cos'])

            try:
                X_scaled = self.scaler.transform(Xx[self.knn_features])
                return self.model.predict(X_scaled)
            except Exception as e:
                print(f"KNN prediction failed, using fallback: {e}")
                return np.full(len(X), self.fallback)

models['knn'] = KNNTIMEModel()

```

Fig. 70 & 71: KNN class

```

models['knn'] = KNNTIMEModel()

# Time-series cross-validation
tscv = TimeSeriesSplit(n_splits=5)
metrics = {}

for name, model in models.items():
    mae_scores = []
    for train_idx, test_idx in tscv.split(X):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        mae_scores.append(mean_absolute_error(y_test, preds))

    metrics[name] = np.mean(mae_scores)
    print(f"{name:15} MAE: {metrics[name]:.2f} km/h")

```

Fig. 72: Running all models & comparing

A hybrid model was also tried, combining knn & time-series average, and while that better than avg. output, knn still outperformed it. Here are the results of the training:

time_avg	MAE: 5.78 km/h
last_value	MAE: 14.47 km/h
random_forest	MAE: 10.13 km/h
knn	MAE: 7.88 km/h

```

# Prediction function for new timestamps
def predict_speed(timestamp):
    """Timestamp format: '2025-06-30 09:03:22'"""
    ts = pd.to_datetime(timestamp)
    features = pd.DataFrame({
        'hour_sin': [np.sin(2 * np.pi * ts.hour/24)],
        'hour_cos': [np.cos(2 * np.pi * ts.hour/24)],
        'minute': [ts.minute],
        'day_of_week': [ts.dayofweek],
        'is_weekend': [int(ts.dayofweek in [5,6])]
    })
    return final_model.predict(features)[0]

# Train final model on all data
best_model_name = min(metrics, key=metrics.get)
final_model = models[best_model_name]
final_model.fit(X, y)

# Example usage
time_now = '2025-07-14 18:03:22'
print(predict_speed(time_now))

```

82.0

Fig. 73 & 74: Result of comparison

Putting the results together:

random_forest	running	time_avg	running
random_forest Analysis:		time_avg Analysis:	
- Historical points: 20		- Historical points: 20	
- Historical median: 62 km/h		- Historical median: 62 km/h	
- Model prediction: 63 km/h		- Model prediction: 82 km/h	
- Difference: +1 km/h		- Difference: +20 km/h	
- Model MAE: 10.13 km/h		- Model MAE: 5.78 km/h	
knn	running	last_value	running
knn Analysis:		last_value Analysis:	
- Historical points: 20		- Historical points: 20	
- Historical median: 62 km/h		- Historical median: 62 km/h	
- Model prediction: 64 km/h		- Model prediction: 110 km/h	
- Difference: +2 km/h		- Difference: +48 km/h	
- Model MAE: 7.88 km/h		- Model MAE: 14.47 km/h	

Fig. 75: Visualising model performance, compared to ideal value

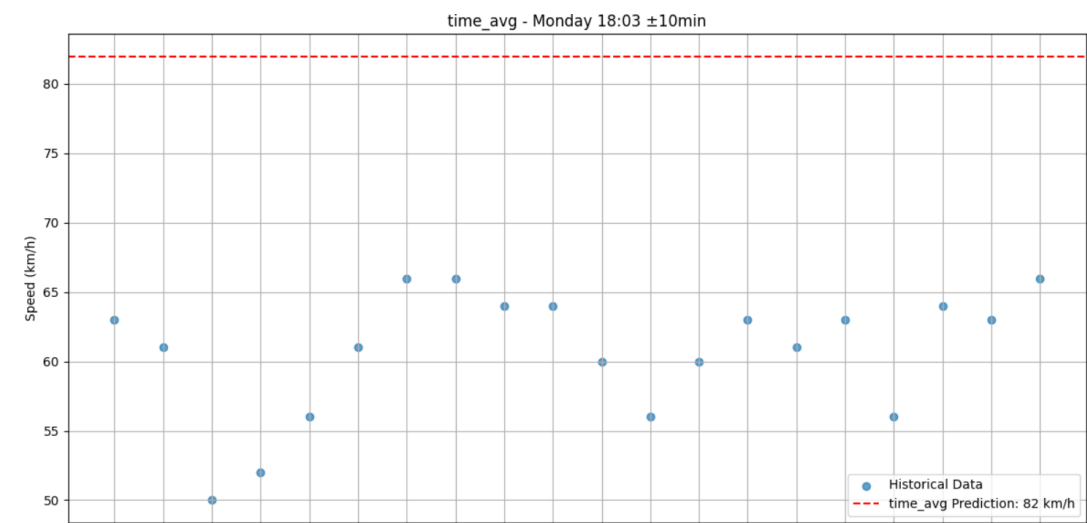


Fig. 76: time_avg performing moderately

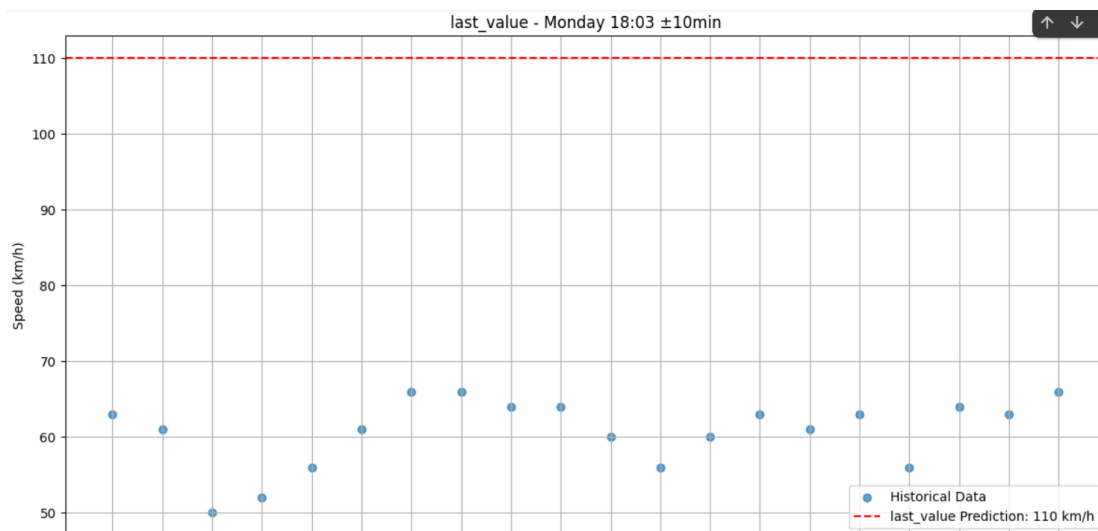


Fig. 77: last_value performing badly

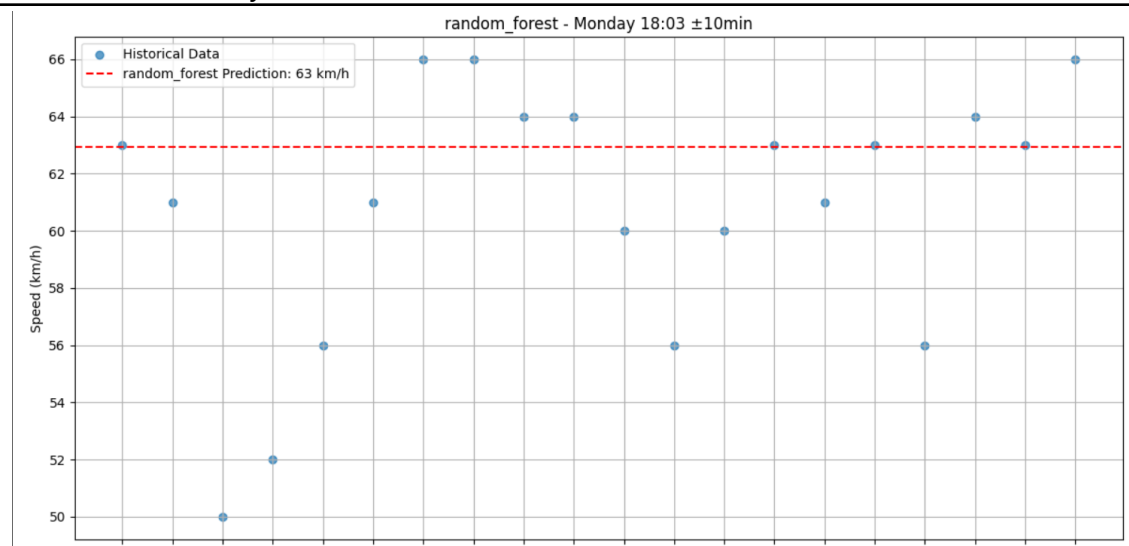


Fig. 78: random_forest performing moderately

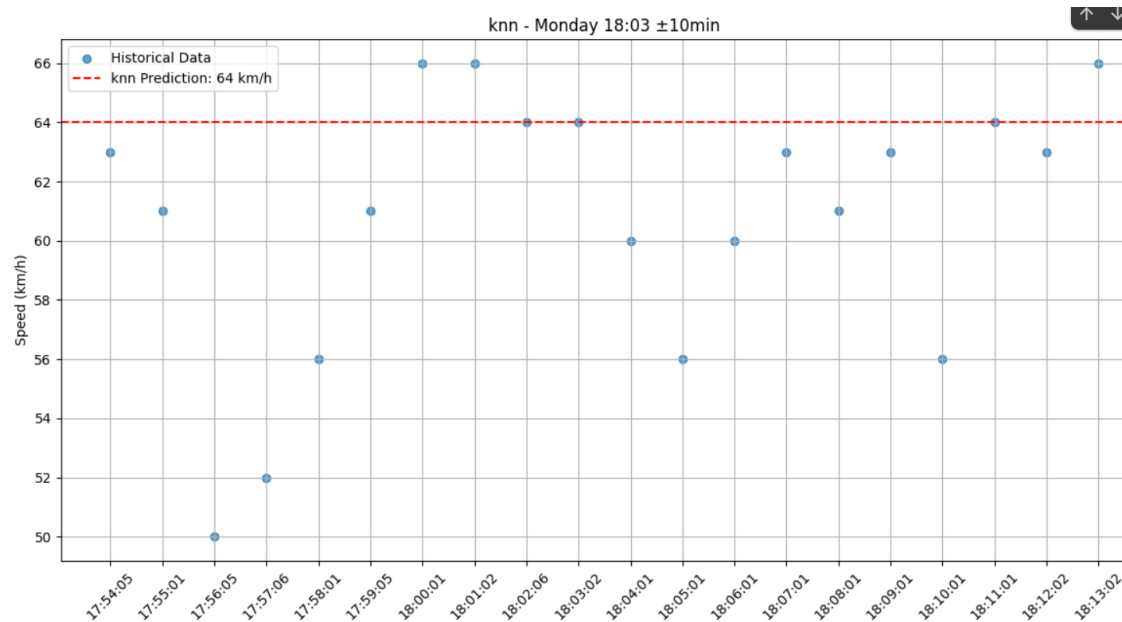


Fig. 79: knn performing well

Dashboard

The dashboard was designed with HTML, CSS & JavaScript. It embeds the Google Maps API, and lists our 3 locations on the side panel. When one is clicked, it zooms into that spot, highlighting the part of the highway in blue, and shows its recommended speed limit, through HTTP requests to the server which is also hosted by the GCP. The map allows the user to zoom & move around, and the roads are color-coded green, red or yellow to indicate traffic level.

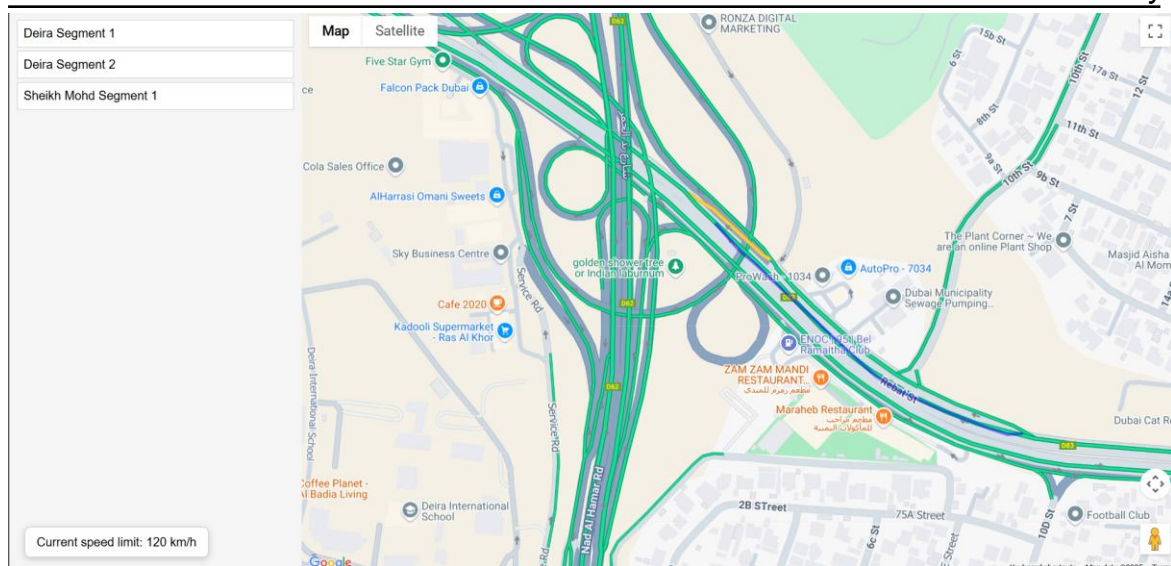


Fig. 80: Dashboard

7 TESTING & RESULTS

7.8 Types of Testing

- Unit testing: As mentioned in the software development plan, since the beginning, this project was approached with modularity in mind, making the pieces work & then putting them together.
- Functional testing: Verifies functionality of the system. Working simulation was combined with implemented SAC model for training.
- Integration testing: Model running & showing results on dashboard.

Simulation

The model could not be tested without being plugged into an environment, but the environment was run with fixed limits and a variety of them to observe behaviour, whether VSL was being applied properly & whether it was reacting to it. This is how the speeds looked like raw, with the standard 120 limit:

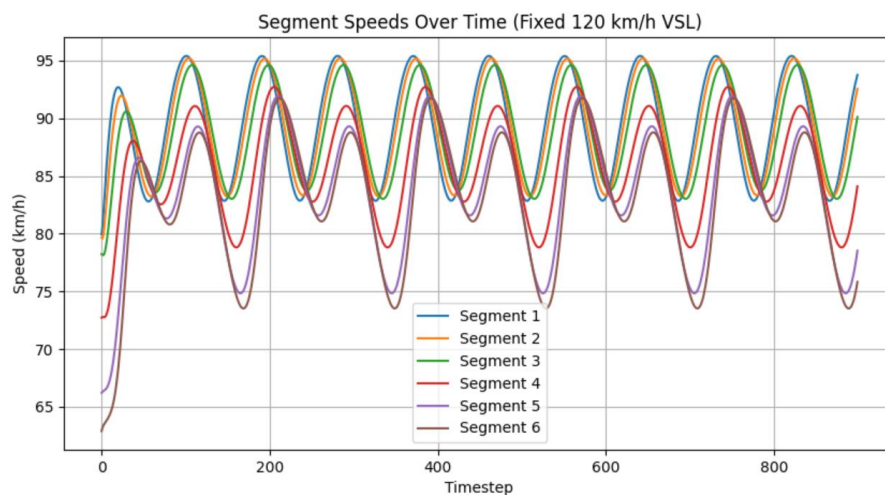


Fig. 81: Speed patterns for fixed 120 km/h

Next, different speeds were tried, one fixed per episode. This confirmed correct simulation with VSL.

```

# Try different constant speed limits
test_speeds = [120, 100, 80, 60]
results = {}

for speed_limit in test_speeds:
    env = MetaNetEnv()
    state = env.reset()
    rewards = []
    tts_over_time = []

    done = False
    while not done:
        action = np.array([speed_limit] * env.vsl_count)
        state, reward, done, _ = env.step(action)
        rewards.append(reward)
        rho = np.sum(np.array(env.rho).flatten())
        tts_over_time.append(rho)

    total_reward = sum(rewards)
    tts = sum(tts_over_time) * env.T
    results[speed_limit] = {'reward': total_reward, 'tts': tts}

    print(f"Speed: {speed_limit}, Total Reward: {total_reward:.2f}, TTS: {tts:.2f}")

plt.figure()

```

Fig. 82: Script to test VSL impact

As shown in the below figure, TTS is high when limiting speed to just 60, and reduces with higher limits, and stabilises up to the standard highest limit. Negative reward (penalty) is proportional to TTS.

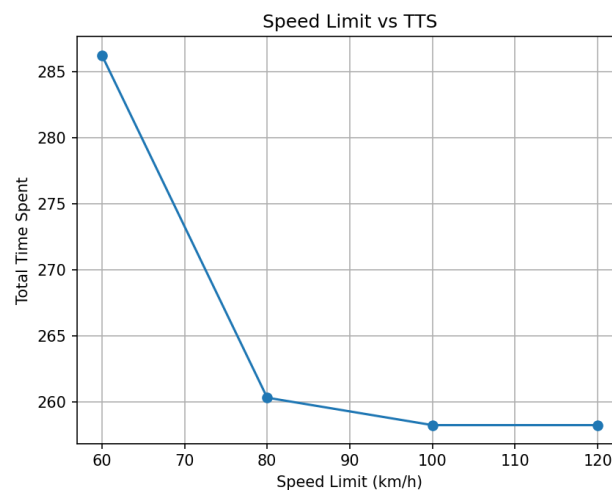


Fig. 83: Shows VSL impact

When run with the model, referring to the reward function, the initial raw value was oscillating around 500. This was too large for the model to react to, so it was initially rescaled with the tanh function. But the result was a standard value of -0.01 in most steps, so it was suspected that tanh was suppressing variation in larger values. Clipping to boundaries throughout all scripts resulted in reward outputs around 0 to -1. Below result is when reward = $-\text{np.tanh}(\text{total_hours} * \text{self.reward_scale})$. Key aspect is tanh function applied.

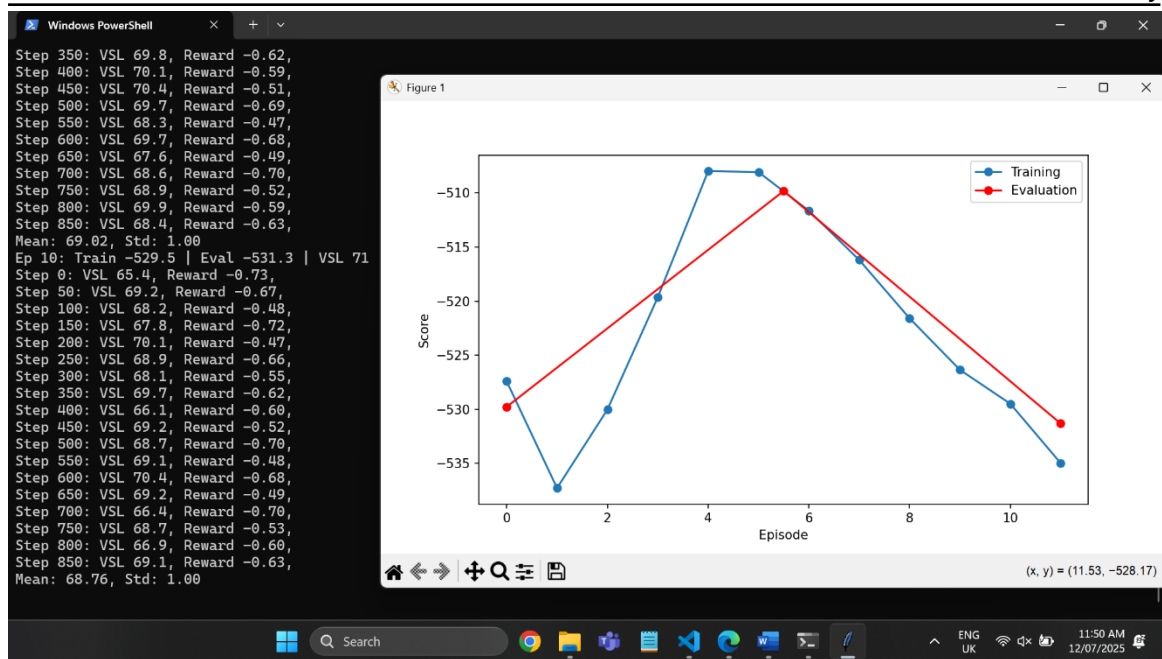


Fig. 84: Poorly-performing model

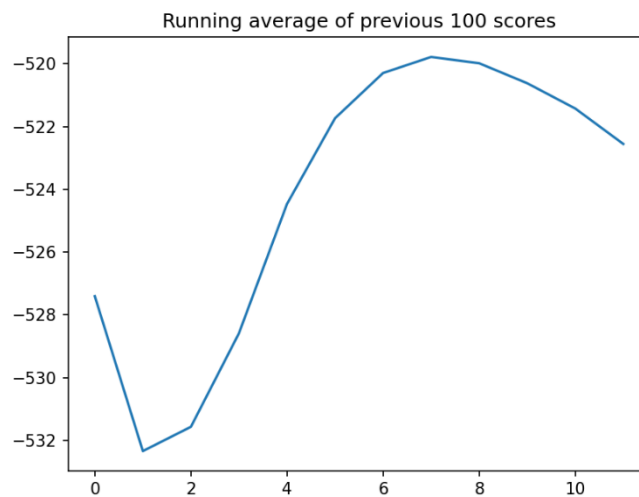


Fig. 85: Highly negative scores

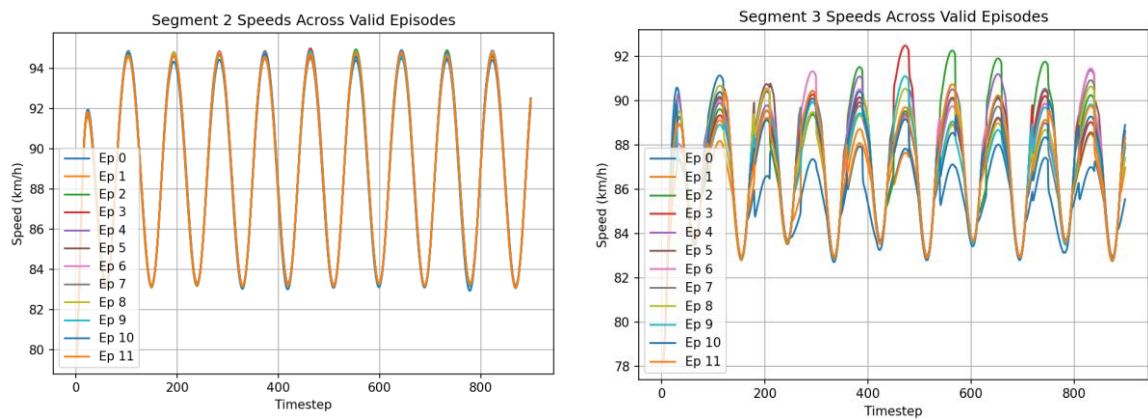


Fig. 86 & 87: Speed patterns of different segments Part 1

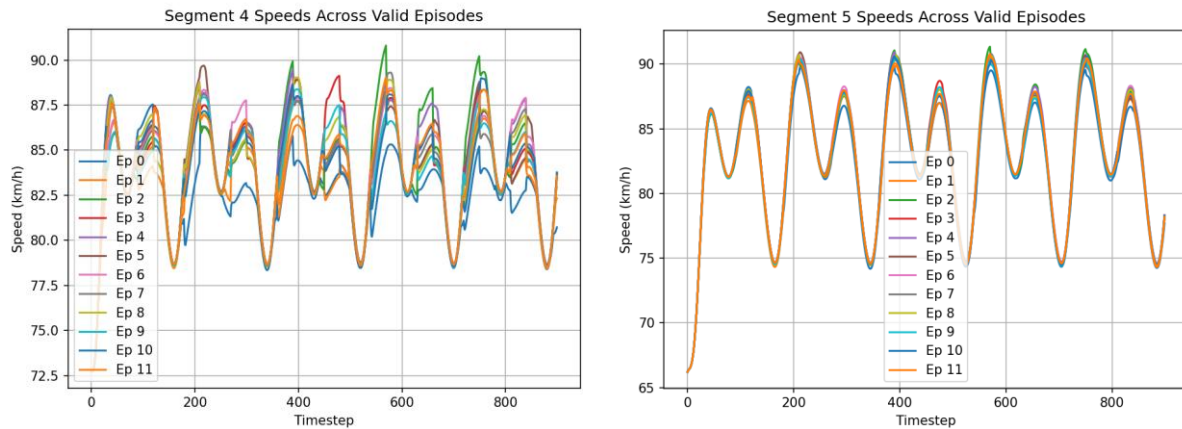


Fig. 88 & 89: Speed patterns of different segments Part 2

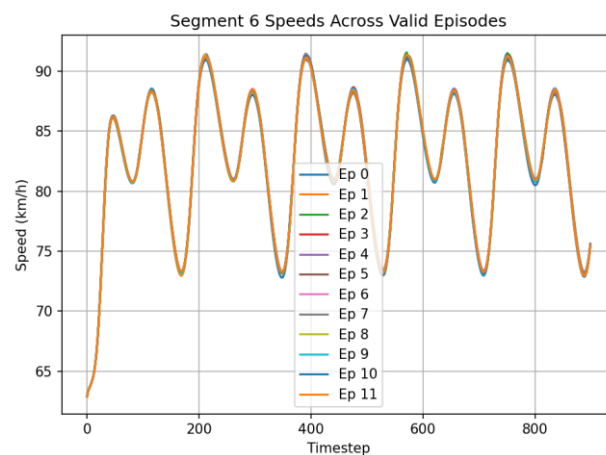


Fig. 90: Speed patterns of different segments Part 3

```

Mean: 66.47, Std: 1.00
Ep 10: Train -5.6 | Eval -5.5 | VSL 62
Step 0: VSL 64.3, Reward -0.01,
Step 50: VSL 65.9, Reward -0.01,
Step 100: VSL 65.8, Reward -0.01,
Step 150: VSL 66.8, Reward -0.01,
Step 200: VSL 61.9, Reward -0.00,
Step 250: VSL 60.7, Reward -0.01,
Step 300: VSL 62.9, Reward -0.01,
Step 350: VSL 65.8, Reward -0.01,
Step 400: VSL 64.4, Reward -0.01,
Step 450: VSL 67.2, Reward -0.01,
Step 500: VSL 62.2, Reward -0.01,
Step 550: VSL 62.4, Reward -0.01,
Step 600: VSL 61.6, Reward -0.01,
Step 650: VSL 65.6, Reward -0.01,
Step 700: VSL 66.0, Reward -0.01,
Step 750: VSL 66.4, Reward -0.01,
Step 800: VSL 62.9, Reward -0.01,
Step 850: VSL 62.9, Reward -0.01,
Mean: 62.13, Std: 1.00

```

Fig. 91: Shows low variance in rewards

Reward is not penalizing unnecessarily low speeds enough. Sometimes if initial value isn't affecting speeds, it stays there instead of maximizing the value. We do not want to restrict the limit without reason, so this term was added to reward later on.

Similar when $\text{reward} = -\text{total_hours} * \text{self.reward_scale}$. Rewards are either -0.00 or -0.01, hence, reward scale of 0.01 isn't a good choice. But model is increasing TTS instead of decreasing.

With regards to the inconsistent results of each reward change, it was suspected that the slope occurs when the random initialisation of the model weights output a value outside of the 75-100 range of vehicle speeds based on demands, i.e. when the TTS is likely to change more prominently, like applying 60, with is the minimum & most speeds are above it. But when current speed is already in the range (e.g. 80 or 90), the VSL doesn't seem to effect the simulation, and therefore the rewards don't reflect it & we get a flat plot. But ideally, the limit should be higher like 120 only, unless there is a reason to limit it, like if there is no/less congestion up ahead. If 90 & 100 give the same result, the limit should be 100, not 90. We don't want to unnecessarily restrict the flow. Term was added to result.

Steps followed to fix model:

- Rewards were transformed to better indicate good vs. bad actions
- VSL reactions will not show in one step, so rewarding it at every step didn't make sense. Accumulating rewards over x steps & adding to model memory resulted in an expected trajectory.
- Functions were flipped to observe model behaviour.

Model learnt to minimise negative reward:

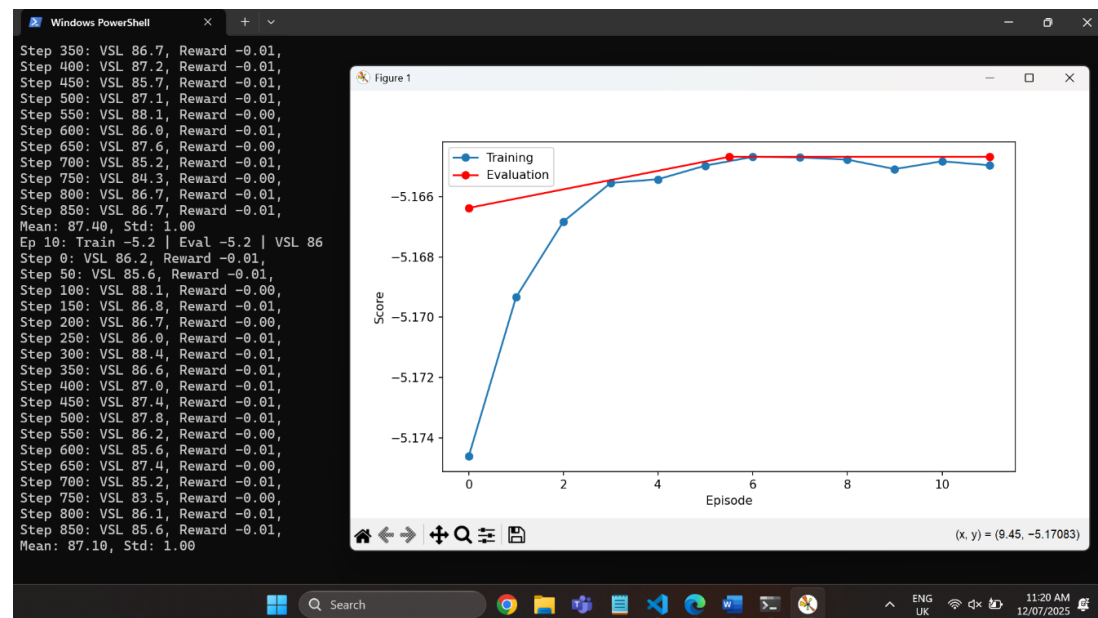


Fig. 92: Model moving positively

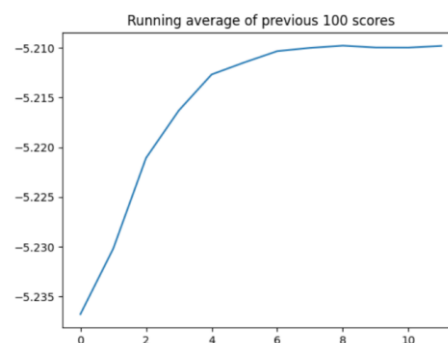


Fig. 93: Positive score build

Basic behaviour in a non-congested scenario would be to maximise speed. Below outputs show rewards encouraging higher speeds:

```
Step 250: VSL 98.1, Reward -0.78,
Step 300: VSL 101.0, Reward -0.64,
Step 350: VSL 99.7, Reward -0.81,
Step 400: VSL 103.1, Reward -0.67,
Step 450: VSL 103.0, Reward -0.67,
```

Fig. 94: Rewards behaving properly Part 1

```
Step 650: VSL 72.6, Reward -0.48,
Updating memory - avg speed: 72.5606689453125   Reward for 30 steps: -14.559753886931755
Updating memory - avg speed: 70.12633514404297   Reward for 30 steps: -19.47873314014687
```

Fig. 95: Rewards behaving properly Part 2

Playing around with the entropy parameter showed high exploration:

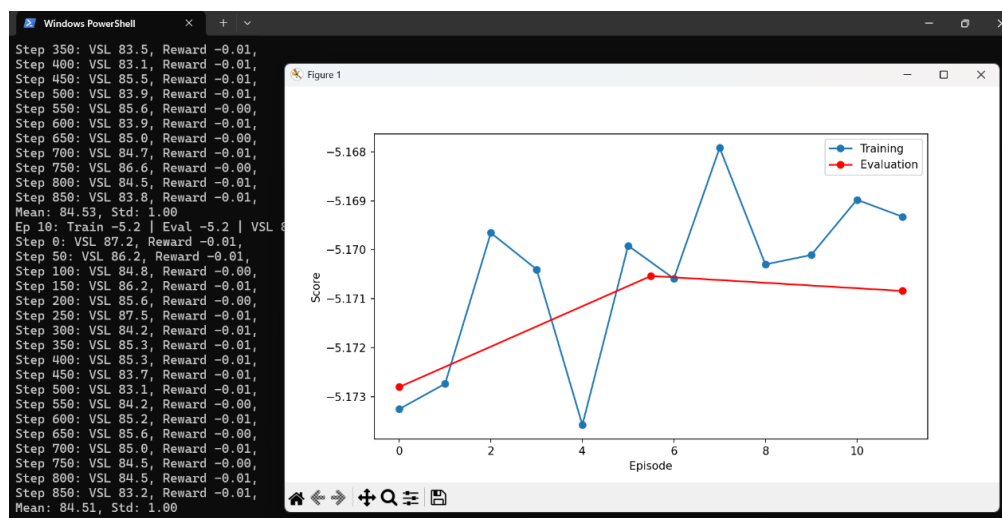


Fig. 96: Entropy leading to exploration

Hyperparameters & other changes: For activation functions for the inner layers, the most common option ReLu was replaced with LeakyReLu, since there were negative values flowing through the network, and they shouldn't be processed to 0. Adam & SGD were tested for optimisers, both performed similarly. Most values were moved around to observe effects - entropy parameter, driver compliance parameter, etc. More explained below.

Data analysis results

Timestamp	Mohd_CurrentSpeed	Mcitv_S1	Mcitv_S2	Mcitv_S3	Deira_S1	Deira_S2	hour	minute	day_of_week	is_weekend	date
2025-06-22 16:43:02	111, 109, 83	110	105	106	96	77	16	43	6	1	2025-06-22
2025-06-22 16:44:02	111, 109, 83	110	105	106	96	74	16	44	6	1	2025-06-22
2025-06-22 16:45:02	111, 109, 83	110	105	106	96	72	16	45	6	1	2025-06-22
2025-06-22 16:46:01	111, 109, 83	110	105	106	96	70	16	46	6	1	2025-06-22
2025-06-22 16:47:02	111, 109, 83	110	105	106	96	71	16	47	6	1	2025-06-22

Fig. 97: Data collected Part 1

(t)	Timestamp	Mcitty_S1	Mcitty_S2	Mcitty_S3	Deira_S1	Deira_S2	hour	minute	day_of_week	is_weekend	date	Mohd_S1	Mohd_S2	Mohd_S3
45	2025-06-22 16:43:02	110.0	105.0	106.0	96	77	16	43	6	1	2025-06-22	111	109	83
46	2025-06-22 16:44:02	110.0	105.0	106.0	96	74	16	44	6	1	2025-06-22	111	109	83
47	2025-06-22 16:45:02	110.0	105.0	106.0	96	72	16	45	6	1	2025-06-22	111	109	83
48	2025-06-22 16:46:01	110.0	105.0	106.0	96	70	16	46	6	1	2025-06-22	111	109	83
49	2025-06-22 16:47:02	110.0	105.0	106.0	96	71	16	47	6	1	2025-06-22	111	109	83

Fig.98: Data collected Part 2

Key features for plotting:

1. **One plot per day** - Each figure shows a full day's data
2. **All speed columns** - Each location/sensor gets its own line
3. **Clean time formatting** - X-axis shows only hours:minutes
4. **Automatic handling** - Works for any number of days in your data

Below was the first check after one night of setting up the automation, in one segment from MCity. Each value was visualized to validate API data. The speed, incident & TTS patterns match.

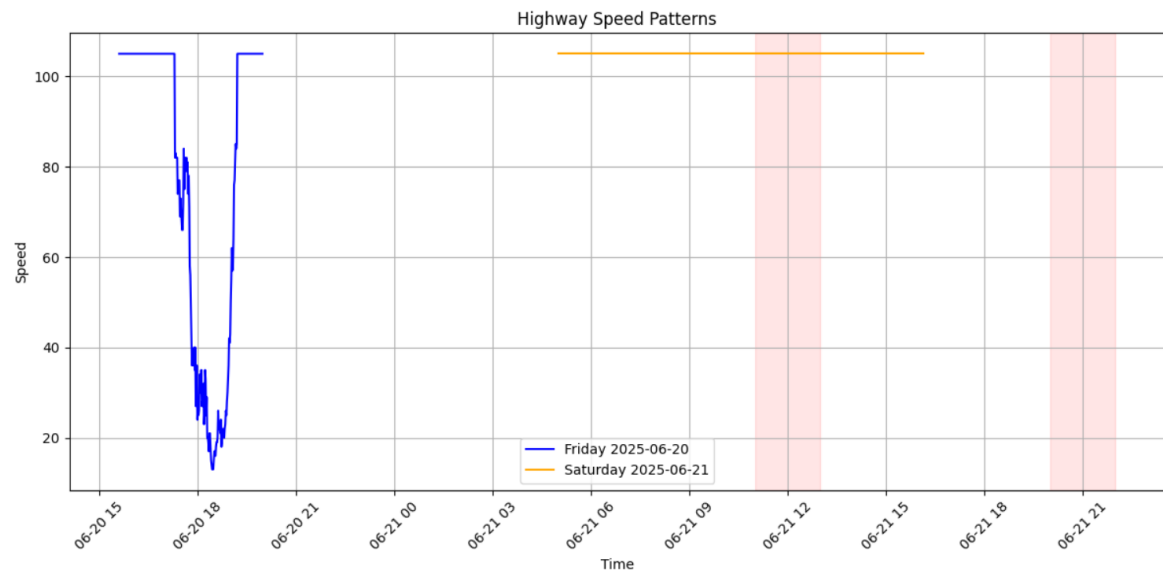


Fig. 99: Single segment speed pattern

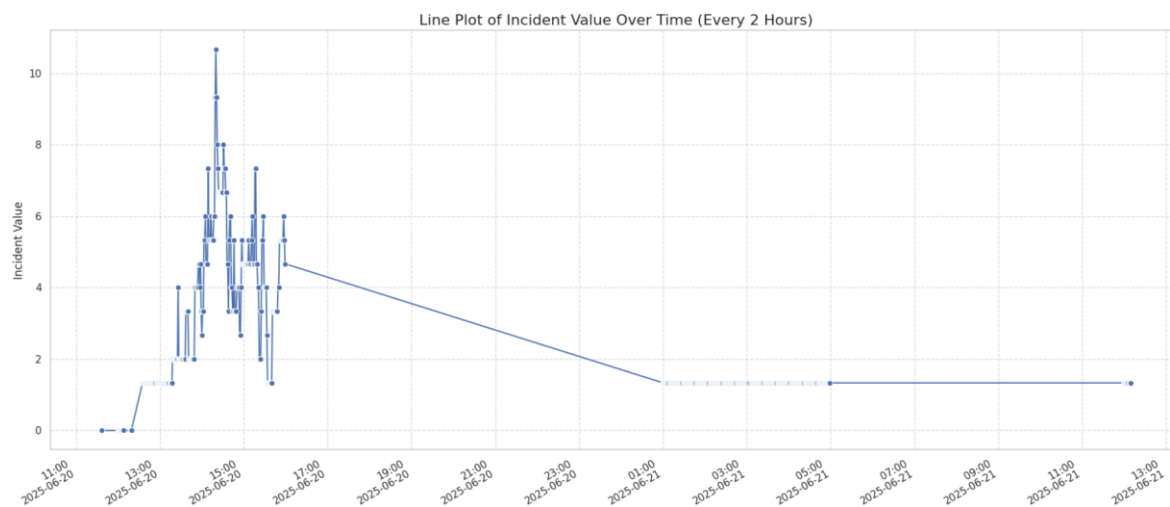


Fig.100: Incident patterns match that of speeds

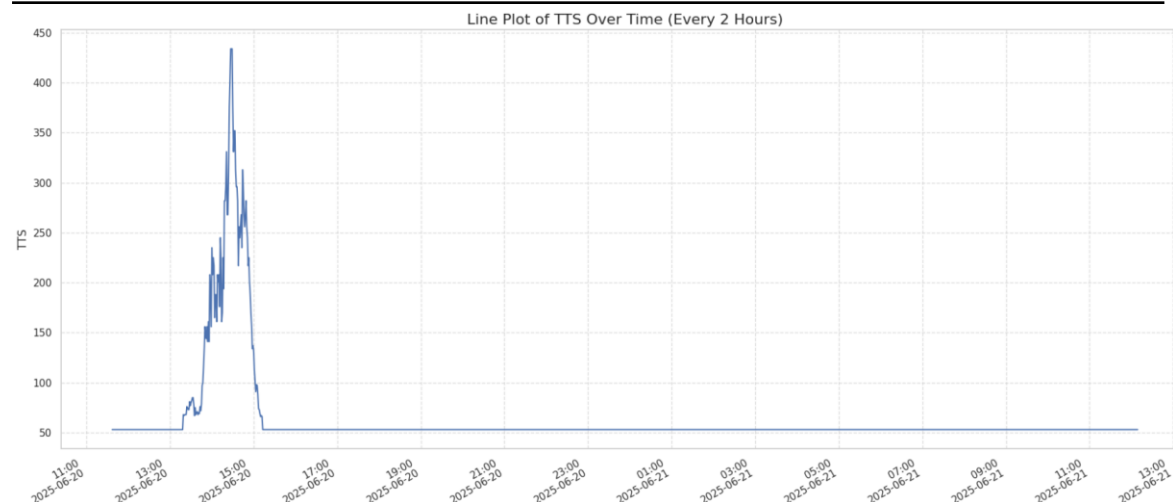


Fig.101: TTS patterns match that of speeds

Below plots show speed patterns of segments of MCity, across different days. The directions of the segments go from south to north – from the areas where companies tend to be & where rent is expensive, to the areas with cheaper rent. So as expected, the traffic patterns show the main peak hour on weekdays being evening times when people go back home from their workplaces.

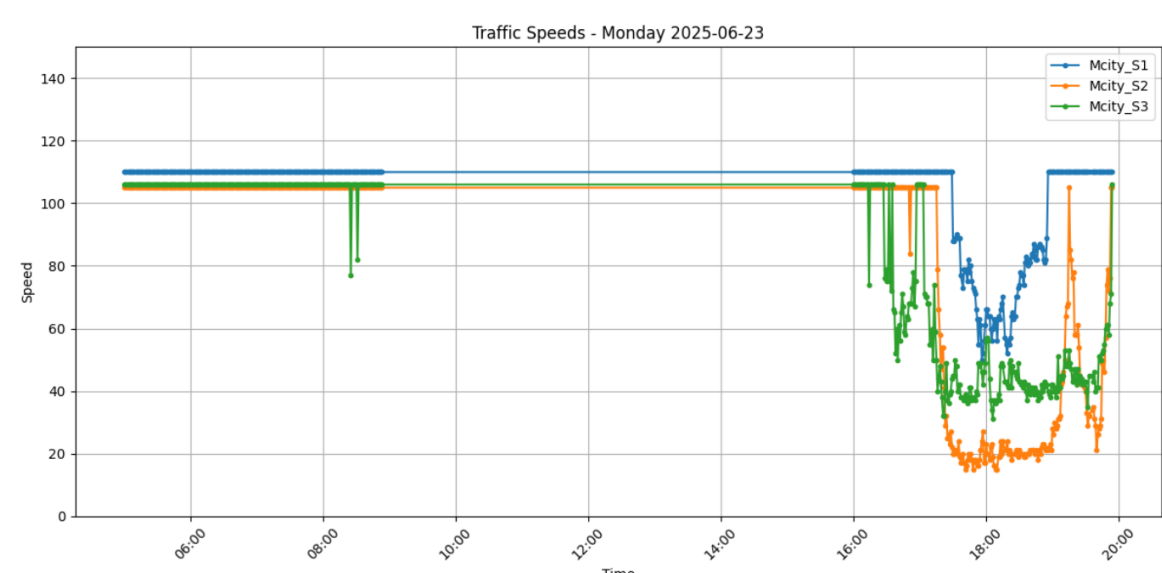


Fig. 102: Speed patterns of Mcity segments on Monday

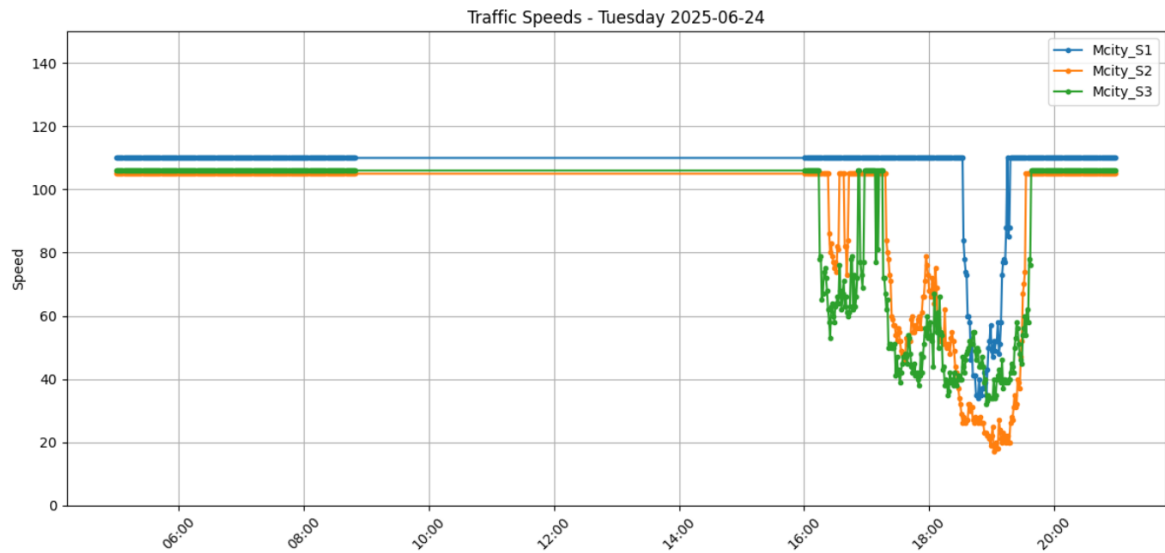


Fig. 103: Speed patterns of Mcity segments on Tuesday

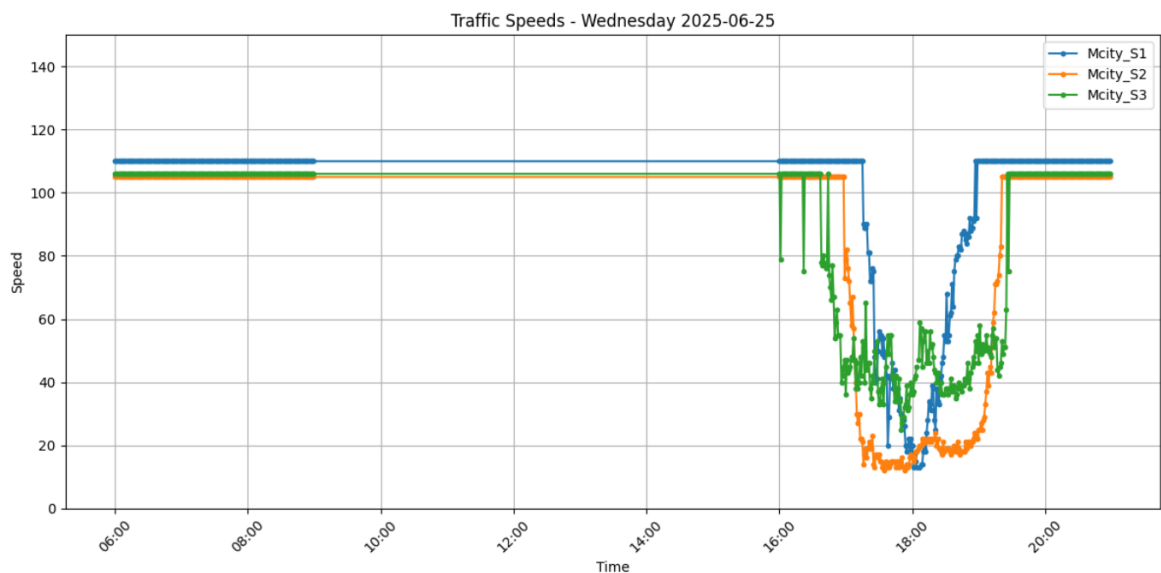


Fig. 104: Speed patterns of Mcity segments on Wednesday

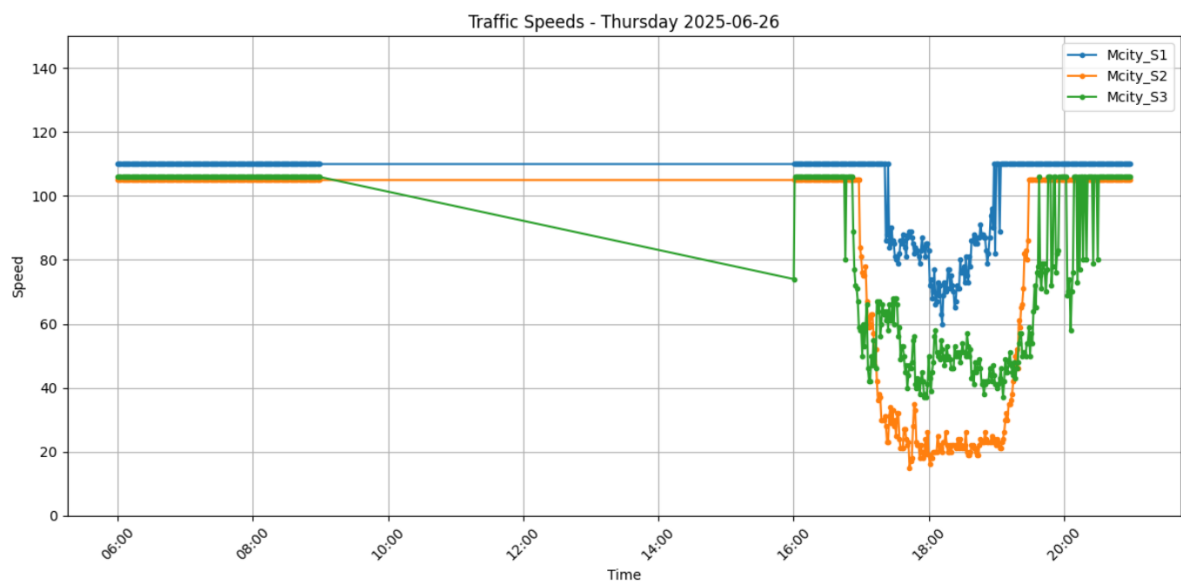


Fig. 104: Speed patterns of Mcity segments on Thursday

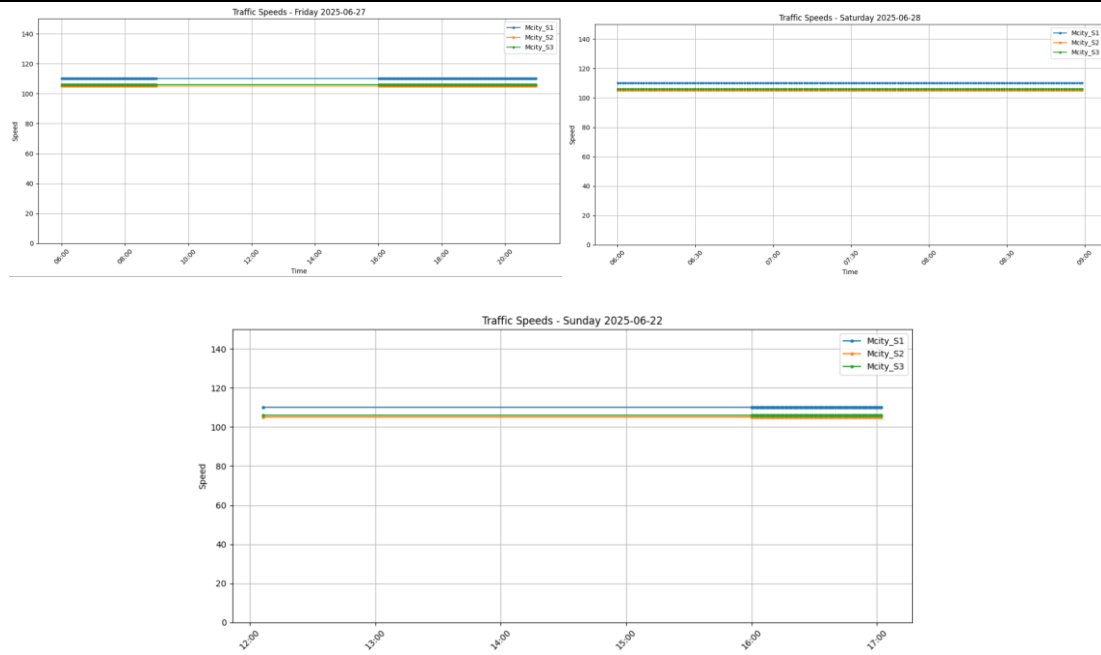


Fig. 105-107: Speed patterns of Mcity segments Fri-Sun

Deira: Same patterns – Peaks on evening of Monday to Thursday, fairly normal Friday to Sunday.

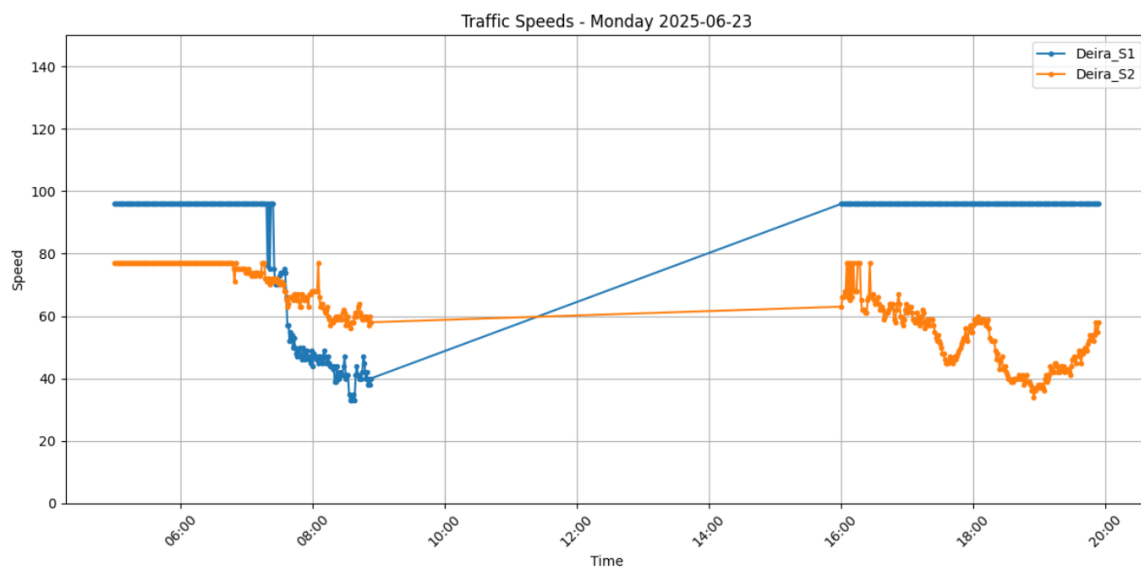
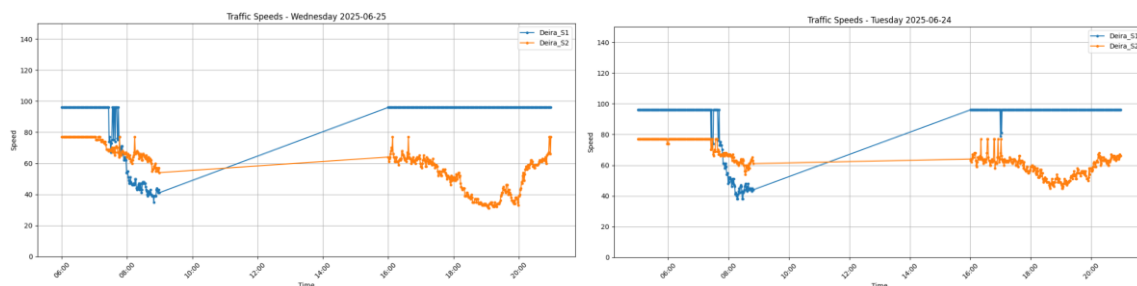


Fig. 108: Speed patterns of Deira segments on Monday



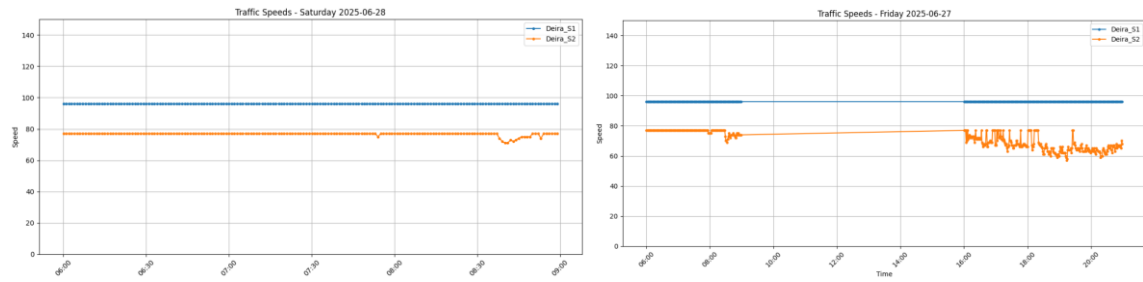
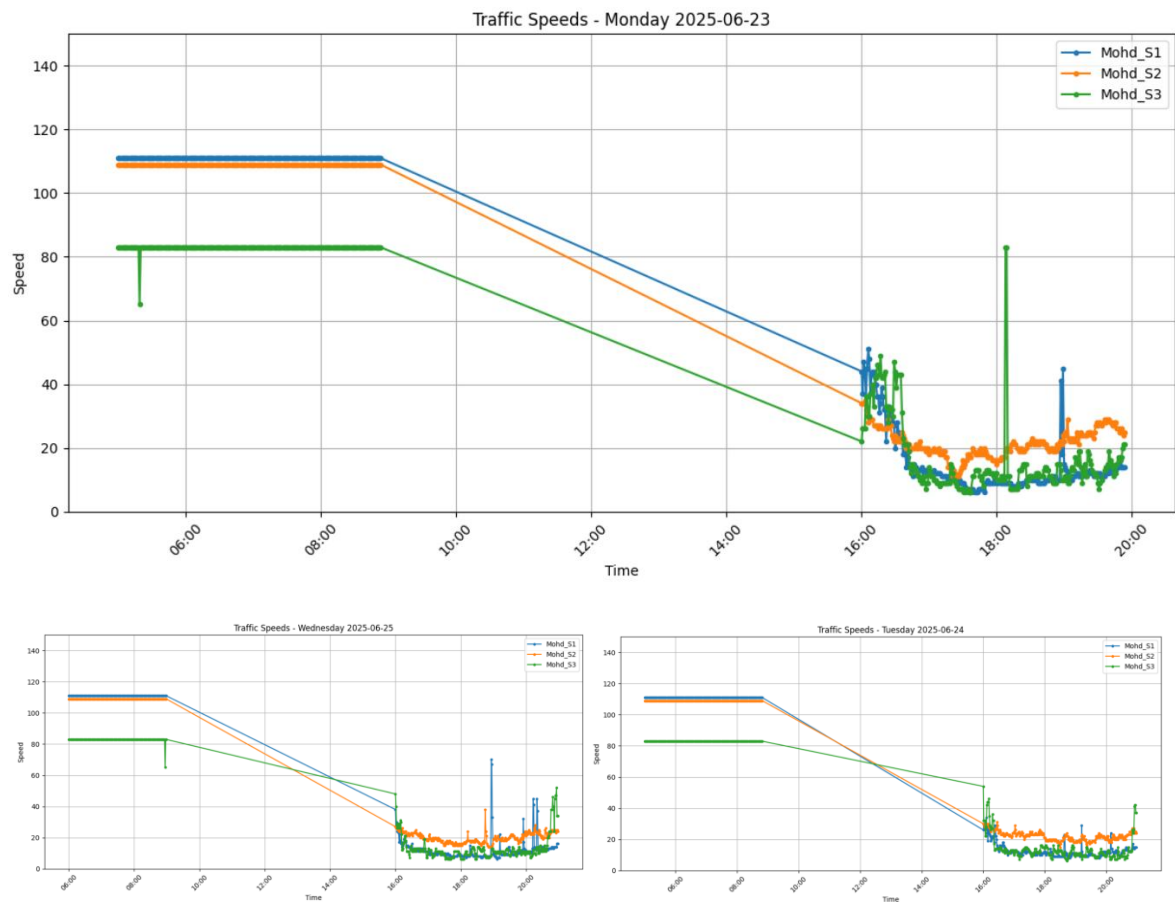


Fig. 109-112: Speed patterns of Deira segments Wed-Sat

As expected by the most congested stretches, the Dubai-Sharjah route E311 shows high congestion throughout early to late evenings.



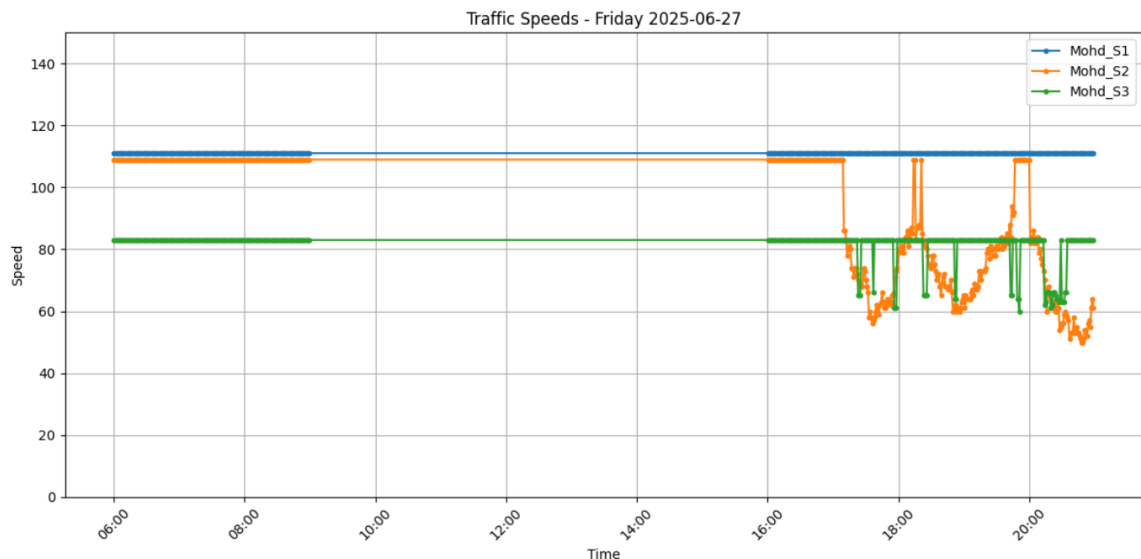


Fig. 113-116: Speed patterns of Muhaisnah segments

7.9 Issues & Debugging

5.2.1 Flat Learning Curve & NaN values during training: The following options were tried in order to find & resolve the issue -

- Data Issues: Checked data going through the model at different stages to identify at which point the NaN values pop up. RL problems do not have large, labelled datasets to preprocess, only an initial state. So the input data here was pre-defined.
- Data Scaling/Normalisation: Normalisation was applied, just like the main paper. This prevents values from becoming too large or small, reducing the chance of numerical instability.
- Learning Rate & Optimization: Tried different ones & checked for changes in behaviour. A learning rate that is too high can lead to exploding gradients, causing NaN values. Switching from SGD to Adam had helped solve this problem earlier when I had worked on a smaller project with NNs. So to begin with, I chose Adam as the optimizer, along with Relu as the activation function in inner layers. But due to this error, I tried SGD & leaky ReLu & ELU.
- Gradient Clipping: Clipped inputs & outputs at every step of processing within network functions. It basically limits values to the given range. Also helps with exploding gradients.
- Network Architecture and Operations: Rechecked all steps I wrote.
- Scaling Mismatch: Rechecked.
- Regularization: Not considered for this case since no. of layers & neurons were limited. This is usually done in larger networks (e.g., L1/L2 regularization, dropout) to prevent overfitting & other issues [23][24].

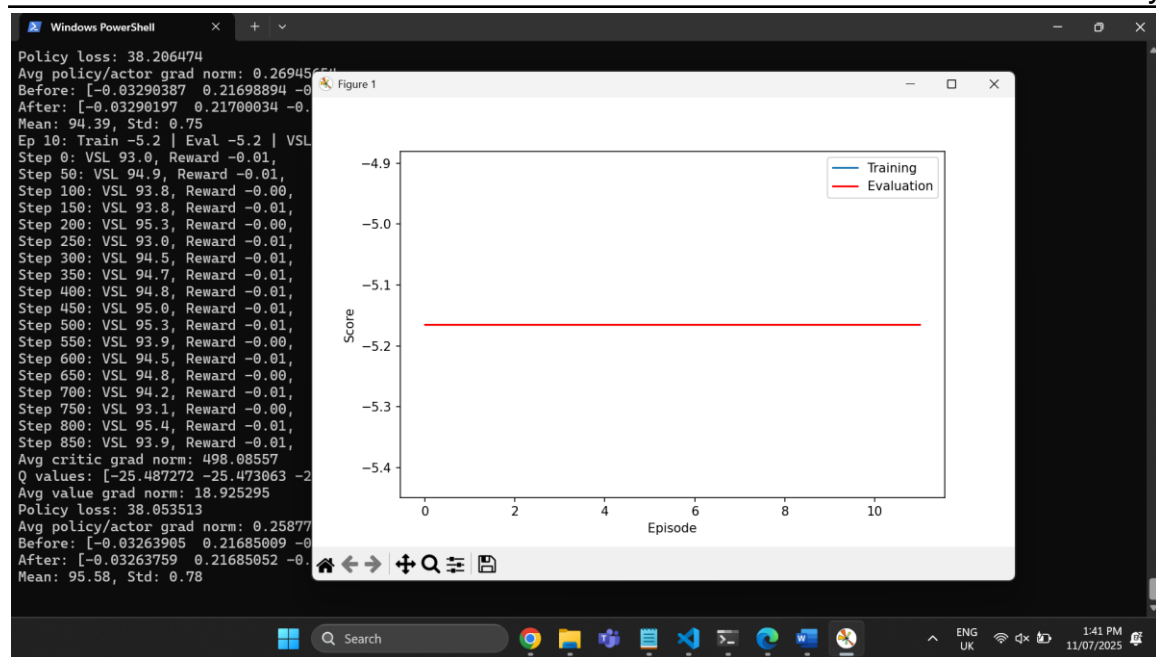


Fig. 117: Flat learning curve

The main solution was gradient clipping. Once all values like inputs, calculation & outputs were checked & restricted within a standard range, it resulted in usable, expected values within a reasonable range.

Unstable learning: Earlier, every time it was run, the output would show different behaviours – some times correct, sometimes more negative, sometimes flat. This was due to a single critic, leading to instability. After rechecking network definitions & adding 2 critics, model was much more interpretable.

5.2.2 Simulation not reacting to VSL: Tested in isolation with hard-coded limits. Rechecked example scripts from simulation framework repo, tried a few changes since I doubted if limits is being correctly passed for the model to apply it & show results. Error was found in the segment definition, where a separate subclass specifically for VSL application was supposed to be defined, but I was passing the limit value to a segment that was generic. Upon some fixes, simulation showed reactions, that also made sense.

5.2.3 Tile conversion misalignment: TomTom had straightforward functions to convert latitude/longitude to X/Y tile coordinates for a given zoom level, following the standard Web Mercator projection where each tile is conceptually 256x256 pixels. However, Azure Maps' vector tiles were also influenced by an 'extent' value, which we found to be 4096 in the PBF data. This extent value didn't change the display dimensions of the map tile (which remain 256x256 pixels on the screen). Instead, it defined the internal resolution of the coordinate system *within* the PBF tile. So, while the tile covers a 256x256 pixel area on the map, the coordinates for features *inside* the PBF were represented on a more granular 4096x4096 grid for enhanced precision. Therefore, when we extracted X,Y coordinates from the Azure PBF, they needed to be scaled down from this 4096-unit internal grid to the standard 256-pixel tile grid first. Only after this scaling (and then adding the global pixel offset of the tile) could these coordinates be correctly converted back to latitude and longitude using the standard Web Mercator inverse projection.

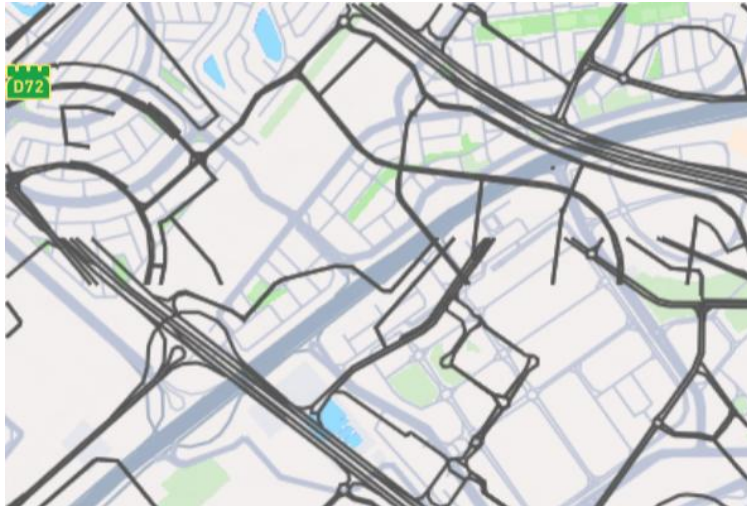


Fig. 118: Misaligned outputs Part 1

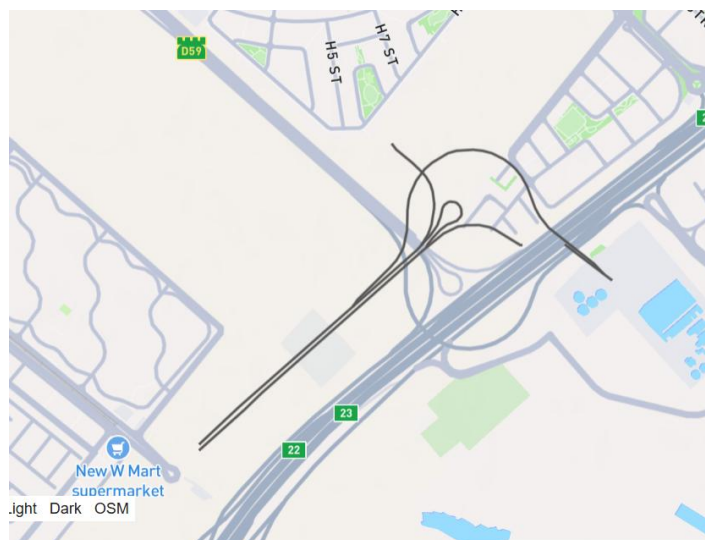


Fig. 119: Misaligned outputs Part 2

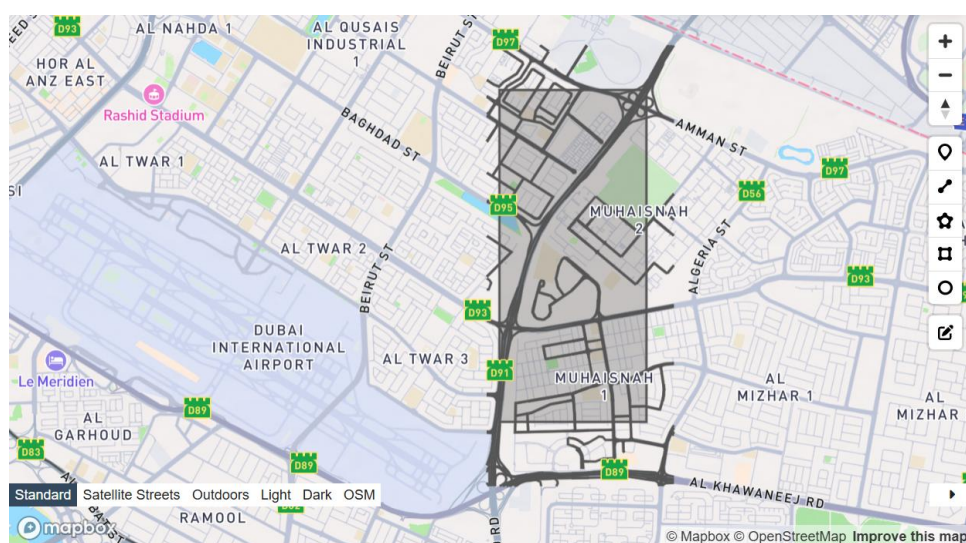


Fig. 120: Aligned outputs

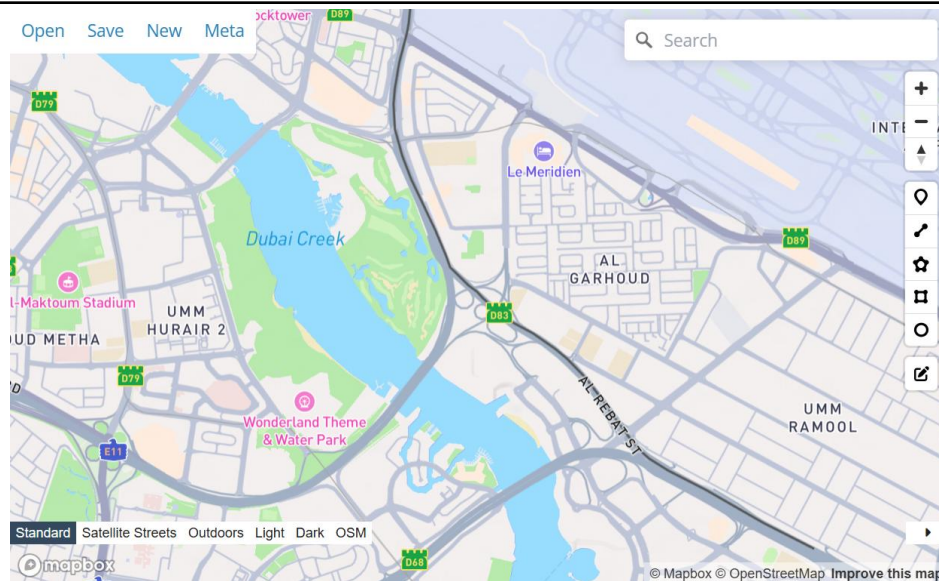


Fig. 121: Aligned segment

Contribution

This project presents a novel application of reinforcement learning by adapting the Soft Actor-Critic (SAC) algorithm for macroscopic Variable Speed Limit (VSL) control, marking one of the first such implementations outside of microscopic, CAV-focused studies. A realistic traffic simulation was developed using the MetaNet framework, modeling actual road stretches in the UAE with segment-level fidelity. The simulation environment was enhanced to support real-world data integration, including speed profiles from Dubai roads, and control over multiple VSL segments. In addition to technical development, the project includes detailed documentation of the reinforcement learning system architecture, training flow, state design, and safety-aware extensions (WCSAC). These efforts contribute not only a working macroscopic RL-VSL prototype but also a practical blueprint for future smart mobility research using real data and reinforcement learning.

Limitations & Other Considerations

A study of VSL results at different points of a road done in [3] is that it worsens congestion when placed right after an on-ramp. The solution to handling this would be to limit the number of vehicles leaving the ramp until road capacity is back to normal, a technique known as ‘ramp metering’. Failing to consider the possibility of phantom jams occurring, leads to too many cars entering the circuit [6].

Segment lengths were not consistent & fully intentional for the data shown in this study, they came directly from the api. Nor could there be a small overlap set between segments for smoother transitions. Uniform data would benefit the model & lead to more reliable performance.

Even the modelling was improvised to some extent to be able to run the simulations within the available framework. Information about the ramps were not available, not were the densities & some other parameters. They were often estimated.

There have also been several discussions on METANET, with improvements suggested to more accurately model traffic, but this project did not dive into modifying the only available public implementation library.

The effectiveness of VSL is highly dependent on drivers’ adherence to them [2]. UAE has been constantly advancing their government services, as well as their enforcement mechanisms. While cameras used to depend on manual checks & radar technologies, there are now mobile cameras on police cars, and cameras that observe speeds over time & average them. They’re using LIDAR sensors & inductive loops on roads, and fines are instantly applied and sent to violators via SMS. AI-powered analyses detect vehicle details & other safety aspects like presence of safety belts, phone usage, etc. These initiatives have significantly contributed to safer roads and better compliance [7]. Along with strict regulations and heavy fines, the UAE

enforces a black points system, which can result in the confiscation of a driver's license, to promote compliance with traffic regulations [8]. The number of traffic deaths were reduced by 66% over the last decade through these proactive measures. The highways of Dubai also have Variable Message Signs alerting drivers of nearby traffic accidents and unstable weather conditions. With proven strong compliance, infrastructure and most importantly a thirst for innovative solutions, VSL is highly likely to positively impact this region.

8 Future Prospects

This section covers improvements that can be added to this project, the next steps to get it implemented in real roadways, and other observations to keep in mind during the process.

8.8 Potential Enhancements

First would be continuous optimisation, and overcoming the limitations being currently faced, as mentioned above.

Multi-agent soft actor-critic (MA-SAC) algorithm can be implemented for traffic signal control, where agents are deployed at each intersection to optimize traffic flow collectively.

The UAE utilizes a 'SCOOT' system, which is a centralized traffic management system equipped with sensors to monitor vehicle volumes. These sensors measure traffic flow, vehicle counts, and the average speeds. This data could be integrated into this VSL project to make decisions based on more accurate real-time traffic data without the involvement of a third party [8].

[17] combines VSL with ramp metering, another promising idea to combat congestion. The focus of this paper was only VSL, but the system may perform even better if such ideas with similar data requirements were integrated.

8.9 Future Plans

This project will be presented to the road & Transport Authority (RTA), aiming to be implemented on the roads of UAE, and eventually in other parts of the world.

These are realistic stages planned for deploying this VSL system:

Phase 1: Data Collection & Feasibility Study

- Collect real-time data: Speeds, flow, incidents, weather, congestion hotspots. More extensively, attempt to eliminate this study's current limitations.
- Feasibility study: Evaluate where VSL could help - bottlenecks, merge areas, near exits/entries.

Phase 2: Offline Model Development & Testing

- Simulate traffic environment (what you did with MetaNet) using real network topologies, increase current accuracy in modelling.
- Train RL agents offline using historical data and simulated feedback.
- Extensively compare with other similar models.
- Validate safety with scenario testing (e.g., sudden slowdowns, congestion waves).

Phase 3: Test Deployment (Closed Loop)

- Choose a test highway stretch with dynamic signage and monitoring systems.
- Deploy trained RL model in "shadow mode" - observe what it would do without letting it control.
- Log decisions and compare them with actual outcomes.

Phase 4: Real Discussion

- Propose/Pitch to authorities
- Discuss possibility of implementation & secure a contract

- Obtain necessary permissions to officially implement it
- Iterate over collected feedback

Phase 5: Real-Time Control (Open Loop)

- Integrate RL agent with dynamic VMS (Variable Message Signs).
- Run real-time decision loop every minute or so using latest traffic state.
- Continuously log data, retrain weekly/monthly as needed.

Phase 6: Scaling & Continuous Learning

- Expand to more segments/roads.
- Include adaptive retraining, incident handling, weather-awareness, and safety guarantees (e.g., WCSAC)
- Integrate with connected vehicles or AVs for personalized speed guidance.

Business Model Ideas

Here are ways this project can turn into a real-world product or offering:

1. SaaS for Smart Cities

Offer a cloud-based VSL optimization platform that municipalities can plug their road data into.

Revenue: Subscription/license fees from city transport departments.

2. B2B Licensing to Highway Ops / ITS Firms

Integrate the RL control backend into ITS (Intelligent Transport Systems) products sold to cities/highway authorities.

Revenue: Per-road license, consulting fees, deployment contracts.

3. Platform for AV/Connected Cars

Build an RL-based speed advisory system for autonomous vehicles (AVs) or connected vehicles (CVs).

Revenue: Integration licensing, OEM partnerships.

4. Data + AI Consulting

Offer VSL simulation, optimization, and RL consulting to transportation authorities.

Revenue: One-time or ongoing project-based payments.

Driver Opinions Questionnaire

Purpose: As part of this project, which focuses on the development of an intelligent Variable Speed Limit (VSL) system using reinforcement learning, it was essential to incorporate perspectives from real road users. The goal of the questionnaire was to gather feedback regarding:

- Awareness and understanding of VSL systems,
- Driver behavior and compliance willingness,
- Perceived risks or concerns,
- Trust in automation and traffic technologies,
- Relevance of demographic or driving patterns.

By gathering this data, we ensured that the technical system is informed by human factors, thereby improving its practical applicability and anticipating behavioural constraints in real-world deployment.

Topics Covered in the Questionnaire:

- Awareness and Familiarity: Whether drivers understand what VSL is.
- Perceived Benefits or Drawbacks: Opinions on whether VSL could reduce congestion or cause confusion.
- Willingness to Comply: How likely drivers are to follow VSL in practice.
- Preferences for VSL Presentation: Whether they prefer fixed limits or context-aware dynamic limits.

This aligns with human-in-the-loop design principles often cited in transportation system design literature.

No Personally Identifiable Information was collected during this process. Google Forms was used, which is a reliable platform. All participants were informed that their responses will be evaluated for academic purposes only, and their consent was taken. No harm was caused, no sensitive topics were included.

The form has collected 24 responses so far, here are the results:

1. This confirms that the majority of respondents are active drivers, providing relevant insights.

Do you drive regularly?
24 responses

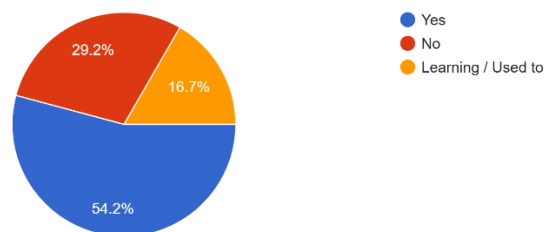


Fig. 122: Pie chart for question about regular driving

2. This shows that the target areas for VSL systems are familiar territory for many, giving an idea of the impact it will have.

How often do you drive on highways (e.g. Sheikh Zayed Road)?
24 responses

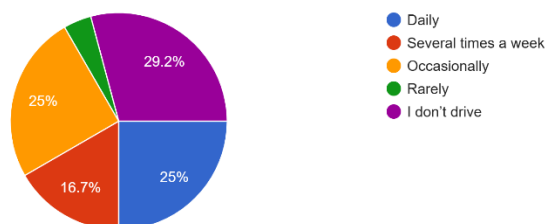


Fig. 123: Pie chart for question about driving on highways

3.

Which of the following best describes your usual driving location?
23 responses

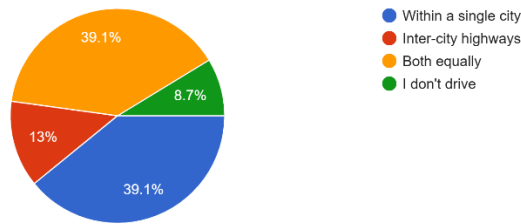


Fig. 124: Pie chart for question about inter-city driving

4.

Are you familiar with the concept of Variable Speed Limits (VSL)?
24 responses

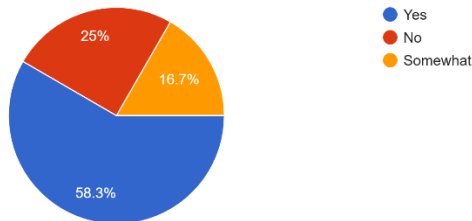


Fig. 125: Pie chart for question about VSL familiarity

5. Encouraging, but still shows the need to build credibility and transparency.

Would you be in favor of such a system being implemented in your city?
24 responses

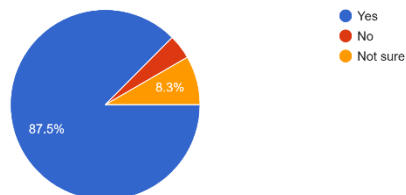


Fig. 126: Pie chart for question about supporting VSL

6.

What benefits do you believe VSL could offer? (Select all that apply)
23 responses

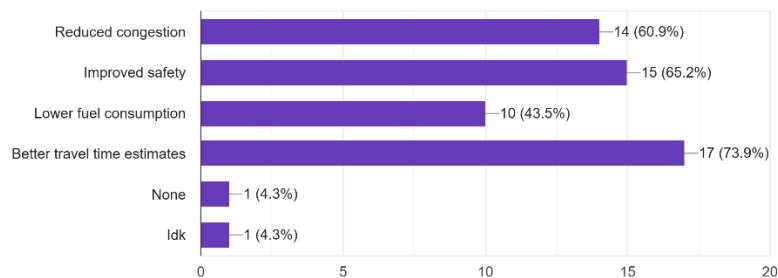


Fig. 127: Bar graph for question about benefits of VSL

7. The most common concern was fines, followed by confusion, non-compliance by others, and some distrust. These insights are valuable for simulation modeling: not all drivers will comply with VSLs, and this can be modeled as partial compliance or noisy response in RL.

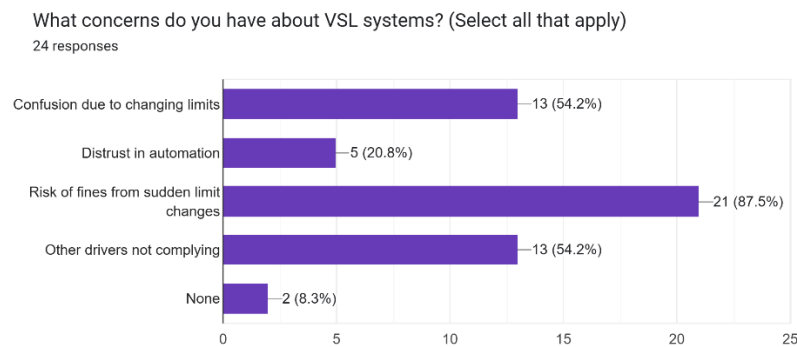


Fig. 128: Bar graph for question about concerns

8.

Do you think VSL would be effective in reducing traffic jams during peak hours?
24 responses

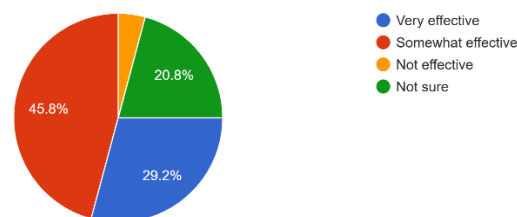


Fig. 129: Pie chart for question about VSL affecting traffic

9. Confirms the basis for enforcing VSLs via traffic law, if implemented.

Would you personally follow speed limits shown on an electronic VSL sign, if legally enforced?
24 responses

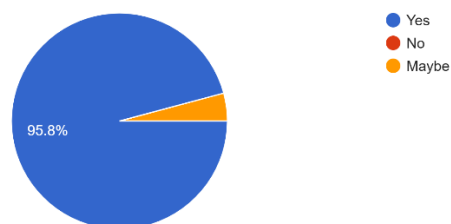


Fig. 130: Pie chart for question about compliance Part 1

10. These are promising figures for real-world deployment readiness.

Would you follow them even if not enforced by fines?
24 responses

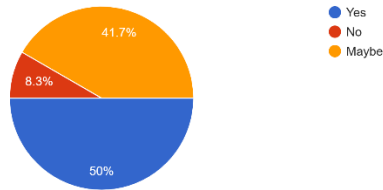


Fig. 131: Pie chart for question about compliance Part 2

11.

If field-tests demonstrate that VSL reduces accident risk and fuel use, would it change your willingness to comply?
24 responses

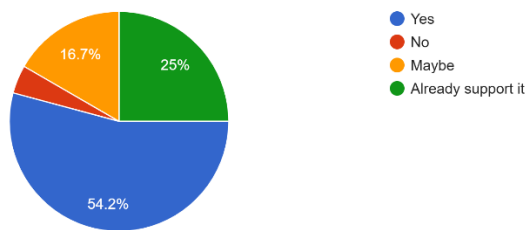


Fig. 132: Pie chart for question about demonstrations

12.

How familiar are you with AI-based technologies (e.g., self-driving cars, traffic prediction, automated assistants)?
24 responses

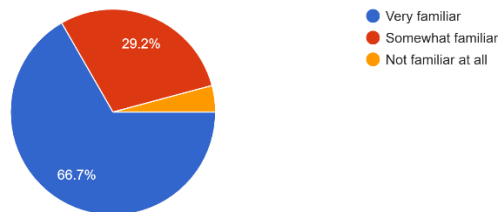


Fig. 133: Pie chart for question about familiarity with AI

13.

What is your level of trust in AI systems used by institutions (e.g., governments, transport authorities)?
24 responses

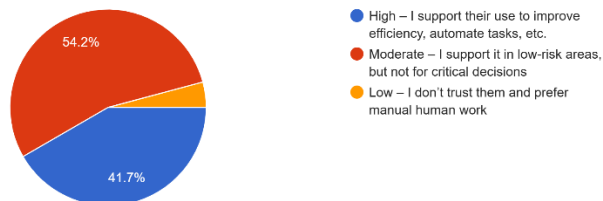


Fig. 134: Pie chart for question about trust

14.

Experience with deployed AI systems (ChatGPT, Tesla, etc.):
24 responses

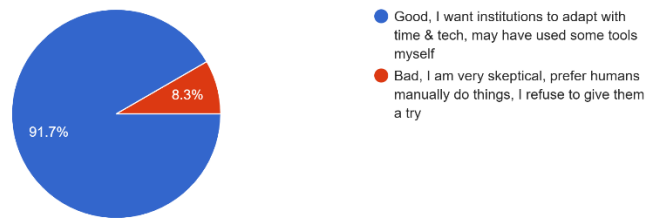


Fig. 135: Pie chart for question about experience with AI

15. This may explain the openness to advanced traffic solutions.

Age Group:
24 responses

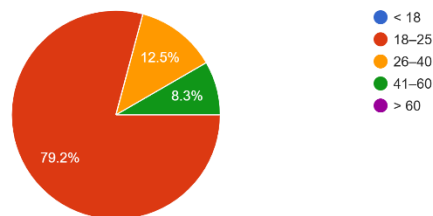


Fig. 136: Pie chart for question about age group

16.

What best describes your occupation sector?
24 responses

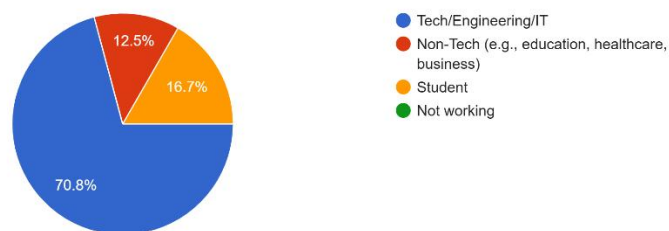


Fig. 137: Pie chart for question about occupation

17. Comments:

- Innovative and life changing technology
- The rise of AVs and connected AVs presents new opportunities for integrating VSLs with intelligent traffic systems for better flow management.
- Ensure clear and consistent Signal
- Would be recommended to alert drivers about speed limit changes beforehand, so they get the required time to safely change their vehicle's speed limit, without sudden actions that could instead alter the VSL's safety aspect (if they are alerted exactly on the point where the speed limit changes, they'll have to comply with the same in very little time, which could in turn lead to increased accidents/crashes).

The questionnaire responses show a **high level of awareness, support, and willingness to adopt VSL systems**, especially among young and tech-savvy users. However, the feedback also highlights key

implementation challenges such as communication clarity and driver reaction time. These insights were useful in shaping parts of the simulation logic (e.g., incident modeling, speed change predictions, partial compliance) and can help guide future real-world deployment strategies.

CONCLUSION

The study demonstrates the potential of Variable Speed Limits (VSL) implemented using the Soft Actor-Critic (SAC) reinforcement learning algorithm to optimize traffic flow on long highway networks. By leveraging real traffic data from Dubai and simulating with the METANET traffic model, this project addresses key challenges in traffic congestion, safety, and environmental impact. The integration of a safety metric further enhances the system's practical applicability.

Future research directions include real-world implementation and testing, exploring lane-wise VSL with solutions to mitigate lane-changing issues, and extending the approach to mixed traffic scenarios, including autonomous and manual vehicles. The study paves the way for scalable and efficient Intelligent Traffic Solutions in urban and inter-city networks.

REFERENCES

- [1] V. Knoop, *Traffic Flow theory: an Introduction with Exercises*. TU Delft Open, 2021. A.J. Steven and C. Williams, *43rd ISSC Conference*, Florida, USA, May 2000.
- [2] B. Khondaker and L. Kattan, "Variable speed limit: an overview," *Transportation Letters*, vol. 7, no. 5, pp. 264–278, Jan. 2015.
- [3] B. S. Kerner, "Study of Freeway Speed Limit Control Based on Three-Phase Traffic Theory," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 1999, no. 1, pp. 30–39, Jan. 2007.
- [4] "How Dubai-Sharjah commuters struggle with exhaustion, work-life balance due to peak traffic," *Khaleej Times*, 2025.
- [5] M. Papageorgiou and A. Kotsialos, "Freeway ramp metering: an overview," *IEEE Transactions on Intelligent Transportation Systems*, vol. 3, no. 4, pp. 271–281, Dec. 2002.
- [6] K. Goldmann and G. Sieg, "Economic implications of phantom traffic jams: evidence from traffic experiments," *Transportation Letters*, vol. 12, no. 6, pp. 386–390, Apr. 2019.
- [7] "How Do Speed Cameras Work? A Complete Guide for UAE Drivers," *Noorhan Auto Spare Parts*, Jan. 31, 2025.
- [8] "Road safety | The Official Portal of the UAE Government," 2024.
- [9] E. Walraven, M. T. J. Spaan, and B. Bakker, "Traffic flow optimization: A reinforcement learning approach," *Engineering Applications of Artificial Intelligence*, vol. 52, pp. 203–212, May 2016.
- [10] M. Naeem, S. T. H. Rizvi, and A. Coronato, "A Gentle Introduction to Reinforcement Learning and its Application in Different Fields," *IEEE Access*, vol. 8, pp. 209320–209344, 2020.
- [11] Pavel Ševera, "The Ripple Effect of Traffic Congestion," *Goodvisionlive.com*, Mar. 07, 2023.
- [12] "Traffic Congestion," *Sustainable Development Goals - Resource Centre*, 2022.
- [13] H. Tabrez, "UAE's road infrastructure: All you need to know," *Gulf News*, 2021.
- [14] J. Lu, "Reinforcement Learning for Freeway Variable Speed Limit Control: A Mixed Traffic Flow Case Study," *Department of Civil and Environmental Engineering, University of California, Berkeley*, Berkeley, CA 94720, USA, 2022.
- [15] T. Haarnoja *et al.*, "Soft Actor-Critic Algorithms and Applications," 2018.
- [16] Q. Yang, T. D. Simão, S. H. Tindemans, and J. Spaan, "WCSAC: Worst-Case Soft Actor Critic for Safety-Constrained Reinforcement Learning," *ResearchGate*, May 18, 2021.
- [17] N. X.-Y. Lu, T. Z. Qiu, P. Varaiya, R. Horowitz, and S. E. Shladover, "Combining Variable Speed Limits with Ramp Metering for freeway traffic control," *American Control Conference*, pp. 2266–2271, Jun. 2010.
- [18] G. Hollinger, R. Simmons, T. Smith, T. Cassandra, M. Littman, and L. Kaelbling, "Partially Observable Markov Decision Processes (POMDPs) Outline for POMDP Lecture," 2007.
- [19] Y. Zhang, Isik Ilber Sirmatel, Faisal Alasiri, P. A. Ioannou, and Nikolas Geroliminis, "Comparison of Feedback Linearization and Model Predictive Techniques for Variable Speed Limit Control," *21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 3000–3005, Nov. 2018.
- [20] TomTom, "Developer Tools".
- [21] S. Zhang and R. S. Sutton, "A Deeper Look at Experience Replay," *NIPS 2017 Deep Reinforcement Learning Symposium*, 2017.

- [22] Azure, "Azure Maps Traffic Service Coverage".
- [23] Emmanuella Budu (2025). "Common Causes of NaNs During Training".
- [24] Josh Tobin (2021). "Lecture 7 – Troubleshooting Neural Networks", *UC Berkley Spring 2021*.
- [25] Ruici Zhang, Shoulong Xu, Rongjie Yu, Jiqing Yu. "Enhancing multi-scenario applicability of freeway variable speed limit control strategies using continual learning", 2024.
- [26] P. Allaby, B. Hellinga, and M. Bullock, "Variable Speed Limits: Safety and Operational Impacts of a Candidate Control Strategy for an Urban Freeway".
- [27] Yuankai Wu, Huachun Tan, Lingqiao Qin, Bin Ran (2020). "Differential variable speed limits control for freeway recurrent bottlenecks via deep actor-critic algorithm", *Transportation Research Part C: Emerging Technologies, Volume 117, 2020*.
- [28] Jingwen Yang, Ping Wang, Fatemeh Golpayegani, Shen Wang. "DVS-RG: Differential Variable Speed Limits Control using Deep Reinforcement Learning with Graph State Representation", *JOURNAL OF LATEX CLASS FILES, VOL. , NO. 1, 2024*.
- [29] Yuhang Zhang, Marcos Quinones-Grueiro, Zhiyao Zhang, Yanbing Wang, William Barbour, Gautam Biswas, Daniel Work, "MARVEL: Multi-Agent Reinforcement-Learning for Large-Scale Variable Speed Limits", 2024.
- [30] Opdahl, Markus, "Autonomous Driving in Simulated Environments with Soft Actor-Critic Reinforcement Learning", NTNU, 2022.
- [31] Raffaele Galliera, Alessandro Morelli, Roberto Fronteddu, Niranjani Suri. "MARLIN: Soft Actor-Critic based Reinforcement Learning for Congestion Control in Real Networks", *Florida Institute for Human & Machine Cognition (IHMC), Department of Intelligent Systems & Robotics - The University of West Florida (UWF)*.
- [32] IQAir, 2025.
- [33] Shambhavi Sinha, "Soft Actor-Critic Reinforcement Learning algorithm", 2024.
- [34] Tao Li, Wei Cui and Naxin Cui, "Soft Actor-Critic Algorithm-Based Energy Management Strategy for Plug-In Hybrid Electric Vehicle", *World Electr. Veh. J.* 2022.
- [35] Jiawei Kong, "Reinforcement Learning Tutorial: Soft Actor-Critic (SAC)", 2024.
- [36] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, Sergey Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor", 2018.
- [37] Nimisha Singh, "Tackling Underflow with Log Probability: Exploring Laplace Smoothing and Probability Distributions in Naïve Bayes", 2023.
- [38] Jason Brownlee, "A Gentle Introduction to Information Entropy", 2020.
- [39] Aeryan, "Does SAC perform better than PPO in sample-expensive tasks with discrete action spaces?", 2022.
- [40] Magdy Fares, huỳnhpham37, "Comparative Analysis of Reinforcement Learning Algorithms in Multi-Agent Soccer Environment", 2024.
- [41] Haarnoja et al., "Soft Actor-Critic Algorithms and Applications", 2018.
- [42] Zhang et al., "Efficient Off-Policy RL via Compressed Replay", *NeurIPS*, 2022.
- [43] Papoudakis et al., "Benchmarking Multi-Agent SAC in SMAC", *AAMAS*, 2021.
- [44] Rusu et al., "SAC-ADR: Sim-to-Real Transfer with Stochastic Dynamics", *Science Robotics*, 2022.
- [45] Dalal et al., "SafeSAC: Real-World RL with Collision Avoidance", *RSS*, 2023.
- [46] Zhang et al., "Edge-SAC: Real-Time RL on Embedded Systems", *IEEE IoT Journal*, 2023.
- [47] Denis Yarats, Ilya Kostrikov, Rob Fergus, "Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels", *ICLR*, 2021.
- [48] Dong, Sheng; Fan, Feihu; Chen, Yinuo; Guo, Shangpeng; Liu, Jiayu, "Gait Planning, and Motion Control Methods for Quadruped Robots: Achieving High Environmental Adaptability: A Review", *Computer Modeling in Engineering & Sciences (CMES)*, 2025, Vol 143, Issue 1, p1, 2025.
- [49] Papageorgiou, M., et al., "Rule-Based Variable Speed Limits: A 20-Year Review", *Transportation Research Part C.*, 2022.
- [50] He, Q., et al., "SAC-VSL: Learning to Smooth Speed Transitions". *IEEE ITSC*, 2023.
- [51] Texas Department of Transportation, "Edge Computing Requirements for Adaptive Traffic Control", 2024.
- [52] apamichail, I., "METANET Calibration", *IEEE ITS*, 2019.
- [53] Caltrans, "I-880 Safety Report", 2022.
- [54] Lee & Park, "Edge Deployment of RL for Traffic Control". *IEEE IoT Journal*, 11(3), 45-52, 2024.
- [55] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, Sergey Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor", 2018.
- [56] OpenAI, "Soft Actor-Critic".

- [57] Berg, J., et al., "Driver Adaptation to Predictive Speed Controls", *Transportation Research Part F.*, 2022.
- [58] Wu, T., et al., "RL for Human-Centric VSL", *IEEE ITSC*, 2023.
- [59] NHTSA, "National Driver Survey on VSL Systems", DOT HS 813 456. 2023.
- [60] Loadview, "Types of Software Testing: Differences and Examples", 2023.
- [61] Pressman, Roger/Maxim, Bruce, "Software Engineering: A Practitioner's Approach", 2019.
- [62] Creswell, John W., Plano Clark, Vicki L, "Designing and Conducting Mixed Methods Research", 2018.
- [63] Giulia Del Serrone, Giuseppe Cantisani, Paolo Peluso, "Speed data collection methods: a review",
- [64] *Transportation Research Procedia*, Volume 69, 2023, Pages 512-519, ISSN 2352-1465, 2023.
- [65] U.S. Department of Highway Administration, "Collecting/Analyzing Speed and Crash Data
- [66] "The need for speed management is generally determined by reviewing actual speeds and crash histories".
- [67] Zhou Z, Yang Z, Zhang Y, Huang Y, Chen H, Yu Z, "A comprehensive study of speed prediction in transportation system: From vehicle to traffic", *iScience*, 2022.
- [68] Nesa, M., Yoon, Y, "Speed prediction and nearby road impact analysis using machine learning and ensemble of explainable AI techniques", *Sci Rep* 14, 2024.
- [69] Zheng & Casari, "Feature Engineering for Machine Learning", 2018.
- [70] Smith, Zhang, "KNN for Short-Term Traffic Forecasting", *IEEE Trans. ITS*, 2022.
- [71] Yisheng Lv, Yanjie Duan, Wenwen Kang, Zhengxi Li, Fei-Yue Wang, "Traffic Flow Prediction With Big Data: A Deep Learning Approach", *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 2, pp. 865-873, 2015.
- [72] Yi Qi, Qun Zhao, Wenrui Qu, Mehdi Azimi, "Short-Term Intersection Traffic Flow
- [73] *Forecasting*", 2021.
- [74] Xiaolei Ma, Zhuang Dai, Zhengbing He, Jihui Ma, Yong Wang, Yunpeng Wang, "Learning Traffic as Images: A Deep Convolutional Neural Network for Large-Scale Transportation Network Speed Prediction", 2017.
- [75] Yuhang Zhang, Zhiyao Zhang, Junyi Ji, Marcos Quiñones-Grueiro, William Barbour, Derek Gloudemans, Gergely Zachár, Clay Weston, Gautam Biswas, Daniel B. Work, "Real-World Deployment and Assessment of a Multi-Agent Reinforcement Learning-Based Variable Speed Limit Control System", 2025.
- [76] National Operations Center of Intelligence, "Interstate 95 Variable Speed Limit System", 2023.
- [77] DriveSafe Online Staff, "Unlocking Traffic Efficiency: The Power of Variable Speed Limit (VSL) Systems", *Traffic School*, 2023.
- [78] Albania Nissan, "Evaluation of Variable Speed Limits: Empirical Evidence and Simulation Analysis of Stockholm's Motorway Control System", 2010.
- [79] Bruce Hellinga, Peter Allaby, Mara Bullock, "Variable Speed Limit Systems: Safety and Operational Impacts for Freeways Applications",
- [80] Mark Robinson, "Examples of Variable Speed Limit Applications", *Transportation Research Board Annual Meeting*.

16. Appendix

16.1 Blog Link - [Anushka Narsima's Portfolio](#)

16.2 Repository Link - [GitHub - Anushka-N12/variable-speed-limits: Final year project that optimizes traffic flow using VSL using Reinforcement Learning, specifically the SAC algorithm](#)

16.3 Ethics Assessment Form